

The Coding Interview Playbook

A complete DSA workbook with bonus chapters for coding rounds, system design, behavioral interviews, and offer negotiation

From your first practice problem to your final offer, this book helps you build the patterns, communication, and decision-making skills that strong interviewers actually measure.

“I will help you speak and teach you what to say.”

Exodus 4:12

Author: [interviewprep.cafe](https://www.linkedin.com/in/lamhotsiagian) - Lamhot Siagian

LinkedIn: <https://www.linkedin.com/in/lamhotsiagian>

April 4, 2026

Preface

This ebook started as a DSA workbook and grew into a broader interview playbook. That expansion matters because real interview loops rarely test only one thing. Companies care about whether you can solve algorithmic problems, explain your reasoning clearly, discuss design trade-offs, tell credible stories about your work, and evaluate an offer with good judgment.

The core of this book is still hands-on DSA practice. That part helps you build the pattern recognition and coding discipline needed for technical rounds. Around that core, this edition adds practical bonus chapters on how to crack the coding interview, system design basics for software engineers, behavioral and HR questions, and negotiation and offer evaluation.

Use this book in one of two ways. You can study it sequentially from front to back, which works well if you want a full interview roadmap. Or you can treat it like a field guide: use the DSA chapters for daily technical practice and revisit the bonus chapters before live interviews so your communication, judgment, and strategy feel as prepared as your code.

What this ebook covers

- DSA foundations and problem-solving patterns
- How to approach coding interviews with better structure and communication
- System design basics for software engineers
- Behavioral and HR interview preparation
- Negotiation and offer evaluation

Contents

Preface	i
How to use this ebook	viii
1 Introduction to DSA	1
1.1 What DSA really means	1
1.2 Why complexity matters	1
1.3 Big-O mental model	1
1.4 Python pattern: timing intuition	2
2 How to crack the coding interview?	3
2.1 What interviewers are really measuring	3
2.2 The myths that waste the most time	3
2.3 How to approach a problem you have never seen	4
2.4 A practical script for talking through the problem	4
2.5 Interview execution from start to finish	5
2.6 A STAR-style frame for technical interviews	5
2.7 What separates good from great	5
2.8 What to practice beyond LeetCode	6
I Bonus Chapters	7
3 System design basics for SWEs	8
3.1 Why this round shows up, especially at L4+	8
3.2 The six-step framework	8
3.3 1. Requirements	9
3.4 2. Scale	9
3.5 3. API	9
3.6 4. Data model	9
3.7 5. Components	9
3.8 6. Bottlenecks and trade-offs	10
3.9 Patterns that show up again and again	10
3.10 Three classic interview problems	10
3.11 What interviewers usually want to hear	10
3.12 How to keep learning	11

4	Behavioral and HR questions	12
4.1	Why this round matters more than people think	12
4.2	Build three to five core stories	12
4.3	STAR still works, but use it naturally	12
4.4	Questions that show up almost everywhere	13
4.5	What makes a strong behavioral answer	13
4.6	Common mistakes in behavioral rounds	13
4.7	Tell me about yourself	14
4.8	Questions to ask the interviewer	14
5	Negotiation and offer evaluation	15
5.1	What total compensation actually means	15
5.2	How to compare offers sensibly	15
5.3	When and how to negotiate	16
5.4	What you can negotiate	16
5.5	Use leverage carefully	16
5.6	Questions to ask before you sign	17
5.7	A final word on confidence	17
6	Arrays and Strings	18
6.1	Two Sum	20
6.2	Best Time to Buy and Sell Stock	21
6.3	Product of Array Except Self	23
6.4	Maximum Subarray	24
6.5	Next Permutation	25
6.6	Majority Element	26
6.7	Sort Colors	27
6.8	Search a 2D Matrix II	29
6.9	Longest Substring Without Repeating Characters	30
6.10	Minimum Window Substring	31
6.11	Longest Palindromic Substring	33
6.12	Longest Repeating Character Replacement	34
6.13	Valid Palindrome	35
6.14	Valid Anagram	36
6.15	Encode and Decode Strings	37
6.16	String to Integer (atoi)	39
7	Linked Lists	41
7.1	Reverse Linked List	42
7.2	Merge Two Sorted Lists	43
7.3	Copy List with Random Pointer	45
7.4	Linked List Cycle	47
7.5	Merge K Sorted Lists	48
7.6	Remove Nth Node From End of List	49
7.7	Reorder List	51
7.8	Reverse Nodes in k-Group	52
7.9	Intersection of Two Linked Lists	54
7.10	Reverse Linked List II	55

8	Stacks and Queues	57
8.1	Valid Parentheses	59
8.2	Daily Temperatures	60
8.3	Sliding Window Maximum	61
8.4	Min Stack	62
8.5	Largest Rectangle in Histogram	64
8.6	Trapping Rain Water	65
8.7	Evaluate Reverse Polish Notation	66
8.8	Implement Queue Using Stacks	68
8.9	Car Fleet	69
8.10	Valid Parentheses (Stack Focus)	71
9	Hash Maps and Sets	73
9.1	Group Anagrams	74
9.2	Top K Frequent Elements	75
9.3	Longest Consecutive Sequence	76
9.4	Valid Sudoku	78
9.5	Intersection of Two Arrays II	79
9.6	Insert Delete GetRandom O(1)	81
9.7	Group Anagrams (Hashing Focus)	82
9.8	Longest Consecutive Sequence (Set Focus)	83
9.9	Top K Frequent Elements (Hashing Focus)	85
10	Trees	87
10.1	Maximum Depth of Binary Tree	88
10.2	Validate Binary Search Tree	89
10.3	Binary Tree Level Order Traversal	91
10.4	Binary Tree Maximum Path Sum	92
10.5	Invert Binary Tree	94
10.6	Same Tree	95
10.7	Lowest Common Ancestor of a Binary Search Tree	96
10.8	Construct Binary Tree from Preorder and Inorder Traversal	98
10.9	Serialize and Deserialize Binary Tree	99
10.10	Diameter of Binary Tree	101
11	Heaps and Priority Queues	103
11.1	K Closest Points to Origin	104
11.2	Task Scheduler	105
11.3	Find Median from Data Stream	106
11.4	Kth Largest Element in a Stream	108
11.5	Last Stone Weight	109
11.6	Kth Largest Element in an Array	110
11.7	Design Twitter	111
11.8	Merge K Sorted Lists (Heap Focus)	113
12	Graphs	115
12.1	Number of Islands	116
12.2	Course Schedule	118
12.3	Alien Dictionary	119
12.4	Word Ladder	121

12.5	Clone Graph	123
12.6	Pacific Atlantic Water Flow	124
12.7	Number of Connected Components in an Undirected Graph	126
12.8	Rotting Oranges	128
12.9	Network Delay Time	129
13	Sorting and Searching	132
13.1	Find Minimum in Rotated Sorted Array	134
13.2	Koko Eating Bananas	135
13.3	Median of Two Sorted Arrays	137
13.4	Binary Search	138
13.5	Search in Rotated Sorted Array	139
13.6	Search a 2D Matrix	141
13.7	Time Based Key-Value Store	142
13.8	Find Peak Element	143
13.9	Kth Smallest Element in a Sorted Matrix	144
14	Two Pointers and Sliding Window	146
14.1	Two Sum II - Input Array Is Sorted	147
14.2	3Sum	148
14.3	Container With Most Water	150
14.4	Permutation in String	151
14.5	Minimum Size Subarray Sum	152
14.6	Find All Anagrams in a String	153
14.7	Trapping Rain Water (Two-Pointer Focus)	155
14.8	Minimum Window Substring (Sliding-Window Focus)	156
15	Recursion and Backtracking	158
15.1	Subsets	159
15.2	Permutations	160
15.3	Combination Sum	162
15.4	N-Queens	163
15.5	Word Search	165
15.6	Palindrome Partitioning	167
15.7	Letter Combinations of a Phone Number	168
15.8	Combination Sum II	170
16	Greedy Algorithms	172
16.1	Jump Game	173
16.2	Merge Intervals	174
16.3	Gas Station	175
16.4	Partition Labels	177
16.5	Jump Game II	178
16.6	Candy	179
16.7	Non-overlapping Intervals	180
16.8	Minimum Number of Arrows to Burst Balloons	182
16.9	Queue Reconstruction by Height	183

17 Dynamic Programming	185
17.1 Word Break	186
17.2 Decode Ways	187
17.3 Burst Balloons	188
17.4 Longest Increasing Subsequence	190
17.5 Climbing Stairs	191
17.6 House Robber	192
17.7 Coin Change	193
17.8 Longest Common Subsequence	194
17.9 Partition Equal Subset Sum	195
17.10 Maximum Product Subarray	197
18 Tries and Advanced Structures	199
18.1 Implement Trie	203
18.2 Design Add and Search Words Data Structure	205
18.3 Redundant Connection	206
18.4 Word Search II	208
18.5 Prefix and Suffix Search	210
18.6 Replace Words	211
18.7 Maximum XOR of Two Numbers in an Array	213
19 Bit Manipulation	215
19.1 Number of 1 Bits	216
19.2 Counting Bits	217
19.3 Missing Number	218
19.4 Reverse Bits	219
19.5 Sum of Two Integers	220
19.6 Single Number	221
19.7 Single Number III	222
20 Math and Numbers	224
20.1 Count Primes	225
20.2 Pow(x, n)	227
20.3 Multiply Strings	228
20.4 Reverse Integer	229
20.5 Plus One	230
20.6 Divide Two Integers	231
20.7 Factorial Trailing Zeroes	233
21 Common Patterns and Templates	235
21.1 Sliding window template	235
21.2 DFS template	235
21.3 Binary search on answer template	235
22 Common Beginner Mistakes	237
22.1 Edge cases	237
22.2 Off-by-one errors	237
22.3 Using the wrong structure	237
22.4 A calm debugging checklist	237

How to use this ebook

Suggested rhythm

Read the chapter overview first. Then try to solve the problem yourself before reading the code. Start with the brute-force idea on purpose. That gives you a clean baseline and makes the optimized version feel earned instead of magical.

What to say in an interview

A simple pattern works well: restate the problem, name the brute-force baseline, explain the bottleneck, then introduce the improved data structure or algorithm that removes that bottleneck. Interviewers usually care as much about that progression as they do about the final code.

Quick prep plans

- **1 week:** Focus on arrays, hash maps, trees, graphs, and one solid DP review.
- **2 weeks:** Add heaps, binary search, greedy, and backtracking.
- **1 month:** Work chapter by chapter, then revisit weak spots with timed practice.

Introduction to DSA

This chapter gives you the big picture. If DSA has ever felt abstract, the goal here is to make it feel practical: data structures organize information, and algorithms are the repeatable steps we use to do useful work with that information.

1.1 What DSA really means

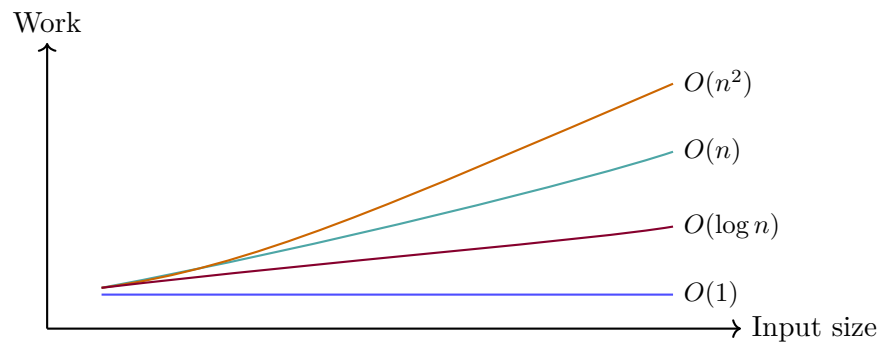
A data structure is the shape of your data. An algorithm is the process you run on top of that shape. A list of numbers, a tree of folders, and a network graph all store information differently, so the best algorithm depends on the structure you pick.

1.2 Why complexity matters

In interviews and in real systems, the right answer is not just correct - it should also scale. A solution that feels instant on 10 items can become painfully slow on 1,000,000 items.

1.3 Big-O mental model

Think of Big-O as a growth label. It does not tell you the exact number of milliseconds. It tells you how the work grows as the input gets bigger.



1.4 Python pattern: timing intuition

Here is a tiny function that counts basic operations so you can connect the idea of growth to actual loops.

```
def count_pairs(nums):  
    operations = 0  
    for i in range(len(nums)):  
        for j in range(i + 1, len(nums)):  
            operations += 1  
    return operations
```

How to crack the coding interview?

The best candidates do not look magical. They look organized. They understand the problem, make their reasoning visible, and turn that reasoning into clean code without spiraling.

2.1 What interviewers are really measuring

A coding round may look like a test of syntax, but that is not the full story. Interviewers usually grade a bundle of signals at the same time. They want to know whether you can understand the prompt, choose a sensible approach, write code with low error rate, and stay collaborative while you do it.

This is why someone can know the right data structure and still get a weak signal. The algorithm might be fine, but the execution feels messy. Maybe they skipped clarifying questions. Maybe they coded too early. Maybe they never tested edge cases. Maybe the interviewer had no idea why they made half their choices.

What most companies quietly score during a coding round

- Correctness. Did you solve the problem that was actually asked?
- Efficiency. Did you land on a solution that fits the expected level of the round?
- Communication. Can the interviewer follow your thinking without guessing?
- Code quality. Does your implementation look like something another engineer could trust?

Those four pillars matter more than raw speed alone. Fast typing is nice. Fast thinking is better. Clear thinking that becomes clean code is better still.

2.2 The myths that waste the most time

A lot of smart people slow themselves down with bad assumptions.

A few myths worth dropping early

- "I need to memorize hundreds of exact LeetCode questions." Not really. Patterns matter more than clones.
- "If I start with brute force, I look weak." Not if you label it as a baseline and improve from there.
- "Silence makes me look serious." In interviews, silence often looks like confusion.
- "If I have not seen this exact problem, I am done." Most unseen problems are still built from familiar parts.
- "The strongest candidate is the one who codes fastest." Usually it is the person who makes fewer bad decisions.

If you remember one thing, let it be this: interview prep is pattern learning plus execution. It is not a giant memorization contest.

2.3 How to approach a problem you have never seen

This is where people either calm down or fall apart.

When you see a new problem, do not ask yourself, "Have I seen this exact question before?" Ask, "What smaller ideas in here do I recognize?" That shift helps because most coding questions are combinations of familiar building blocks: hash maps, two pointers, sliding windows, stacks, trees, graphs, heaps, dynamic programming, binary search, or sorting.

A dependable workflow looks like this:

1. Restate the problem in your own words.
2. Ask clarifying questions about input, output, constraints, and edge cases.
3. Walk through a tiny example out loud.
4. Propose a simple baseline solution.
5. Identify what makes that baseline too slow or too messy.
6. Refine to a better approach and explain why it is better.
7. Implement carefully, then test before you declare victory.

That structure does two things at once. It helps you think, and it helps the interviewer trust the way you think.

2.4 A practical script for talking through the problem

A lot of candidates know the answer but do not sound like they know the answer. The fix is not to become overly polished. The fix is to narrate the right moments.

You can say things like:

- "Let me restate the problem to make sure I understand it correctly."
- "I see an easy baseline first, then I think there is a more efficient version."

- "The repeated work here is scanning the same values again and again, so I want to store information as I go."
- "Before I code, I want to confirm that duplicate values are allowed."
- "I am choosing this because it keeps lookup at $O(1)$ on average and simplifies the implementation." That is enough. You do not need a speech. You just need to make your decision points visible.

2.5 Interview execution from start to finish

This part gets ignored far too often. A good coding round is not just about the middle twenty minutes when you are typing. The entire round matters.

A common 45-minute rhythm looks like this:

A useful pace for a standard coding round

1. Minute 0–3: short introduction, friendly energy, no rambling.
2. Minute 3–8: clarify the problem, walk through examples, confirm edge cases.
3. Minute 8–14: explain the baseline and refine to the target approach.
4. Minute 14–30: code carefully while narrating meaningful decisions.
5. Minute 30–38: test with examples, check boundary cases, fix mistakes.
6. Minute 38–45: discuss complexity, trade-offs, and any follow-up variations.

That pacing is not rigid, but it helps prevent the classic failure mode: spending too long staring at the prompt and then rushing the last third of the round.

2.6 A STAR-style frame for technical interviews

People usually connect STAR with behavioral interviews, but the same logic helps in coding rounds too. You are still telling a story. It is just a smaller and more technical one.

Frame	How it shows up in a coding round
Situation	What is the prompt actually asking? What are the constraints and corner cases?
Task	What exactly must the algorithm return, optimize, or preserve?
Action	What solution are you choosing, and what trade-offs come with it?
Result	Does the code work on examples, edge cases, and complexity checks?

That frame is useful because it stops your explanation from becoming a stream of random thoughts. It gives your answer a beginning, middle, and end.

2.7 What separates good from great

The difference is usually not brilliance. It is steadiness.

Good candidates can solve the problem. Great candidates feel collaborative while they solve it. They notice ambiguity early. They check assumptions. They recover from mistakes without looking rattled. When they hit a wall, they stay structured instead of going silent.

That matters because real engineering work feels more like that than like speed chess. You are rarely paid to instantly blurt out the perfect answer. You are paid to make progress without creating chaos.

A simple rule for hard moments

If you get stuck, say what you know, what you are unsure about, and what you want to test next. Interviewers are often much more forgiving of temporary confusion than of invisible confusion.

2.8 What to practice beyond LeetCode

LeetCode is useful, but it does not cover everything. To get better at live interviews, practice these separately:

- explaining a solution out loud in plain English,
- writing code on a blank editor without autocomplete,
- testing by hand with your own sample inputs,
- recovering after a bug instead of mentally collapsing,
- finishing a round with calm trade-off discussion.

If your preparation only trains pattern recall, you will feel good while studying and shaky while interviewing. The missing piece is live execution.

Part I

Bonus Chapters

System design basics for SWEs

System design rounds are not asking whether you can invent the internet in forty minutes. They are asking whether you can structure a large problem, make sane trade-offs, and explain those trade-offs in a way another engineer would want to work with.

3.1 Why this round shows up, especially at L4+

Once companies expect you to own larger pieces of work, they need a different signal than coding alone. System design gives them that signal. It shows how you think when the problem is open ended, when there are multiple reasonable answers, and when trade-offs matter more than a single exact algorithm.

For many L4 and above roles, a design round is standard because the day-to-day job already includes questions like these: How should this service expose data? Where does the bottleneck live? What breaks first if traffic jumps? Should we cache this, shard it, queue it, denormalize it, or keep it simple?

That is what the round is really about. Not drawing the prettiest boxes. Not cramming in every distributed systems buzzword you know.

3.2 The six-step framework

A practical framework keeps you from wandering. One that works well is:

1. requirements,
2. scale,
3. API,
4. data model,
5. components,
6. bottlenecks.

Let us slow that down.

3.3 1. Requirements

Start by asking what you are building and what matters most. Functional requirements cover what the system must do. Non-functional requirements cover how well it should do it.

For example, in a URL shortener, the functional part is obvious: shorten long links and redirect users back to the original URL. The non-functional part matters just as much: fast redirects, high availability, reasonable durability, and maybe analytics if the product needs them.

If you skip requirements, the rest of the round becomes shaky. You can build an elegant design for the wrong product.

3.4 2. Scale

Before choosing fancy components, estimate the scale. How many users? How many reads and writes? Is read traffic much higher than write traffic? How much data do you keep, and for how long?

You do not need perfect math. Ballpark reasoning is enough. Saying "I expect read heavy traffic with occasional writes, so I am optimizing the read path first" already tells the interviewer you are designing with intent.

3.5 3. API

Now define the interface. What requests hit the system? What responses come back?

Even a simple sketch helps. For a rate limiter, you might have an internal API like:

- `allow(userId, resourceId)`
- return whether the request is allowed and how much quota remains

The API forces clarity. It makes hidden assumptions visible. You start noticing whether you need idempotency, pagination, authentication, or TTL behavior.

3.6 4. Data model

What data do you need to store? What fields matter? What gets indexed? What can expire?

This is where many interviews become more concrete. A news feed design might need users, follow relationships, posts, and feed entries. A URL shortener might need short code, original URL, creator, creation time, and optional expiration. A rate limiter might need counters, windows, and policy configuration.

Keep the model simple at first. The best designs usually start boring and only get more complex when the requirements demand it.

3.7 5. Components

Now outline the major building blocks. You might have clients, an API gateway or load balancer, stateless application servers, cache, database, queue, worker, and storage. The exact mix depends on the problem.

At this point you want to describe request flow clearly. For example: the client hits the API layer, the application checks cache first, falls back to the database if needed, writes updates asynchronously to a queue, and workers process background tasks. One clean story beats ten disconnected boxes.

3.8 6. Bottlenecks and trade-offs

This is where the round becomes interesting. What fails first? What becomes expensive at scale? What are you optimizing for, and what are you willing to accept?

Maybe you cache hot reads to protect the database. Maybe you shard by user ID to distribute write load. Maybe you use pub/sub so fan-out work does not block the request path. Maybe you accept eventual consistency in exchange for better throughput.

Trade-offs are the whole game. If your design sounds perfect, it usually sounds fake.

A strong design answer usually sounds like this

"Given the read heavy nature of the system, I would start with a simple stateless service behind a load balancer, a primary data store, and a cache in front of it. If write amplification becomes a problem later, I would introduce asynchronous processing rather than overbuilding from day one."

3.9 Patterns that show up again and again

You do not need to memorize every distributed systems pattern on earth. A short list gets reused constantly.

Caching. Useful when reads dominate and the same data is requested often. It improves latency and reduces database load, but you have to think about expiration and stale data.

Sharding. Useful when one database instance or partition cannot handle the load. It spreads data across multiple shards, but makes joins and rebalancing more painful.

Pub/sub. Useful when many downstream consumers need to react to an event without tightly coupling everything together. Great for notifications, activity streams, and background processing.

Queues. Useful when work can happen asynchronously. Queues protect the user-facing path from heavy background work.

Load balancing. Almost always part of the picture once you have multiple service instances.

Replication. Useful for availability and read scaling, but it introduces consistency questions.

3.10 Three classic interview problems

URL shortener. Good for talking about key generation, storage, redirect latency, caching, and analytics.

Rate limiter. Good for talking about counters, sliding windows, token buckets, low-latency decision paths, and distributed coordination.

News feed. Good for talking about fan-out models, ranking, timeline generation, caching, storage, and hot-user problems.

These show up so often because they exercise common instincts. They force you to talk about APIs, data models, read/write patterns, and where complexity belongs.

3.11 What interviewers usually want to hear

They want to hear a design that is reasonable, not theatrical. They want to see that you can ask clarifying questions, make sensible assumptions, and refine the design when new constraints appear.

A weak answer often jumps straight to exotic infrastructure. A strong answer usually starts with a simple version, names the likely bottlenecks, and then evolves the design only where needed.

A simple system design checklist

- Start with requirements before architecture.
- Estimate rough scale before picking storage or cache strategy.
- Explain request flow from client to storage and back.
- Name trade-offs directly. Do not hide them.
- Keep the first version simple, then evolve it.

3.12 How to keep learning

If system design still feels slippery, that is normal. It gets easier when you study a few patterns deeply instead of trying to collect every example online.

A practical path is to rotate through classic problems, watch how the same ideas repeat, and sketch your own answer before reading anyone else's. After that, compare: What assumptions did they make? What bottleneck did you miss? What complexity did they avoid that you added too early?

That kind of practice builds taste, and taste matters a lot in design rounds.

Behavioral and HR questions

A surprising number of engineers under-prepare for behavioral rounds because the questions sound simple. They are not simple. They are broad, subjective, and often decisive.

4.1 Why this round matters more than people think

Behavioral interviews are not filler. Companies use them to figure out what it is like to work with you when things are messy, political, stressful, or unclear.

The interviewer is often listening for things like ownership, judgment, honesty, self-awareness, conflict handling, and how you think about impact. A technically strong candidate can still lose momentum here if every story sounds vague, defensive, or disconnected from results.

The good news is that this round becomes much easier once you stop treating it like improv.

4.2 Build three to five core stories

You do not need a different story for every possible question. Usually three to five strong stories cover a surprising amount of ground.

Try to prepare stories around:

- a project with clear impact,
- a difficult bug or failure,
- a disagreement or conflict,
- a time you led without formal authority,
- a moment you learned something the hard way.

The trick is to choose stories with enough texture. If the story has no tension, no stakes, and no result, it will feel flat.

4.3 STAR still works, but use it naturally

STAR is useful because it keeps you from rambling, but robotic STAR answers are painful to listen to. The interviewer does not want a memorized template. They want a real story told clearly.

Part	What to include
Situation	A short setup. What was happening? Why did it matter?
Task	What were you responsible for? What was the challenge?
Action	What did you personally do? This should be the biggest part.
Result	What happened? What changed? What did you learn?

If your answer sounds too polished, simplify it. Real stories sound human because they include trade-offs, uncertainty, and lessons learned.

4.4 Questions that show up almost everywhere

Different companies phrase them differently, but the themes repeat.

- Tell me about yourself.
- Tell me about a time you disagreed with someone.
- Tell me about a failure.
- Tell me about a project you are proud of.
- Tell me about a time you took ownership.
- Tell me about a time you had to influence without authority.
- Tell me about a time you had conflicting priorities.
- Why this company? Why this role?

You do not need different personalities for each one. You do need stories that reveal judgment.

4.5 What makes a strong behavioral answer

A strong answer is specific. It sounds like one real event, not a summary of your whole career. It also makes your contribution clear. "We" is fine sometimes, but if the interviewer cannot tell what you did, the signal gets weak.

A strong answer also includes reflection. Not fake reflection. Real reflection. What would you do differently now? What did that moment teach you about how to work with people, scope projects, or escalate risk earlier?

A quick self-check before you tell a story

Can the listener answer these questions after your story ends: What was the problem? What did you do? Why did it matter? What happened in the end?

4.6 Common mistakes in behavioral rounds

The mistakes are predictable.

What hurts otherwise good candidates

- answers that are too long before they get to the point,
- answers with no measurable result or takeaway,
- stories where your role is unclear,
- blaming others too aggressively,
- saying the "right" value without backing it up with behavior.

This is why preparation helps so much. Once you tighten the story, the interview feels calmer.

4.7 Tell me about yourself

This question sounds casual, but it sets the tone for everything that follows. A good answer is short, relevant, and pointed toward the role.

One reliable structure is past, present, future.

Past: what kind of work shaped you.

Present: what you are doing now and what you are strongest at.

Future: why this role is the logical next step.

You do not need your entire life story. Aim for something that gives the interviewer a clean frame for the rest of the conversation.

4.8 Questions to ask the interviewer

This part matters too. Good questions show maturity and help you evaluate the company, not just impress it.

You can ask things like:

- What does strong performance look like in the first six months?
- What kinds of problems is the team spending the most time on right now?
- Where do new hires usually struggle at first?
- How does the team make technical decisions when people disagree?

Those questions are useful because they give you signal back. Interviews are a two-way filter, even when it does not always feel like it.

Negotiation and offer evaluation

Getting the offer is not the end of the process. It is the point where a lot of candidates finally relax, and then accidentally leave money, leverage, or useful information on the table.

5.1 What total compensation actually means

A lot of candidates fixate on base salary because it is the easiest number to compare. That is only one part of the package.

Total compensation often includes:

- base salary,
- annual or performance bonus,
- equity,
- signing bonus,
- refreshers or future stock grants,
- benefits that may matter more than people admit.

The right offer is not always the one with the biggest base. Sometimes a company with a slightly lower base makes up for it with stronger equity or a better first-year package.

5.2 How to compare offers sensibly

Do not compare offers with only one number in your head. Put them side by side.

Category	What to compare
Base salary	Fixed cash each year
Bonus	Target percentage, payout history, and guarantee details
Equity	Grant size, vesting schedule, liquidity, and refresh potential
Signing bonus	First-year cash, clawback terms, and repayment language
Role fit	Scope, growth, manager quality, and learning potential
Practical details	Location, remote policy, immigration support, and work-life reality

This matters because a slightly smaller number can still be the stronger career move if the team, growth path, and equity upside are better.

5.3 When and how to negotiate

Most candidates know they should negotiate, but they wait until the moment feels perfect. It rarely does. The cleaner rule is simpler: if you have an offer you would seriously consider, there is usually room to ask thoughtful questions and make a respectful counter.

Good negotiation is calm and specific. It is not aggressive theater. You do not need to "win" the conversation. You need to make a credible case.

A simple script looks like this:

A negotiation script that sounds professional

"Thank you again. I am excited about the role and the team. After reviewing the package, I wanted to ask whether there is flexibility on the base or equity portion. Based on my background and the other opportunities I am weighing, I was hoping we could explore something closer to [target]."

That is direct without sounding combative.

5.4 What you can negotiate

People often assume only base salary is negotiable. Not true. Depending on the company, you may be able to negotiate:

- base salary,
- equity,
- signing bonus,
- start date,
- level or title in some cases,
- relocation or remote details.

Different companies move different parts of the package more easily. Large public companies may have tighter bands on base but more flexibility on sign-on or equity. Smaller companies may lean harder on upside and scope.

5.5 Use leverage carefully

Competing offers help, but they are not the only source of leverage. Clear market data, a strong interview performance, a scarce skill set, and recruiter urgency all help too.

If you do have another offer, use it honestly. Do not bluff casually. Bluffing creates risk for very little upside.

A clean way to frame it is:

Using another offer without sounding dramatic

"I wanted to be transparent that I am in late-stage conversations elsewhere, and I also have another package that is stronger on compensation. This role is genuinely interesting to me, so I wanted to ask whether there is room to narrow the gap."

That keeps the tone respectful and gives the company something concrete to respond to.

5.6 Questions to ask before you sign

Once you feel the emotional relief of getting an offer, slow down. This is the moment to ask the boring but important questions.

A final offer review checklist

- What level am I being hired at, and how is performance evaluated?
- How does equity vest, and are refreshers common?
- Is the bonus target guaranteed for year one?
- Are there clawback terms on the sign-on bonus?
- Who will I report to, and what will the first six months likely look like?
- What risks or constraints should I understand before joining?

A thoughtful candidate asks these questions because a good offer is not just about cash. It is about what the role becomes after you join.

5.7 A final word on confidence

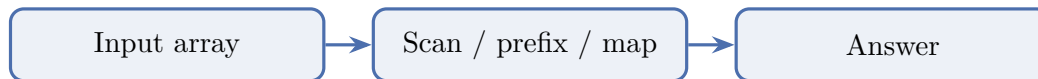
Negotiation often feels awkward because it happens right after a long period where you were being evaluated. The power dynamic starts to shift, and many people do not trust themselves in that moment.

Try to remember this: a respectful negotiation does not make you difficult. It makes you careful. Companies do this all the time. You are allowed to do it too.

Arrays and Strings

Chapter overview

Arrays give you constant-time indexed access, but they reward careful thinking about space. Most interview wins in this chapter come from spotting whether you need a raw scan, a prefix product, or a small helper map.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

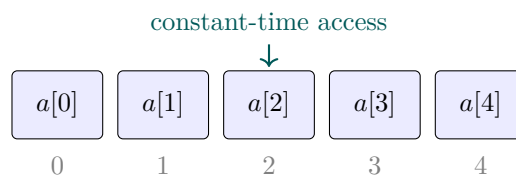
Arrays primer

This chapter opening folds in the handbook theory for arrays so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Arrays are often the first stop in DSA, and honestly that makes sense. They are simple, fast for indexing, and flexible enough to teach a lot of core problem-solving patterns.

Mental model

Elements live next to each other in memory, so reading an item by index is fast.



Traversal, insertion, and deletion

Traversal is cheap because you just walk left to right. Inserting or deleting in the middle is more expensive because everything after that point may need to shift.

Prefix sums

Prefix sums turn repeated range-sum work into subtraction.

```
def build_prefix(nums):
    prefix = [0]
    for num in nums:
        prefix.append(prefix[-1] + num)
    return prefix

def range_sum(prefix, left, right):
    return prefix[right + 1] - prefix[left]
```

Sliding window

A window keeps a contiguous segment in focus. Instead of recomputing from scratch, you update the running answer as the window moves.

```
def max_sum_subarray(nums, k):
    window_sum = sum(nums[:k])
    best = window_sum
    for right in range(k, len(nums)):
        window_sum += nums[right] - nums[right - k]
        best = max(best, window_sum)
    return best
```

Kadane's algorithm

Kadane's algorithm is the classic example of a local decision creating a global win: if your running sum becomes harmful, drop it and start fresh.

```
def kadane(nums):
    best = nums[0]
    current = nums[0]
    for num in nums[1:]:
        current = max(num, current + num)
        best = max(best, current)
    return best
```

Strings primer

This chapter opening folds in the handbook theory for strings so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Strings are just arrays of characters, but they come with their own flavor of problems: comparisons, patterns, frequencies, and careful pointer movement.

Palindrome check

Use two pointers: one at the left, one at the right. Move inward while the characters match.

```
def is_palindrome(text):
    left, right = 0, len(text) - 1
    while left < right:
        if text[left] != text[right]:
            return False
        left += 1
        right -= 1
    return True
```

Anagram check

Two strings are anagrams when they use the same characters with the same frequencies.

```
from collections import Counter

def are_anagrams(a, b):
    return Counter(a) == Counter(b)
```

Pattern matching overview

The naive approach checks every possible starting position. KMP avoids redoing work by using a prefix table. Rabin-Karp uses hashing. The right choice depends on the problem size and what trade-off you want.

Prefix-function pattern for KMP

You do not need to memorize KMP all at once. Start with the helper array. Once that clicks, the full search feels much more approachable.

```
def prefix_function(pattern):
    pi = [0] * len(pattern)
    j = 0
    for i in range(1, len(pattern)):
        while j > 0 and pattern[i] != pattern[j]:
            j = pi[j - 1]
        if pattern[i] == pattern[j]:
            j += 1
            pi[i] = j
    return pi
```

6.1 Two Sum

Problem statement

Given an array of integers `nums` and an integer `target`, return the indices of the two numbers that add up to `target`. You may assume exactly one answer exists and you may not reuse the same element twice.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [2, 7, 11, 15]`, `target = 9` -> `[0, 1]`
- `nums = [3, 2, 4]`, `target = 6` -> `[1, 2]`

Brute-force: Time $O(n^2)$, space $O(1)$ **Best solution:** Time $O(n)$, space $O(n)$ **Brute-force idea**

The simple move is to try every pair. It is slow, but it is honest, easy to explain, and often a good starting point in an interview.

Brute-force solution

```
def two_sum_bruteforce(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

Best solution

A hash map lets you ask, 'Have I already seen the complement I need?' That shrinks pair lookup from $O(n)$ to average $O(1)$.

Best solution

```
def two_sum_best(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        need = target - num
        if need in seen:
            return [seen[need], i]
        seen[num] = i
    return []
```

6.2 Best Time to Buy and Sell Stock

Problem statement

You are given prices where `prices[i]` is the stock price on day `i`. Pick one day to buy and a later day to sell. Return the maximum profit. If no profit is possible, return 0.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- prices = [7, 1, 5, 3, 6, 4] -> 5
- prices = [7, 6, 4, 3, 1] -> 0

Brute-force: Time $O(n^2)$, space $O(1)$

Best solution: Time $O(n)$, space $O(1)$

Brute-force idea

The brute-force version checks every possible buy and sell pair. It works, but it repeats a lot of comparison work.

Brute-force solution

```
def max_profit_bruteforce(prices):
    best = 0
    n = len(prices)
    for buy in range(n):
        for sell in range(buy + 1, n):
            best = max(best, prices[sell] - prices[buy])
    return best
```

Best solution

Track the lowest price seen so far and ask, at each step, what profit you would lock in by selling today.

Best solution

```
def max_profit_best(prices):
    min_price = float("inf")
    best = 0
    for price in prices:
        min_price = min(min_price, price)
        best = max(best, price - min_price)
    return best
```

6.3 Product of Array Except Self

Problem statement

Return an array answer where $\text{answer}[i]$ is the product of all elements of `nums` except `nums[i]`. Do not use division. The optimized solution should run in $O(n)$ time.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1, 2, 3, 4] -> [24, 12, 8, 6]`
- `nums = [-1, 1, 0, -3, 3] -> [0, 0, 9, 0, 0]`

Brute-force: Time $O(n^2)$, space $O(1)$ excluding output **Best solution:** Time $O(n)$, space $O(1)$ extra excluding output

Brute-force idea

For every index, multiply every other value. It is straightforward but does too much repeated multiplication.

Brute-force solution

```
def product_except_self_bruteforce(nums):
    result = []
    for i in range(len(nums)):
        product = 1
        for j in range(len(nums)):
            if i != j:
                product *= nums[j]
        result.append(product)
    return result
```

Best solution

Build prefix products from the left and suffix products from the right. Each answer becomes left product times right product.

Best solution

```
def product_except_self_best(nums):
    n = len(nums)
    result = [1] * n

    prefix = 1
    for i in range(n):
```

```

        result[i] = prefix
        prefix *= nums[i]

    suffix = 1
    for i in range(n - 1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]

    return result

```

6.4 Maximum Subarray

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/maximum-subarray/>

Problem statement

Given an integer array `nums`, return the contiguous subarray with the largest sum.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [-2,1,-3,4,-1,2,1,-5,4]` -> 6
- `nums = [1]` -> 1

Brute-force: $O(n^2)$, space $O(1)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

Try every starting index and keep a running sum as you extend the subarray to the right.

Brute-force solution

```

def maximum_subarray_bruteforce(nums):
    best = nums[0]
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best

```

Best solution

Kadane's algorithm decides whether to extend the current subarray or start fresh at the current value.

Best solution

```
def maximum_subarray_best(nums):
    current = best = nums[0]
    for num in nums[1:]:
        current = max(num, current + num)
        best = max(best, current)
    return best
```

6.5 Next Permutation

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/next-permutation/>

Problem statement

Rearrange nums into the next lexicographically greater permutation. If that is not possible, rearrange it into the lowest possible order in-place.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1,2,3] -> [1,3,2]`
- `nums = [3,2,1] -> [1,2,3]`

Brute-force: $O(n! \cdot n)$, space $O(n!)$ **conceptually Best solution:** $O(n)$, space $O(1)$

Brute-force idea

A baseline way is to generate all permutations, sort them, and pick the next one. That works for tiny inputs only.

Brute-force solution

```
from itertools import permutations

def next_permutation_bruteforce(nums):
    perms = sorted(set(permutations(nums)))
```

```

current = tuple(nums)
idx = perms.index(current)
nxt = perms[(idx + 1) % len(perms)]
nums[:] = list(nxt)
return nums

```

Best solution

The efficient method finds the first drop from the right, swaps with the next larger value, then reverses the suffix.

Best solution

```

def next_permutation_best(nums):
    i = len(nums) - 2
    while i >= 0 and nums[i] >= nums[i + 1]:
        i -= 1
    if i >= 0:
        j = len(nums) - 1
        while nums[j] <= nums[i]:
            j -= 1
        nums[i], nums[j] = nums[j], nums[i]
    left, right = i + 1, len(nums) - 1
    while left < right:
        nums[left], nums[right] = nums[right], nums[left]
        left += 1
        right -= 1
    return nums

```

6.6 Majority Element

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/majority-element/>

Problem statement

Given an array `nums` of size `n`, return the majority element, which appears more than $\text{floor}(n/2)$ times.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [3,2,3] -> 3`

- `nums = [2,2,1,1,1,2,2] -> 2`

Brute-force: $O(n)$, space $O(n)$ **Best solution:** $O(n)$, space $O(1)$

Brute-force idea

A frequency map is a clean baseline because it directly counts how often each number appears.

Brute-force solution

```
def majority_element_bruteforce(nums):
    counts = {}
    limit = len(nums) // 2
    for num in nums:
        counts[num] = counts.get(num, 0) + 1
        if counts[num] > limit:
            return num
```

Best solution

Boyer-Moore keeps one candidate and a vote counter. Because the majority element appears more than half the time, it survives cancellation.

Best solution

```
def majority_element_best(nums):
    candidate = None
    count = 0
    for num in nums:
        if count == 0:
            candidate = num
        count += 1 if num == candidate else -1
    return candidate
```

6.7 Sort Colors

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/sort-colors/>

Problem statement

Sort the array `nums` containing only 0s, 1s, and 2s in-place.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [2,0,2,1,1,0] -> [0,0,1,1,2,2]`
- `nums = [2,0,1] -> [0,1,2]`

Brute-force: $O(n \log n)$, space $O(1)$ or $O(n)$ depending on sort **Best solution:** $O(n)$, space $O(1)$

Brute-force idea

As a baseline, you can just sort the array. It solves the problem, but it misses the in-place one-pass idea interviewers usually want.

Brute-force solution

```
def sort_colors_bruteforce(nums):
    nums.sort()
    return nums
```

Best solution

The Dutch National Flag pattern keeps three regions: 0s on the left, 2s on the right, and scans 1s in the middle.

Best solution

```
def sort_colors_best(nums):
    low = mid = 0
    high = len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1
            mid += 1
        elif nums[mid] == 2:
            nums[mid], nums[high] = nums[high], nums[mid]
            high -= 1
        else:
            mid += 1
    return nums
```

6.8 Search a 2D Matrix II

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/search-a-2d-matrix-ii/>

Problem statement

Write an efficient algorithm that searches for a target value in an $m \times n$ matrix where each row and column is sorted in ascending order.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- matrix = [[1,4,7],[2,5,8],[3,6,9]], target = 5 -> True
- matrix = [[1,4,7],[2,5,8],[3,6,9]], target = 10 -> False

Brute-force: $O(mn)$, space $O(1)$

Best solution: $O(m+n)$, space $O(1)$

Brute-force idea

The simplest baseline scans every cell until it finds the target.

Brute-force solution

```
def search_matrix_ii_bruteforce(matrix, target):
    for row in matrix:
        for value in row:
            if value == target:
                return True
    return False
```

Best solution

Starting from the top-right corner is better because each comparison lets you discard a row or a column.

Best solution

```
def search_matrix_ii_best(matrix, target):
    if not matrix or not matrix[0]:
        return False
    row, col = 0, len(matrix[0]) - 1
    while row < len(matrix) and col >= 0:
```

```

    value = matrix[row][col]
    if value == target:
        return True
    if value > target:
        col -= 1
    else:
        row += 1
    return False

```

6.9 Longest Substring Without Repeating Characters

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/longest-substring-without-repeating-characters/>

Problem statement

Given a string *s*, return the length of the longest substring without repeating characters.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- *s* = 'abcabcbb' -> 3
- *s* = 'bbbb' -> 1

Brute-force: $O(n^2)$, space $O(n)$ **Best solution:** $O(n)$, space $O(\min(n, \text{alphabet}))$

Brute-force idea

The baseline starts at every index and grows the substring until a repeat appears.

Brute-force solution

```

def length_of_longest_substring_bruteforce(s):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            if s[j] in seen:
                break
            seen.add(s[j])
            best = max(best, j - i + 1)
    return best

```

Best solution

A sliding window keeps only the current unique substring. When a duplicate appears, move the left side forward.

Best solution

```
def length_of_longest_substring_best(s):
    seen = {}
    left = 0
    best = 0
    for right, ch in enumerate(s):
        if ch in seen and seen[ch] >= left:
            left = seen[ch] + 1
        seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

6.10 Minimum Window Substring

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/minimum-window-substring/>

Problem statement

Given strings *s* and *t*, return the minimum window in *s* that contains every character in *t*. If there is no such window, return the empty string.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- *s* = 'ADOBECODEBANC', *t* = 'ABC' -> 'BANC'
- *s* = 'a', *t* = 'aa' -> ""

Brute-force: $O(n^2)$, space $O(\text{alphabet})$

Best solution: $O(n)$, space $O(\text{alphabet})$

Brute-force idea

A direct baseline tries every starting point and expands until the window covers *t*.

Brute-force solution

```

from collections import Counter

def minimum_window_substring_bruteforce(s, t):
    need = Counter(t)
    answer = ""
    for i in range(len(s)):
        have = Counter()
        for j in range(i, len(s)):
            have[s[j]] += 1
            if all(have[c] >= need[c] for c in need):
                window = s[i:j + 1]
                if not answer or len(window) < len(answer):
                    answer = window
                break
    return answer

```

Best solution

The efficient solution uses a sliding window, counts how many required characters are satisfied, and shrinks whenever the window is valid.

Best solution

```

from collections import Counter

def minimum_window_substring_best(s, t):
    need = Counter(t)
    have = {}
    formed = 0
    required = len(need)
    left = 0
    best = (float("inf"), 0, 0)

    for right, ch in enumerate(s):
        have[ch] = have.get(ch, 0) + 1
        if ch in need and have[ch] == need[ch]:
            formed += 1

        while formed == required:
            if right - left + 1 < best[0]:
                best = (right - left + 1, left, right)
            left_ch = s[left]
            have[left_ch] -= 1
            if left_ch in need and have[left_ch] < need[left_ch]:
                formed -= 1
            left += 1

    if best[0] == float("inf"):
        return ""
    return s[best[1]:best[2] + 1]

```

6.11 Longest Palindromic Substring

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/longest-palindromic-substring/>

Problem statement

Given a string *s*, return the longest palindromic substring in *s*.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- *s* = 'babad' -> 'bab' or 'aba'
- *s* = 'cbdd' -> 'bb'

Brute-force: $O(n^3)$, space $O(1)$

Best solution: $O(n^2)$, space $O(1)$

Brute-force idea

The baseline checks every substring and tests whether it reads the same forward and backward.

Brute-force solution

```
def longest_palindromic_substring_bruteforce(s):
    best = ""
    for i in range(len(s)):
        for j in range(i, len(s)):
            piece = s[i:j + 1]
            if piece == piece[::-1] and len(piece) > len(best):
                best = piece
    return best
```

Best solution

Expand around each center. Every palindrome grows from either one middle character or a gap between two characters.

Best solution

```
def longest_palindromic_substring_best(s):
    def expand(left, right):
        while left >= 0 and right < len(s) and s[left] == s[right]:
```

```

        left -= 1
        right += 1
    return s[left + 1:right]

best = ""
for i in range(len(s)):
    odd = expand(i, i)
    even = expand(i, i + 1)
    best = max(best, odd, even, key=len)
return best

```

6.12 Longest Repeating Character Replacement

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/longest-repeating-character-replacement/>

Problem statement

You can replace at most k characters in s . Return the length of the longest substring that can be made of the same character.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $s = \text{'ABAB'}$, $k = 2 \rightarrow 4$
- $s = \text{'AABABBA'}$, $k = 1 \rightarrow 4$

Brute-force: $O(n^2)$, space $O(\text{alphabet})$

Best solution: $O(n)$, space $O(\text{alphabet})$

Brute-force idea

The baseline tries every window and checks how many replacements it needs.

Brute-force solution

```

def character_replacement_bruteforce(s, k):
    best = 0
    for i in range(len(s)):
        counts = {}
        max_freq = 0
        for j in range(i, len(s)):
            counts[s[j]] = counts.get(s[j], 0) + 1

```

```

        max_freq = max(max_freq, counts[s[j]])
        if (j - i + 1) - max_freq <= k:
            best = max(best, j - i + 1)
    return best

```

Best solution

The sliding-window version keeps the most frequent character count in the window and shrinks only when replacements exceed k.

Best solution

```

def character_replacement_best(s, k):
    counts = {}
    left = 0
    max_freq = 0
    best = 0
    for right, ch in enumerate(s):
        counts[ch] = counts.get(ch, 0) + 1
        max_freq = max(max_freq, counts[ch])
        while (right - left + 1) - max_freq > k:
            counts[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best

```

6.13 Valid Palindrome

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/valid-palindrome/>

Problem statement

Return true if s is a palindrome after converting uppercase letters to lowercase and removing non-alphanumeric characters.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- s = 'A man, a plan, a canal: Panama' -> True
- s = 'race a car' -> False

Brute-force: $O(n)$, space $O(n)$ **Best solution:** $O(n)$, space $O(1)$

Brute-force idea

A simple baseline cleans the string first, then compares it with its reverse.

Brute-force solution

```
def valid_palindrome_bruteforce(s):
    cleaned = "".join(ch.lower() for ch in s if ch.isalnum())
    return cleaned == cleaned[::-1]
```

Best solution

Two pointers avoid building a full cleaned copy. Move inward while skipping punctuation and spaces.

Best solution

```
def valid_palindrome_best(s):
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

6.14 Valid Anagram

Problem quick facts

Difficulty: Easy**LeetCode:** <https://leetcode.com/problems/valid-anagram/>

Problem statement

Given two strings s and t , return true if t is an anagram of s .

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `s = 'anagram', t = 'nagaram' -> True`
- `s = 'rat', t = 'car' -> False`

Brute-force: $O(n \log n)$, space $O(1)$ or $O(n)$ **Best solution:** $O(n)$, space $O(\text{alphabet})$

Brute-force idea

Sorting is the direct baseline because anagrams become identical after ordering the letters.

Brute-force solution

```
def valid_anagram_bruteforce(s, t):  
    return sorted(s) == sorted(t)
```

Best solution

A counting map is better because it compares character frequencies in linear time.

Best solution

```
def valid_anagram_best(s, t):  
    if len(s) != len(t):  
        return False  
    counts = {}  
    for ch in s:  
        counts[ch] = counts.get(ch, 0) + 1  
    for ch in t:  
        if ch not in counts:  
            return False  
        counts[ch] -= 1  
        if counts[ch] == 0:  
            del counts[ch]  
    return not counts
```

6.15 Encode and Decode Strings

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/encode-and-decode-strings/>

Problem statement

Design an algorithm to encode a list of strings to a single string and decode it back to the same list.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- ['lint', 'code', 'love', 'you'] -> decodes back to the same list
- [", '#hash', '12'] -> decodes back to the same list

Brute-force: $O(\text{total length})$, space $O(\text{total length})$

Best solution: $O(\text{total length})$, space $O(\text{total length})$

Brute-force idea

A fragile baseline is to join on a delimiter and escape that delimiter inside each string first.

Brute-force solution

```
def encode_strings_bruteforce(strs):
    escaped = [s.replace('#', '##') for s in strs]
    return '#|#'.join(escaped)

def decode_strings_bruteforce(s):
    parts = []
    current = []
    i = 0
    while i < len(s):
        if s[i:i + 3] == '#|#':
            parts.append("".join(current))
            current = []
            i += 3
        elif s[i:i + 2] == '##':
            current.append('#')
            i += 2
        else:
            current.append(s[i])
            i += 1
    parts.append("".join(current))
    return parts if s else [""]
```

Best solution

Length-prefix encoding is cleaner. Store each string as `<length>#<content>`, so decoding always knows exactly how many characters to read next.

Best solution

```
def encode_strings_best(strs):
    return "".join(f"{len(s)}#{s}" for s in strs)

def decode_strings_best(s):
    result = []
    i = 0
    while i < len(s):
        j = i
        while s[j] != '#':
            j += 1
        length = int(s[i:j])
        j += 1
        result.append(s[j:j + length])
        i = j + length
    return result
```

6.16 String to Integer (atoi)

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/string-to-integer-atoi/>

Problem statement

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer using the usual `atoi` rules.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `s = '42' -> 42`
- `s = '-42abc' -> -42`

Brute-force: $O(n)$, space $O(n)$ **Best solution:** $O(n)$, space $O(1)$

Brute-force idea

A baseline can trim spaces, collect the optional sign and digit run, then clamp the parsed value to the 32-bit range.

Brute-force solution

```
def my_atoi_bruteforce(s):
    s = s.lstrip()
    if not s:
        return 0
    sign = 1
    if s[0] in '+-':
        sign = -1 if s[0] == '-' else 1
        s = s[1:]
    digits = []
    for ch in s:
        if not ch.isdigit():
            break
        digits.append(ch)
    if not digits:
        return 0
    value = sign * int("".join(digits))
    return max(-(2 ** 31), min(2 ** 31 - 1, value))
```

Best solution

The tighter one-pass version builds the integer digit by digit and clamps as soon as it would overflow.

Best solution

```
def my_atoi_best(s):
    i = 0
    n = len(s)
    while i < n and s[i] == ' ':
        i += 1
    if i == n:
        return 0

    sign = 1
    if s[i] in '+-':
        sign = -1 if s[i] == '-' else 1
        i += 1

    value = 0
    INT_MAX = 2 ** 31 - 1
    INT_MIN = -(2 ** 31)

    while i < n and s[i].isdigit():
        digit = ord(s[i]) - ord('0')
        value = value * 10 + digit
        if sign == 1 and value > INT_MAX:
            return INT_MAX
        if sign == -1 and -value < INT_MIN:
            return INT_MIN
        i += 1

    return sign * value
```

Linked Lists

Chapter overview

Linked lists are all about pointer control. The data is simple; the real skill is rewiring next pointers without losing the rest of the list.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

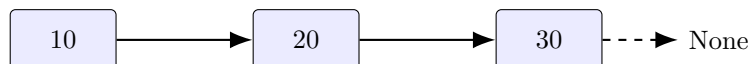
Linked Lists primer

This chapter opening folds in the handbook theory for linked lists so the workbook stays self-contained. Read this primer first, then move into the interview problems.

A linked list trades fast random access for flexible pointer updates. That trade-off matters a lot in interviews because it changes how you think about movement, insertion, and deletion.

Singly linked list

Each node stores a value and a reference to the next node.



Reverse a linked list

This is one of the most common linked-list questions because it tests pointer discipline. The trick is to keep track of previous, current, and next.

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
  
```

```

        self.next = next

def reverse_list(head):
    prev = None
    current = head
    while current:
        nxt = current.next
        current.next = prev
        prev = current
        current = nxt
    return prev

```

Cycle detection

Floyd's tortoise-and-hare method uses two speeds. If a cycle exists, the fast pointer eventually laps the slow one.

```

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    if slow is fast:
        return True
    return False

```

7.1 Reverse Linked List

Problem statement

Given the head of a singly linked list, reverse the list and return the new head.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- 1 -> 2 -> 3 -> 4 -> 5 becomes 5 -> 4 -> 3 -> 2 -> 1
- Empty list stays empty

Brute-force: Time $O(n)$, space $O(n)$

Best solution: Time $O(n)$, space $O(1)$

Brute-force idea

A beginner-friendly path is to collect the node values and rebuild the list in reverse order.

Brute-force solution

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_list_bruteforce(head):
    values = []
    current = head
    while current:
        values.append(current.val)
        current = current.next

    new_head = None
    for value in values:
        new_head = ListNode(value, new_head)
    return new_head

```

Best solution

The in-place version is the classic interview move: keep `prev`, `curr`, and `next_node`, then rewire one edge at a time.

Best solution

```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_list_best(head):
    prev = None
    current = head
    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node
    return prev

```

7.2 Merge Two Sorted Lists

Problem statement

Given two sorted linked lists, merge them into one sorted linked list and return the head of the merged list.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- 1 -> 2 -> 4 and 1 -> 3 -> 4 becomes 1 -> 1 -> 2 -> 3 -> 4 -> 4
- Empty list and 0 becomes 0

Brute-force: Time $O((m+n) \log(m+n))$, space $O(m+n)$ **Best solution:** Time $O(m+n)$, space $O(1)$ extra

Brute-force idea

The easy way is to copy all values into a list, sort them, and rebuild the answer.

Brute-force solution

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def merge_two_lists_bruteforce(list1, list2):
    values = []
    current = list1
    while current:
        values.append(current.val)
        current = current.next
    current = list2
    while current:
        values.append(current.val)
        current = current.next

    values.sort()
    dummy = ListNode()
    tail = dummy
    for value in values:
        tail.next = ListNode(value)
        tail = tail.next
    return dummy.next
```

Best solution

Use a dummy head and always attach the smaller current node. This is one of the most reusable linked-list patterns.

Best solution

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

```
def merge_two_lists_best(list1, list2):
    dummy = ListNode()
    tail = dummy

    while list1 and list2:
        if list1.val <= list2.val:
            tail.next = list1
            list1 = list1.next
        else:
            tail.next = list2
            list2 = list2.next
        tail = tail.next

    tail.next = list1 if list1 else list2
    return dummy.next
```

7.3 Copy List with Random Pointer

Problem statement

Each node has a next pointer and a random pointer that may point anywhere in the list or be null. Return a deep copy of the list.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- A list with random jumps should be copied so that next and random relationships match, but copied nodes are distinct objects
- Empty list returns None

Brute-force: Time $O(n^2)$, space $O(n)$

Best solution: Time $O(n)$, space $O(n)$

Brute-force idea

You can first copy all values into an array of new nodes, then use repeated scans to discover the random target index for each original node.

Brute-force solution

```
class RandomNode:
    def __init__(self, val=0, next=None, random=None):
        self.val = val
        self.next = next
        self.random = random
```

```

def copy_random_list_bruteforce(head):
    if not head:
        return None

    originals = []
    current = head
    while current:
        originals.append(current)
        current = current.next

    copies = [RandomNode(node.val) for node in originals]

    for i in range(len(copies) - 1):
        copies[i].next = copies[i + 1]

    for i, node in enumerate(originals):
        if node.random is None:
            continue
        target_index = 0
        while originals[target_index] is not node.random:
            target_index += 1
        copies[i].random = copies[target_index]

    return copies[0]

```

Best solution

A direct hash map from original node to copied node makes wiring next and random pointers pleasantly clean.

Best solution

```

class RandomNode:
    def __init__(self, val=0, next=None, random=None):
        self.val = val
        self.next = next
        self.random = random

def copy_random_list_best(head):
    if not head:
        return None

    old_to_new = {}
    current = head
    while current:
        old_to_new[current] = RandomNode(current.val)
        current = current.next

    current = head
    while current:
        copy = old_to_new[current]
        copy.next = old_to_new.get(current.next)
        copy.random = old_to_new.get(current.random)
        current = current.next

    return old_to_new[head]

```

7.4 Linked List Cycle

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/linked-list-cycle/>

Problem statement

Given head, determine if the linked list has a cycle in it.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- 3 -> 2 -> 0 -> -4 with tail pointing to index 1 -> True
- 1 -> 2 -> None -> False

Brute-force: $O(n)$, space $O(n)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

A set-based baseline remembers visited nodes. If you see a node again, the list loops.

Brute-force solution

```
def has_cycle_bruteforce(head):
    seen = set()
    node = head
    while node:
        if node in seen:
            return True
        seen.add(node)
        node = node.next
    return False
```

Best solution

Floyd's tortoise-and-hare trick uses two pointers moving at different speeds. In a cycle, they eventually meet.

Best solution

```
def has_cycle_best(head):
    slow = fast = head
```

```

while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
    if slow is fast:
        return True
return False

```

7.5 Merge K Sorted Lists

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/merge-k-sorted-lists/>

Problem statement

You are given an array of k linked lists, each sorted in ascending order. Merge them into one sorted linked list.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `lists = [[1,4,5],[1,3,4],[2,6]] -> [1,1,2,3,4,4,5,6]`
- `lists = [] -> []`

Brute-force: $O(N \log N)$, space $O(N)$

Best solution: $O(N \log k)$, space $O(k)$

Brute-force idea

A simple baseline pulls all node values into a Python list, sorts them, then rebuilds a new linked list.

Brute-force solution

```

def merge_k_lists_bruteforce(lists):
    values = []
    for node in lists:
        while node:
            values.append(node.val)
            node = node.next
    values.sort()

    dummy = ListNode(0)
    tail = dummy

```

```

for value in values:
    tail.next = ListNode(value)
    tail = tail.next
return dummy.next

```

Best solution

The better approach keeps the current smallest head from each list in a min-heap, so you never sort everything twice.

Best solution

```

import heapq

def merge_k_lists_best(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))

    dummy = ListNode(0)
    tail = dummy

    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))

    tail.next = None
    return dummy.next

```

7.6 Remove Nth Node From End of List

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/remove-nth-node-from-end-of-list/>

Problem statement

Given the head of a linked list, remove the *n*th node from the end of the list and return its head.
Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- head = [1,2,3,4,5], n = 2 -> [1,2,3,5]
- head = [1], n = 1 -> []

Brute-force: $O(n)$, space $O(n)$ **Best solution:** $O(n)$, space $O(1)$ **Brute-force idea**

A baseline stores every node in an array so you can index the one that must be removed.

Brute-force solution

```
def remove_nth_from_end_bruteforce(head, n):
    nodes = []
    node = head
    while node:
        nodes.append(node)
        node = node.next

    idx = len(nodes) - n
    if idx == 0:
        return head.next

    nodes[idx - 1].next = nodes[idx].next
    return head
```

Best solution

Two pointers keep a fixed gap of n nodes. When the fast pointer reaches the end, the slow pointer stands right before the node to delete.

Best solution

```
def remove_nth_from_end_best(head, n):
    dummy = ListNode(0, head)
    slow = fast = dummy
    for _ in range(n + 1):
        fast = fast.next
    while fast:
        slow = slow.next
        fast = fast.next
    slow.next = slow.next.next
    return dummy.next
```

7.7 Reorder List

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/reorder-list/>

Problem statement

Given the head of a singly linked list $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$, reorder it to $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow \dots$ in-place.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- head = [1,2,3,4] $\rightarrow$ [1,4,2,3]
- head = [1,2,3,4,5] $\rightarrow$ [1,5,2,4,3]

Brute-force: $O(n)$, space $O(n)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

The straightforward baseline stores nodes in an array and reconnects from the outside in.

Brute-force solution

```
def reorder_list_bruteforce(head):
    if not head:
        return
    nodes = []
    node = head
    while node:
        nodes.append(node)
        node = node.next

    left, right = 0, len(nodes) - 1
    dummy = ListNode(0)
    tail = dummy
    while left <= right:
        tail.next = nodes[left]
        tail = tail.next
        if left != right:
            tail.next = nodes[right]
            tail = tail.next
        left += 1
        right -= 1
    tail.next = None
```

Best solution

The in-place method finds the middle, reverses the second half, then weaves the two halves together.

Best solution

```
def reorder_list_best(head):
    if not head or not head.next:
        return

    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

    prev = None
    node = slow.next
    slow.next = None
    while node:
        nxt = node.next
        node.next = prev
        prev = node
        node = nxt

    first, second = head, prev
    while second:
        n1, n2 = first.next, second.next
        first.next = second
        second.next = n1
        first, second = n1, n2
```

7.8 Reverse Nodes in k-Group

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/reverse-nodes-in-k-group/>

Problem statement

Given the head of a linked list, reverse the nodes of the list k at a time and return the modified list. If the number of nodes is not a multiple of k, leave the remaining nodes as-is.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- head = [1,2,3,4,5], k = 2 -> [2,1,4,3,5]
- head = [1,2,3,4,5], k = 3 -> [3,2,1,4,5]

Brute-force: $O(n)$, space $O(k)$ **Best solution:** $O(n)$, space $O(1)$

Brute-force idea

A reasonable baseline collects each full group of k nodes in a list, reconnects them in reverse order, and keeps leftover nodes untouched.

Brute-force solution

```
def reverse_k_group_bruteforce(head, k):
    dummy = ListNode(0, head)
    prev_group = dummy

    while True:
        group = []
        node = prev_group.next
        for _ in range(k):
            if not node:
                return dummy.next
            group.append(node)
            node = node.next

        next_group = node
        for current in reversed(group):
            prev_group.next = current
            prev_group = prev_group.next
        prev_group.next = next_group
```

Best solution

The in-place version checks that k nodes exist, reverses that block with pointer flips, then moves to the next block.

Best solution

```
def reverse_k_group_best(head, k):
    dummy = ListNode(0, head)
    group_prev = dummy

    while True:
        kth = group_prev
        for _ in range(k):
            kth = kth.next
            if not kth:
                return dummy.next

        group_next = kth.next
        prev, curr = group_next, group_prev.next

        while curr != group_next:
            nxt = curr.next
            curr.next = prev
            prev = curr
```

```

curr = nxt

tail = group_prev.next
group_prev.next = kth
group_prev = tail

```

7.9 Intersection of Two Linked Lists

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/intersection-of-two-linked-lists/>

Problem statement

Given the heads of two singly linked lists, return the node at which the two lists intersect. If they do not intersect, return None.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- A = [4,1,8,4,5], B = [5,6,1,8,4,5] -> node with value 8
- A = [1,2], B = [3,4] -> None

Brute-force: $O(m+n)$, space $O(m)$

Best solution: $O(m+n)$, space $O(1)$

Brute-force idea

A simple baseline stores every node from the first list in a set, then walks the second list until it finds a shared node.

Brute-force solution

```

def get_intersection_node_bruteforce(head_a, head_b):
    seen = set()
    node = head_a
    while node:
        seen.add(node)
        node = node.next

    node = head_b
    while node:
        if node in seen:

```

```

        return node
    node = node.next
    return None

```

Best solution

The elegant trick switches heads when each pointer reaches the end. Both pointers walk the same total distance, so they align at the intersection.

Best solution

```

def get_intersection_node_best(head_a, head_b):
    a, b = head_a, head_b
    while a is not b:
        a = a.next if a else head_b
        b = b.next if b else head_a
    return a

```

7.10 Reverse Linked List II

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/reverse-linked-list-ii/>

Problem statement

Reverse the nodes of a linked list from position left to position right and return the new head.
Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- head = [1,2,3,4,5], left = 2, right = 4 -> [1,4,3,2,5]
- head = [5], left = 1, right = 1 -> [5]

Brute-force: $O(n)$, space $O(n)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

The baseline can collect nodes in an array, reverse the target slice, and reconnect everything.

Brute-force solution

```
def reverse_between_bruteforce(head, left, right):
    nodes = []
    node = head
    while node:
        nodes.append(node)
        node = node.next

    nodes[left - 1:right] = reversed(nodes[left - 1:right])

    for i in range(len(nodes) - 1):
        nodes[i].next = nodes[i + 1]
    if nodes:
        nodes[-1].next = None
    return nodes[0]
return None
```

Best solution

The in-place method walks to the node before the segment and repeatedly moves the next node to the front of that segment.

Best solution

```
def reverse_between_best(head, left, right):
    dummy = ListNode(0, head)
    prev = dummy
    for _ in range(left - 1):
        prev = prev.next

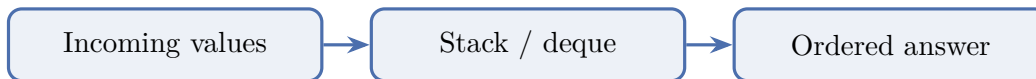
    current = prev.next
    for _ in range(right - left):
        nxt = current.next
        current.next = nxt.next
        nxt.next = prev.next
        prev.next = nxt

    return dummy.next
```

Stacks and Queues

Chapter overview

When data needs to be processed in a disciplined order, stacks and queues keep you sane. In interviews, they often show up hidden inside parsing, monotonic scans, and sliding-window problems.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

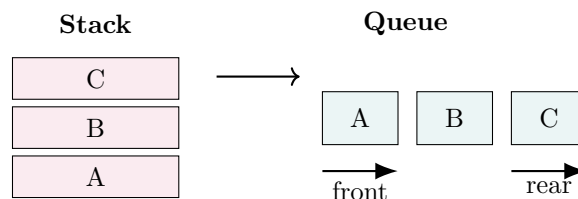
Stacks primer

This chapter opening folds in the handbook theory for stacks so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Stacks shine when the most recent thing matters most. That is why they show up in parsing, backtracking, expression handling, and monotonic problems.

Mental model

A stack is last in, first out. A queue is first in, first out. Keeping those two pictures in your head solves half the confusion.



Valid parentheses

Push opening brackets. When you see a closing bracket, it must match the top of the stack.

```
def is_valid_parentheses(text):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in text:
        if ch in '([{':
            stack.append(ch)
        else:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

Next greater element

This is where the monotonic stack idea starts to click. Keep only the candidates that still matter.

```
def next_greater(nums):
    answer = [-1] * len(nums)
    stack = []
    for i in range(len(nums) - 1, -1, -1):
        while stack and stack[-1] <= nums[i]:
            stack.pop()
        if stack:
            answer[i] = stack[-1]
        stack.append(nums[i])
    return answer
```

Queues and Deques primer

This chapter opening folds in the handbook theory for queues and deques so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Queues are great when order of arrival matters. Deques extend that idea by letting you work from both ends, which turns out to be useful more often than people expect.

Queue basics

Think service line, print jobs, or task scheduling. The oldest item gets handled first.

Deque in Python

Python's `collections.deque` gives you efficient operations at both ends.

```
from collections import deque

queue = deque()
queue.append(10)      # enqueue
queue.append(20)
first = queue.popleft()

dq = deque([1, 2, 3])
dq.appendleft(0)
dq.append(4)
```

Priority queue / heap preview

A priority queue serves the most important item first rather than the oldest item first. In Python, `heapq` gives you a min-heap.

8.1 Valid Parentheses

Problem statement

Given a string containing just the characters `()`, `{}`, and `[]`, determine whether the input string is valid. An opening bracket must be closed by the same type in the correct order.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `s = '()[]{}' -> True`
- `s = '()' -> False`

Brute-force: Time $O(n^2)$, space $O(n)$

Best solution: Time $O(n)$, space $O(n)$

Brute-force idea

A clunky but valid brute-force strategy keeps removing matching pairs until nothing changes.

Brute-force solution

```
def valid_parentheses_bruteforce(s):
    previous = None
    current = s
    while current != previous:
        previous = current
        current = current.replace("()", "")
        current = current.replace("[]", "")
        current = current.replace("{} ", "")
    return current == ""
```

Best solution

A stack is the natural tool: push openers, and whenever you see a closer, check whether it matches the latest opener.

Best solution

```
def valid_parentheses_best(s):
    pairs = {'(': ')', '[': ']', '{': '}'}
```

```

stack = []
for ch in s:
    if ch in '({':
        stack.append(ch)
    else:
        if not stack or stack[-1] != pairs[ch]:
            return False
        stack.pop()
return not stack

```

8.2 Daily Temperatures

Problem statement

Given a list of daily temperatures, return an array `answer` where `answer[i]` is the number of days until a warmer temperature. If no warmer day exists, `answer[i]` should be 0.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `temperatures = [73, 74, 75, 71, 69, 72, 76, 73] -> [1, 1, 4, 2, 1, 1, 0, 0]`
- `temperatures = [30, 40, 50, 60] -> [1, 1, 1, 0]`

Brute-force: Time $O(n^2)$, space $O(1)$ excluding output **Best solution:** Time $O(n)$, space $O(n)$

Brute-force idea

For each day, scan forward until you find a warmer one.

Brute-force solution

```

def daily_temperatures_bruteforce(temperatures):
    n = len(temperatures)
    answer = [0] * n
    for i in range(n):
        for j in range(i + 1, n):
            if temperatures[j] > temperatures[i]:
                answer[i] = j - i
                break
    return answer

```

Best solution

A monotonic decreasing stack stores indices whose warmer answer has not been found yet.

Best solution

```
def daily_temperatures_best(temperatures):
    answer = [0] * len(temperatures)
    stack = []

    for i, temp in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < temp:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)

    return answer
```

8.3 Sliding Window Maximum

Problem statement

Given an array `nums` and an integer `k`, return the maximum value in every sliding window of size `k`.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1, 3, -1, -3, 5, 3, 6, 7]`, `k = 3` -> `[3, 3, 5, 5, 6, 7]`
- `nums = [1]`, `k = 1` -> `[1]`

Brute-force: Time $O(nk)$, space $O(1)$ excluding output **Best solution:** Time $O(n)$, space $O(k)$

Brute-force idea

The brute-force version recomputes the max for every window from scratch.

Brute-force solution

```
def sliding_window_max_bruteforce(nums, k):
    result = []
    for left in range(len(nums) - k + 1):
        result.append(max(nums[left:left + k]))
    return result
```

Best solution

A deque keeps indices in decreasing value order, so the front is always the current window maximum.

Best solution

```
from collections import deque

def sliding_window_max_best(nums, k):
    dq = deque()
    result = []

    for i, num in enumerate(nums):
        while dq and dq[0] <= i - k:
            dq.popleft()

        while dq and nums[dq[-1]] <= num:
            dq.pop()

        dq.append(i)

        if i >= k - 1:
            result.append(nums[dq[0]])

    return result
```

8.4 Min Stack

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/min-stack/>

Problem statement

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- push(-2), push(0), push(-3), getMin() -> -3
- after pop(), top() -> 0 and getMin() -> -2

Brute-force: push/pop/top $O(1)$, getMin $O(n)$

Best solution: all operations $O(1)$

Brute-force idea

A baseline stack can compute the minimum by scanning all values when `getMin` is called.

Brute-force solution

```
class MinStackBruteforce:
    def __init__(self):
        self.stack = []

    def push(self, val):
        self.stack.append(val)

    def pop(self):
        self.stack.pop()

    def top(self):
        return self.stack[-1]

    def getMin(self):
        return min(self.stack)
```

Best solution

The standard solution keeps a second stack of running minimums so each operation stays constant time.

Best solution

```
class MinStackBest:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val):
        self.stack.append(val)
        if not self.min_stack:
            self.min_stack.append(val)
        else:
            self.min_stack.append(min(val, self.min_stack[-1]))

    def pop(self):
        self.stack.pop()
        self.min_stack.pop()

    def top(self):
        return self.stack[-1]

    def getMin(self):
        return self.min_stack[-1]
```

8.5 Largest Rectangle in Histogram

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/largest-rectangle-in-histogram/>

Problem statement

Given an array of bar heights in a histogram, return the area of the largest rectangle.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- heights = [2,1,5,6,2,3] -> 10
- heights = [2,4] -> 4

Brute-force: $O(n^2)$, space $O(1)$

Best solution: $O(n)$, space $O(n)$

Brute-force idea

The baseline treats every bar as the limiting height and expands left and right while bars stay tall enough.

Brute-force solution

```
def largest_rectangle_area_bruteforce(heights):
    best = 0
    for i in range(len(heights)):
        height = heights[i]
        left = right = i
        while left - 1 >= 0 and heights[left - 1] >= height:
            left -= 1
        while right + 1 < len(heights) and heights[right + 1] >= height:
            right += 1
        best = max(best, height * (right - left + 1))
    return best
```

Best solution

A monotonic increasing stack tells you exactly when a bar's rectangle must end, so each index is pushed and popped at most once.

Best solution

```
def largest_rectangle_area_best(heights):
    stack = []
    best = 0
    for i, h in enumerate(heights + [0]):
        start = i
        while stack and stack[-1][1] > h:
            index, height = stack.pop()
            best = max(best, height * (i - index))
            start = index
        stack.append((start, h))
    return best
```

8.6 Trapping Rain Water

Problem quick facts

Difficulty: Hard**LeetCode:** <https://leetcode.com/problems/trapping-rain-water/>

Problem statement

Given n non-negative integers representing an elevation map, compute how much water it can trap after raining.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- height = [0,1,0,2,1,0,1,3,2,1,2,1] -> 6
- height = [4,2,0,3,2,5] -> 9

Brute-force: $O(n^2)$, space $O(1)$ **Best solution:** $O(n)$, space $O(1)$

Brute-force idea

For each index, a baseline scans for the tallest bar on the left and right, then uses the smaller wall height.

Brute-force solution

```
def trap_bruteforce(height):
    water = 0
    for i in range(len(height)):
        left_max = max(height[:i + 1])
```

```

right_max = max(height[i:])
water += min(left_max, right_max) - height[i]
return water

```

Best solution

Two pointers work because the lower side determines how much water you can safely count at that moment.

Best solution

```

def trap_best(height):
    left, right = 0, len(height) - 1
    left_max = right_max = 0
    water = 0
    while left < right:
        if height[left] < height[right]:
            left_max = max(left_max, height[left])
            water += left_max - height[left]
            left += 1
        else:
            right_max = max(right_max, height[right])
            water += right_max - height[right]
            right -= 1
    return water

```

8.7 Evaluate Reverse Polish Notation

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/evaluate-reverse-polish-notation/>

Problem statement

Evaluate the value of an arithmetic expression in Reverse Polish Notation.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- tokens = ['2','1','+','3','*'] -> 9
- tokens = ['4','13','5','/', '+'] -> 6

Brute-force: $O(n)$, space $O(n)$ **Best solution:** $O(n)$, space $O(n)$

Brute-force idea

A recursive baseline can consume the expression from the end and evaluate subexpressions as they appear.

Brute-force solution

```
def eval_rpn_bruteforce(tokens):
    tokens = tokens[:]

    def dfs():
        token = tokens.pop()
        if token not in {'+', '-', '*', '/'}:
            return int(token)
        right = dfs()
        left = dfs()
        if token == '+':
            return left + right
        if token == '-':
            return left - right
        if token == '*':
            return left * right
        return int(left / right)

    return dfs()
```

Best solution

The stack solution is the interview standard: push numbers, pop two operands when you see an operator, and push the result back.

Best solution

```
def eval_rpn_best(tokens):
    stack = []
    for token in tokens:
        if token not in {'+', '-', '*', '/'}:
            stack.append(int(token))
        else:
            b = stack.pop()
            a = stack.pop()
            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            else:
                stack.append(int(a / b))
    return stack[-1]
```

8.8 Implement Queue Using Stacks

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/implement-queue-using-stacks/>

Problem statement

Implement a first in first out queue using only two stacks.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- push(1), push(2), peek() -> 1
- pop() -> 1, empty() -> False

Brute-force: push $O(1)$, pop $O(n)$ worst-case operation

Best solution: amortized $O(1)$ per operation

Brute-force idea

A simple baseline moves everything to the helper stack every time you need the front element.

Brute-force solution

```
class MyQueueBruteforce:
    def __init__(self):
        self.a = []
        self.b = []

    def push(self, x):
        self.a.append(x)

    def pop(self):
        while len(self.a) > 1:
            self.b.append(self.a.pop())
        front = self.a.pop()
        while self.b:
            self.a.append(self.b.pop())
        return front

    def peek(self):
        while len(self.a) > 1:
            self.b.append(self.a.pop())
        front = self.a[-1]
        while self.b:
```

```

        self.a.append(self.b.pop())
    return front

def empty(self):
    return not self.a

```

Best solution

The better design uses one input stack and one output stack. Only pour from input to output when the output stack is empty.

Best solution

```

class MyQueueBest:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def _shift(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())

    def push(self, x):
        self.in_stack.append(x)

    def pop(self):
        self._shift()
        return self.out_stack.pop()

    def peek(self):
        self._shift()
        return self.out_stack[-1]

    def empty(self):
        return not self.in_stack and not self.out_stack

```

8.9 Car Fleet

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/car-fleet/>

Problem statement

There are n cars going to the same target. Each car has a position and speed. A faster car that catches a slower car becomes part of the same fleet. Return the number of fleets that arrive at the target.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3] -> 3
- target = 10, position = [3], speed = [3] -> 1

Brute-force: $O(n^2)$, space $O(n)$

Best solution: $O(n \log n)$, space $O(n)$

Brute-force idea

A slow baseline computes each car's arrival time, sorts by position, and repeatedly merges fleets by scanning from the front.

Brute-force solution

```
def car_fleet_bruteforce(target, position, speed):
    cars = sorted(zip(position, speed))
    times = [(target - p) / s for p, s in cars]

    changed = True
    while changed:
        changed = False
        merged = []
        i = len(times) - 1
        while i >= 0:
            if i > 0 and times[i - 1] <= times[i]:
                merged.append(times[i])
                i -= 2
                changed = True
            else:
                merged.append(times[i])
                i -= 1
        times = list(reversed(merged))
    return len(times)
```

Best solution

Sort cars by starting position from closest to target backward. A new fleet appears only when a car's time is greater than the fleet time in front of it.

Best solution

```
def car_fleet_best(target, position, speed):
    cars = sorted(zip(position, speed), reverse=True)
    fleets = 0
```

```

slowest_time = 0
for pos, spd in cars:
    time = (target - pos) / spd
    if time > slowest_time:
        fleets += 1
        slowest_time = time
return fleets

```

8.10 Valid Parentheses (Stack Focus)

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/valid-parentheses/>

Problem statement

Revisit Valid Parentheses as the classic stack problem.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `s = '()[]{}' -> True`
- `s = ']' -> False`

Brute-force: $O(n)$, space $O(n)$

Best solution: $O(n)$, space $O(n)$

Brute-force idea

The direct baseline keeps removing matching pairs until nothing changes.

Brute-force solution

```

def valid_parentheses_stack_bruteforce(s):
    previous = None
    while previous != s:
        previous = s
        s = s.replace("()", "").replace("[]", "").replace("{} ", "")
    return s == ""

```

Best solution

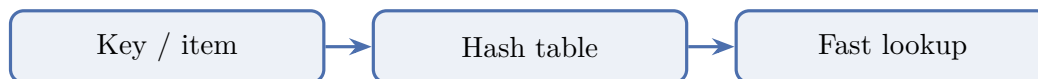
A stack is cleaner: push open brackets, and each closing bracket must match the latest opener.

```
def valid_parentheses_stack_best(s):
    pairs = {'(': ')', '[': ']', '{': '}'
    stack = []
    for ch in s:
        if ch in pairs.values():
            stack.append(ch)
        elif not stack or stack.pop() != pairs[ch]:
            return False
    return not stack
```

Hash Maps and Sets

Chapter overview

Hash tables are one of the biggest interview cheat codes. They turn repeated lookup work into fast membership checks, counts, and groupings.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

Hashing primer

This chapter opening folds in the handbook theory for hashing so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Hashing is one of the biggest interview accelerators. The moment you notice a frequency problem, a lookup problem, or a duplicate problem, a hash-based structure should come to mind.

Mental model

A hash table maps keys to buckets so lookups are usually close to constant time.

Frequency counting

Count first, reason second. This pattern shows up in anagrams, duplicates, grouping, and character problems.

```
from collections import Counter
```

```
def frequency_count(items):  
    return Counter(items)
```

Classic pattern: two sum

This problem is a great example of hashing turning a quadratic brute-force idea into a linear pass.

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        need = target - num
        if need in seen:
            return [seen[need], i]
        seen[num] = i
    return []
```

9.1 Group Anagrams

Problem statement

Given an array of strings, group the anagrams together. You may return the groups in any order.
Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- ['eat', 'tea', 'tan', 'ate', 'nat', 'bat'] -> [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
- [""] -> [[]]

Brute-force: Time $O(n^2 * k \log k)$, space $O(nk)$ **Best solution:** Time $O(nk \log k)$, space $O(nk)$

Brute-force idea

You can compare every word against every other word by sorting characters and grouping matches.

Brute-force solution

```
def group_anagrams_bruteforce(words):
    used = [False] * len(words)
    groups = []

    for i in range(len(words)):
        if used[i]:
            continue
        current_group = [words[i]]
        used[i] = True
        signature_i = ''.join(sorted(words[i]))

        for j in range(i + 1, len(words)):
            if not used[j] and ''.join(sorted(words[j])) == signature_i:
                current_group.append(words[j])
```

```

        used[j] = True

        groups.append(current_group)

    return groups

```

Best solution

Use the sorted word as a canonical signature and collect words with the same signature.

Best solution

```

from collections import defaultdict

def group_anagrams_best(words):
    groups = defaultdict(list)
    for word in words:
        key = ''.join(sorted(word))
        groups[key].append(word)
    return list(groups.values())

```

9.2 Top K Frequent Elements

Problem statement

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements in any order.
Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1, 1, 1, 2, 2, 3]`, `k = 2` -> `[1, 2]`
- `nums = [1]`, `k = 1` -> `[1]`

Brute-force: Time $O(n^2)$, space $O(n)$

Best solution: Time $O(n)$, space $O(n)$

Brute-force idea

The brute-force way counts each value with repeated scans, then sorts by frequency.

Brute-force solution

```

def top_k_frequent_bruteforce(nums, k):
    unique = []

```

```

for num in nums:
    if num not in unique:
        unique.append(num)

pairs = []
for value in unique:
    count = 0
    for num in nums:
        if num == value:
            count += 1
    pairs.append((count, value))

pairs.sort(reverse=True)
return [value for _, value in pairs[:k]]

```

Best solution

Count once with a dictionary, then use bucket sort because frequency cannot exceed n .

Best solution

```

from collections import Counter

def top_k_frequent_best(nums, k):
    counts = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]

    for value, freq in counts.items():
        buckets[freq].append(value)

    result = []
    for freq in range(len(buckets) - 1, 0, -1):
        for value in buckets[freq]:
            result.append(value)
            if len(result) == k:
                return result
    return result

```

9.3 Longest Consecutive Sequence

Problem statement

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [100, 4, 200, 1, 3, 2] -> 4`
- `nums = [0, 3, 7, 2, 5, 8, 4, 6, 0, 1] -> 9`

Brute-force: Time $O(n \log n)$, space $O(n)$

Best solution: Time $O(n)$, space $O(n)$

Brute-force idea

Sorting makes consecutive runs easy to count, even though it costs $O(n \log n)$.

Brute-force solution

```
def longest_consecutive_bruteforce(nums):
    if not nums:
        return 0

    nums = sorted(set(nums))
    best = 1
    current = 1

    for i in range(1, len(nums)):
        if nums[i] == nums[i - 1] + 1:
            current += 1
        else:
            current = 1
            best = max(best, current)

    return best
```

Best solution

A set lets you start only at true sequence beginnings and grow each run in $O(1)$ average lookup time.

Best solution

```
def longest_consecutive_best(nums):
    num_set = set(nums)
    best = 0

    for num in num_set:
        if num - 1 not in num_set:
            length = 1
            while num + length in num_set:
                length += 1
            best = max(best, length)

    return best
```

9.4 Valid Sudoku

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/valid-sudoku/>

Problem statement

Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- board with no duplicates in any row, column, or 3x3 box -> True
- board with duplicate 8 in a row -> False

Brute-force: $O(9^2)$, space $O(9^2)$

Best solution: $O(9^2)$, space $O(9^2)$

Brute-force idea

A direct baseline checks each row, each column, and each box separately with a set.

Brute-force solution

```
def is_valid_sudoku_bruteforce(board):
    for row in board:
        seen = set()
        for value in row:
            if value != '.':
                if value in seen:
                    return False
                seen.add(value)

    for c in range(9):
        seen = set()
        for r in range(9):
            value = board[r][c]
            if value != '.':
                if value in seen:
                    return False
                seen.add(value)

    for sr in range(0, 9, 3):
        for sc in range(0, 9, 3):
            seen = set()
            for r in range(sr, sr + 3):
```

```

        for c in range(sc, sc + 3):
            value = board[r][c]
            if value != '.':
                if value in seen:
                    return False
                seen.add(value)
    return True

```

Best solution

A compact hash-based solution records row, column, and box signatures as you scan each filled cell once.

Best solution

```

def is_valid_sudoku_best(board):
    seen = set()
    for r in range(9):
        for c in range(9):
            value = board[r][c]
            if value == '.':
                continue
            row_key = ('row', r, value)
            col_key = ('col', c, value)
            box_key = ('box', r // 3, c // 3, value)
            if row_key in seen or col_key in seen or box_key in seen:
                return False
            seen.add(row_key)
            seen.add(col_key)
            seen.add(box_key)
    return True

```

9.5 Intersection of Two Arrays II

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/intersection-of-two-arrays-ii/>

Problem statement

Given two integer arrays `nums1` and `nums2`, return an array of their intersection, including duplicates.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums1 = [1,2,2,1], nums2 = [2,2] -> [2,2]`
- `nums1 = [4,9,5], nums2 = [9,4,9,8,4] -> [4,9]` in any order

Brute-force: $O(nm)$, space $O(1)$ **Best solution:** $O(n+m)$, space $O(\min(n,m))$ **Brute-force idea**

A basic baseline checks each value in one array against a still-unused value in the other.

Brute-force solution

```
def intersect_arrays_bruteforce(nums1, nums2):
    used = [False] * len(nums2)
    result = []
    for x in nums1:
        for i, y in enumerate(nums2):
            if not used[i] and x == y:
                result.append(x)
                used[i] = True
                break
    return result
```

Best solution

A frequency map from the smaller array lets you build the intersection in linear time.

Best solution

```
def intersect_arrays_best(nums1, nums2):
    counts = {}
    for num in nums1:
        counts[num] = counts.get(num, 0) + 1

    result = []
    for num in nums2:
        if counts.get(num, 0) > 0:
            result.append(num)
            counts[num] -= 1
    return result
```

9.6 Insert Delete GetRandom O(1)

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/insert-delete-getrandom-o1/>

Problem statement

Design a data structure that supports insert, remove, and getRandom in average $O(1)$ time.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- insert(1) -> True, remove(2) -> False, insert(2) -> True
- getRandom() returns either existing element uniformly

Brute-force: insert $O(1)$, getRandom $O(1)$, remove $O(n)$ **Best solution:** all average $O(1)$

Brute-force idea

A simple baseline can store values in a list. That makes random access easy, but removal has to search for the item.

Brute-force solution

```
import random

class RandomizedSetBruteforce:
    def __init__(self):
        self.values = []

    def insert(self, val):
        if val in self.values:
            return False
        self.values.append(val)
        return True

    def remove(self, val):
        if val not in self.values:
            return False
        self.values.remove(val)
        return True

    def getRandom(self):
        return random.choice(self.values)
```

Best solution

The optimized design combines a list with a value-to-index map. To remove in $O(1)$, swap the target with the last element before popping.

Best solution

```
import random

class RandomizedSetBest:
    def __init__(self):
        self.values = []
        self.index = {}

    def insert(self, val):
        if val in self.index:
            return False
        self.index[val] = len(self.values)
        self.values.append(val)
        return True

    def remove(self, val):
        if val not in self.index:
            return False
        idx = self.index[val]
        last = self.values[-1]
        self.values[idx] = last
        self.index[last] = idx
        self.values.pop()
        del self.index[val]
        return True

    def getRandom(self):
        return random.choice(self.values)
```

9.7 Group Anagrams (Hashing Focus)

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/group-anagrams/>

Problem statement

Group words that are anagrams of each other.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `strs = ['eat','tea','tan','ate','nat','bat']` -> grouped anagrams
- `strs = ["]` -> `[[]]`

Brute-force: $O(n \cdot k \cdot \log k)$ **Best solution:** $O(nk)$, space $O(nk)$ **Brute-force idea**

A baseline sorts each word to build the grouping key.

Brute-force solution

```
def group_anagrams_hash_bruteforce(strs):
    groups = {}
    for word in strs:
        key = "".join(sorted(word))
        groups.setdefault(key, []).append(word)
    return list(groups.values())
```

Best solution

A letter-count tuple avoids sorting each word and becomes the hash key instead.

Best solution

```
def group_anagrams_hash_best(strs):
    groups = {}
    for word in strs:
        count = [0] * 26
        for ch in word:
            count[ord(ch) - 97] += 1
        groups.setdefault(tuple(count), []).append(word)
    return list(groups.values())
```

9.8 Longest Consecutive Sequence (Set Focus)

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/longest-consecutive-sequence/>

Problem statement

Given an unsorted array of integers, return the length of the longest consecutive elements sequence.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [100,4,200,1,3,2]` -> 4
- `nums = [0,3,7,2,5,8,4,6,0,1]` -> 9

Brute-force: $O(n \log n)$

Best solution: $O(n)$, space $O(n)$

Brute-force idea

Sorting gives a correct baseline, then you scan for the longest run.

Brute-force solution

```
def longest_consecutive_set_bruteforce(nums):
    if not nums:
        return 0
    nums = sorted(set(nums))
    best = current = 1
    for i in range(1, len(nums)):
        if nums[i] == nums[i - 1] + 1:
            current += 1
        else:
            current = 1
        best = max(best, current)
    return best
```

Best solution

The set-based trick only starts growing a sequence from numbers that have no predecessor.

Best solution

```
def longest_consecutive_set_best(nums):
    values = set(nums)
    best = 0
    for num in values:
        if num - 1 not in values:
            length = 1
            while num + length in values:
                length += 1
            best = max(best, length)
```

```
return best
```

9.9 Top K Frequent Elements (Hashing Focus)

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/top-k-frequent-elements/>

Problem statement

Return the k most frequent elements in the array.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1,1,1,2,2,3]`, `k = 2` -> `[1,2]`
- `nums = [1]`, `k = 1` -> `[1]`

Brute-force: $O(n \log n)$

Best solution: $O(n)$, space $O(n)$

Brute-force idea

A baseline counts frequencies and sorts by count descending.

Brute-force solution

```
def top_k_frequent_hash_bruteforce(nums, k):
    counts = {}
    for num in nums:
        counts[num] = counts.get(num, 0) + 1
    ordered = sorted(counts.items(), key=lambda item: item[1], reverse=True)
    return [num for num, _ in ordered[:k]]
```

Best solution

Bucket sort uses the fact that a frequency cannot exceed n, so you can place numbers into buckets by count.

Best solution

```
def top_k_frequent_hash_best(nums, k):
    counts = {}
    for num in nums:
        counts[num] = counts.get(num, 0) + 1

    buckets = [[] for _ in range(len(nums) + 1)]
    for num, freq in counts.items():
        buckets[freq].append(num)

    result = []
    for freq in range(len(buckets) - 1, -1, -1):
        for num in buckets[freq]:
            result.append(num)
            if len(result) == k:
                return result
```

Trees

Chapter overview

Tree problems feel scary until you get used to one core idea: every node can ask its children for help. Once that clicks, traversals and path-based questions become much more manageable.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

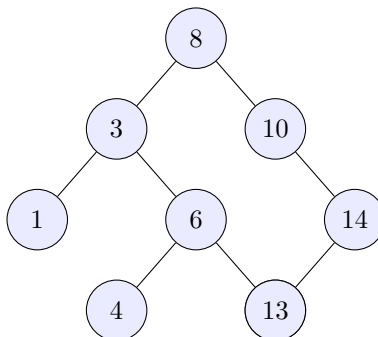
Trees primer

This chapter opening folds in the handbook theory for trees so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Trees look intimidating until you remember that every node defines a smaller tree. Once that clicks, traversal and recursion start to feel natural.

Binary tree picture

Tree problems usually become easier the moment you sketch the shape.



Traversals

Preorder visits root first. Inorder visits left, then root, then right. Postorder visits children before the root. Breadth-first traversal works level by level.

```
def inorder(root):
    if not root:
        return []
    return inorder(root.left) + [root.val] + inorder(root.right)
```

BST search

In a binary search tree, the left subtree is smaller and the right subtree is larger. That ordering gives you directed search.

```
def bst_search(root, target):
    current = root
    while current:
        if current.val == target:
            return current
        if target < current.val:
            current = current.left
        else:
            current = current.right
    return None
```

10.1 Maximum Depth of Binary Tree

Problem statement

Given the root of a binary tree, return its maximum depth.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- A tree with root 3 and children 9, 20, where 20 has children 15 and 7 -> 3
- Empty tree -> 0

Brute-force: Time $O(n^2)$ in skewed trees, space $O(h)$ **Best solution:** Time $O(n)$, space $O(h)$

Brute-force idea

A deliberately naive version recomputes subtree depths again and again while exploring each node.

Brute-force solution

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
```

```

        self.val = val
        self.left = left
        self.right = right

def _depth(node):
    if not node:
        return 0
    return 1 + max(_depth(node.left), _depth(node.right))

def max_depth_bruteforce(root):
    if not root:
        return 0
    left_depth = _depth(root.left)
    right_depth = _depth(root.right)
    return 1 + max(left_depth, right_depth)

```

Best solution

One DFS can return 1 plus the larger of the left and right depths.

Best solution

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def max_depth_best(root):
    if not root:
        return 0
    return 1 + max(max_depth_best(root.left), max_depth_best(root.right))

```

10.2 Validate Binary Search Tree

Problem statement

Given the root of a binary tree, determine whether it is a valid binary search tree. Every node must be strictly greater than all nodes in its left subtree and strictly smaller than all nodes in its right subtree.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- [2, 1, 3] -> True
- [5, 1, 4, null, null, 3, 6] -> False

Brute-force: Time $O(n^2)$, space $O(h)$

Best solution: Time $O(n)$, space $O(h)$

Brute-force idea

The brute-force approach checks each node against the max of its left subtree and the min of its right subtree.

Brute-force solution

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def _subtree_max(node):
    if not node:
        return float("-inf")
    return max(node.val, _subtree_max(node.left), _subtree_max(node.right))

def _subtree_min(node):
    if not node:
        return float("inf")
    return min(node.val, _subtree_min(node.left), _subtree_min(node.right))

def validate_bst_bruteforce(root):
    if not root:
        return True
    if _subtree_max(root.left) >= root.val:
        return False
    if _subtree_min(root.right) <= root.val:
        return False
    return validate_bst_bruteforce(root.left) and validate_bst_bruteforce(root.right)
```

Best solution

Passing allowable min and max bounds down the recursion avoids repeated work and catches deep violations cleanly.

Best solution

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def validate_bst_best(root):
    def dfs(node, low, high):
        if not node:
            return True
        if not (low < node.val < high):
            return False
```

```

        return dfs(node.left, low, node.val) and dfs(node.right, node.val, high)

    return dfs(root, float("-inf"), float("inf"))

```

10.3 Binary Tree Level Order Traversal

Problem statement

Given the root of a binary tree, return the level order traversal of its nodes' values from left to right, level by level.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- [3, 9, 20, null, null, 15, 7] -> [[3], [9, 20], [15, 7]]
- [] -> []

Brute-force: Time $O(n^2)$, space $O(n)$

Best solution: Time $O(n)$, space $O(n)$

Brute-force idea

A depth-collection approach can keep recalculating tree height and scan each level separately.

Brute-force solution

```

from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def _height(node):
    if not node:
        return 0
    return 1 + max(_height(node.left), _height(node.right))

def _collect_level(node, level, bucket):
    if not node:
        return
    if level == 1:
        bucket.append(node.val)
        return
    _collect_level(node.left, level - 1, bucket)
    _collect_level(node.right, level - 1, bucket)

```

```
def level_order_bruteforce(root):
    result = []
    for level in range(1, _height(root) + 1):
        bucket = []
        _collect_level(root, level, bucket)
        result.append(bucket)
    return result
```

Best solution

A queue processes one level at a time naturally, which is why BFS is the clean go-to here.

Best solution

```
from collections import deque

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def level_order_best(root):
    if not root:
        return []

    result = []
    queue = deque([root])

    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)

    return result
```

10.4 Binary Tree Maximum Path Sum

Problem statement

A path may start and end at any nodes in the tree, but it must move through parent-child links. Return the maximum path sum.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- [1, 2, 3] -> 6
- [-10, 9, 20, null, null, 15, 7] -> 42

Brute-force: Time $O(n^2)$, space $O(h)$ **Best solution:** Time $O(n)$, space $O(h)$

Brute-force idea

A naive strategy recomputes the best downward gain for many nodes over and over.

Brute-force solution

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def _best_downward(node):
    if not node:
        return 0
    return max(0, node.val + max(_best_downward(node.left), _best_downward(node.right)))

def max_path_sum_bruteforce(root):
    def visit(node):
        if not node:
            return float("-inf")
        through_node = node.val + _best_downward(node.left) + _best_downward(node.right)
        return max(through_node, visit(node.left), visit(node.right))

    return visit(root)
```

Best solution

A post-order DFS returns the best downward path from each node while updating a global answer for split paths.

Best solution

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def max_path_sum_best(root):
    best = float("-inf")

    def dfs(node):
```

```

    nonlocal best
    if not node:
        return 0

    left_gain = max(dfs(node.left), 0)
    right_gain = max(dfs(node.right), 0)

    best = max(best, node.val + left_gain + right_gain)
    return node.val + max(left_gain, right_gain)

dfs(root)
return best

```

10.5 Invert Binary Tree

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/invert-binary-tree/>

Problem statement

Given the root of a binary tree, invert the tree and return its root.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- root = [4,2,7,1,3,6,9] -> [4,7,2,9,6,3,1]
- root = [] -> []

Brute-force: $O(n)$, space $O(n)$

Best solution: $O(n)$, space $O(h)$

Brute-force idea

A level-order baseline visits every node with a queue and swaps its children.

Brute-force solution

```

from collections import deque

def invert_tree_bruteforce(root):
    if not root:
        return root
    queue = deque([root])

```

```

while queue:
    node = queue.popleft()
    node.left, node.right = node.right, node.left
    if node.left:
        queue.append(node.left)
    if node.right:
        queue.append(node.right)
return root

```

Best solution

The recursive version is the clean interview answer: invert the left subtree, invert the right subtree, then swap them.

Best solution

```

def invert_tree_best(root):
    if not root:
        return None
    root.left, root.right = invert_tree_best(root.right), invert_tree_best(root.left)
    return root

```

10.6 Same Tree

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/same-tree/>

Problem statement

Given the roots of two binary trees p and q, return true if they are the same tree.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- p = [1,2,3], q = [1,2,3] -> True
- p = [1,2], q = [1,None,2] -> False

Brute-force: $O(n)$, space $O(n)$

Best solution: $O(n)$, space $O(h)$

Brute-force idea

A queue-based baseline compares the trees level by level.

Brute-force solution

```
from collections import deque

def is_same_tree_bruteforce(p, q):
    queue = deque([(p, q)])
    while queue:
        a, b = queue.popleft()
        if not a and not b:
            continue
        if not a or not b or a.val != b.val:
            return False
        queue.append((a.left, b.left))
        queue.append((a.right, b.right))
    return True
```

Best solution

The recursive solution mirrors the tree definition itself: values must match, and both left and right subtrees must match.

Best solution

```
def is_same_tree_best(p, q):
    if not p and not q:
        return True
    if not p or not q or p.val != q.val:
        return False
    return is_same_tree_best(p.left, q.left) and is_same_tree_best(p.right, q.right)
```

10.7 Lowest Common Ancestor of a Binary Search Tree

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/lowest-common-ancestor-of-a-binary-search-tree/>

Problem statement

Given a BST and two nodes p and q, return their lowest common ancestor.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- root = [6,2,8,0,4,7,9], p = 2, q = 8 -> 6
- root = [6,2,8,0,4,7,9], p = 2, q = 4 -> 2

Brute-force: $O(n)$, space $O(n)$ **Best solution:** $O(h)$, space $O(1)$ **Brute-force idea**

A generic baseline finds the path from the root to each target and then compares the paths.

Brute-force solution

```
def lowest_common_ancestor_bst_bruteforce(root, p, q):
    def path_to(node, target, path):
        if not node:
            return False
        path.append(node)
        if node is target:
            return True
        if path_to(node.left, target, path) or path_to(node.right, target, path):
            return True
        path.pop()
        return False

    path_p, path_q = [], []
    path_to(root, p, path_p)
    path_to(root, q, path_q)

    answer = None
    for a, b in zip(path_p, path_q):
        if a is b:
            answer = a
        else:
            break
    return answer
```

Best solution

In a BST, if both targets are smaller, go left. If both are larger, go right. The split point is the answer.

Best solution

```
def lowest_common_ancestor_bst_best(root, p, q):
    node = root
    while node:
        if p.val < node.val and q.val < node.val:
            node = node.left
        elif p.val > node.val and q.val > node.val:
            node = node.right
```

```

else:
    return node

```

10.8 Construct Binary Tree from Preorder and Inorder Traversal

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/>

Problem statement

Given preorder and inorder traversal arrays of a binary tree, construct and return the tree.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
- preorder = [-1], inorder = [-1]

Brute-force: $O(n^2)$, space $O(n)$

Best solution: $O(n)$, space $O(n)$

Brute-force idea

The straightforward version repeatedly finds the root in the inorder list by scanning linearly.

Brute-force solution

```

def build_tree_bruteforce(preorder, inorder):
    if not preorder or not inorder:
        return None

    root_val = preorder[0]
    mid = inorder.index(root_val)

    root = TreeNode(root_val)
    root.left = build_tree_bruteforce(preorder[1:1 + mid], inorder[:mid])
    root.right = build_tree_bruteforce(preorder[1 + mid:], inorder[mid + 1:])
    return root

```

Best solution

A hash map from inorder value to index removes repeated scans and makes the recursion linear overall.

Best solution

```
def build_tree_best(preorder, inorder):
    index = {value: i for i, value in enumerate(inorder)}
    preorder_idx = 0

    def helper(left, right):
        nonlocal preorder_idx
        if left > right:
            return None

        root_val = preorder[preorder_idx]
        preorder_idx += 1
        root = TreeNode(root_val)

        mid = index[root_val]
        root.left = helper(left, mid - 1)
        root.right = helper(mid + 1, right)
        return root

    return helper(0, len(inorder) - 1)
```

10.9 Serialize and Deserialize Binary Tree

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/serialize-and-deserialize-binary-tree/>

Problem statement

Design an algorithm to serialize a binary tree to a string and deserialize the string back to the original tree.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- A tree like [1,2,3,None,None,4,5] should round-trip correctly
- An empty tree should serialize and deserialize correctly

Brute-force: $O(n)$, space $O(n)$

Best solution: $O(n)$, space $O(n)$

Brute-force idea

A level-order baseline writes node values and null markers into a comma-separated string, then rebuilds level by level.

Brute-force solution

```
from collections import deque

def serialize_bruteforce(root):
    if not root:
        return ""
    queue = deque([root])
    out = []
    while queue:
        node = queue.popleft()
        if not node:
            out.append("null")
            continue
        out.append(str(node.val))
        queue.append(node.left)
        queue.append(node.right)
    return ",".join(out)

def deserialize_bruteforce(data):
    if not data:
        return None
    values = data.split(",")
    root = TreeNode(int(values[0]))
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if values[i] != "null":
            node.left = TreeNode(int(values[i]))
            queue.append(node.left)
        i += 1
        if i < len(values) and values[i] != "null":
            node.right = TreeNode(int(values[i]))
            queue.append(node.right)
        i += 1
    return root
```

Best solution

A preorder DFS with null markers is compact and easy to rebuild because it preserves the exact tree shape.

Best solution

```
def serialize_best(root):
    out = []

    def dfs(node):
        if not node:
            out.append("null")
```

```

        return
    out.append(str(node.val))
    dfs(node.left)
    dfs(node.right)

dfs(root)
return ", ".join(out)

def deserialize_best(data):
    values = iter(data.split(","))

    def dfs():
        value = next(values)
        if value == "null":
            return None
        node = TreeNode(int(value))
        node.left = dfs()
        node.right = dfs()
        return node

    return dfs()

```

10.10 Diameter of Binary Tree

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/diameter-of-binary-tree/>

Problem statement

Given the root of a binary tree, return the length of the diameter of the tree.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- root = [1,2,3,4,5] -> 3
- root = [1,2] -> 1

Brute-force: $O(n^2)$, space $O(h)$

Best solution: $O(n)$, space $O(h)$

Brute-force idea

A direct baseline computes the height of each subtree separately for every node and takes the best left-height + right-height value.

Brute-force solution

```
def diameter_of_binary_tree_bruteforce(root):
    def height(node):
        if not node:
            return 0
        return 1 + max(height(node.left), height(node.right))

    if not root:
        return 0
    through_root = height(root.left) + height(root.right)
    return max(
        through_root,
        diameter_of_binary_tree_bruteforce(root.left),
        diameter_of_binary_tree_bruteforce(root.right),
    )
```

Best solution

A single DFS can return subtree heights while updating the best diameter seen so far.

Best solution

```
def diameter_of_binary_tree_best(root):
    best = 0

    def dfs(node):
        nonlocal best
        if not node:
            return 0
        left = dfs(node.left)
        right = dfs(node.right)
        best = max(best, left + right)
        return 1 + max(left, right)

    dfs(root)
    return best
```

Heaps and Priority Queues

Chapter overview

Heaps shine when you care about the current smallest or largest item and the data keeps changing. They are especially useful for Top K, scheduling, and stream problems.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

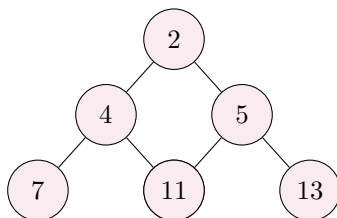
Heaps and Priority Queues primer

This chapter opening folds in the handbook theory for heaps and priority queues so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Heaps help when you need repeated access to the smallest or largest item without sorting everything upfront.

Heap picture

A heap is not fully sorted. It only guarantees that each parent is ordered relative to its children.



Python heapq

Python gives you a min-heap, which is enough for a lot of interview problems.

```
import heapq

def k_largest(nums, k):
    heap = []
    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)
    return sorted(heap, reverse=True)
```

Streaming median idea

Two heaps let you keep the lower half and upper half balanced as numbers arrive.

11.1 K Closest Points to Origin

Problem statement

Given an array of points where $\text{points}[i] = [x, y]$, return the k points closest to the origin.
Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $\text{points} = [[1, 3], [-2, 2]]$, $k = 1 \rightarrow [[-2, 2]]$
- $\text{points} = [[3, 3], [5, -1], [-2, 4]]$, $k = 2 \rightarrow$ any two closest points

Brute-force: Time $O(n \log n)$, space $O(n)$

Best solution: Time $O(n \log k)$, space $O(k)$

Brute-force idea

Sort all points by squared distance and take the first k .

Brute-force solution

```
def k_closest_bruteforce(points, k):
    points = sorted(points, key=lambda p: p[0] * p[0] + p[1] * p[1])
    return points[:k]
```

Best solution

A max-heap of size k keeps only the k closest points seen so far.

Best solution

```
import heapq

def k_closest_best(points, k):
    heap = []
    for x, y in points:
        distance = x * x + y * y
        heapq.heappush(heap, (-distance, [x, y]))
        if len(heap) > k:
            heapq.heappop(heap)
    return [point for _, point in heap]
```

11.2 Task Scheduler

Problem statement

You are given tasks represented by letters and a non-negative cooldown n . The same task must be separated by at least n units. Return the least number of time units required to finish all tasks.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- tasks = ['A', 'A', 'A', 'B', 'B', 'B'], $n = 2 \rightarrow 8$
- tasks = ['A', 'C', 'A', 'B', 'D', 'B'], $n = 1 \rightarrow 6$

Brute-force: Time $O(T * U)$, space $O(U)$ **Best solution:** Time $O(U)$, space $O(U)$, where U is number of unique tasks

Brute-force idea

A simulation can schedule one slot at a time, repeatedly picking the currently allowed task with the largest remaining count.

Brute-force solution

```
from collections import Counter

def task_scheduler_bruteforce(tasks, n):
    counts = Counter(tasks)
    cooldown = {}
    time = 0

    while counts:
```

```

available = [task for task in counts if cooldown.get(task, -1) <= time]
if available:
    task = max(available, key=lambda t: counts[t])
    counts[task] -= 1
    if counts[task] == 0:
        del counts[task]
    cooldown[task] = time + n + 1
    time += 1

return time

```

Best solution

The counting formula based on the most frequent task avoids full simulation and is the clean interview finish.

Best solution

```

from collections import Counter

def task_scheduler_best(tasks, n):
    counts = Counter(tasks).values()
    max_freq = max(counts)
    max_count = sum(1 for freq in counts if freq == max_freq)
    frame = (max_freq - 1) * (n + 1) + max_count
    return max(len(tasks), frame)

```

11.3 Find Median from Data Stream

Problem statement

Design a data structure that supports adding integers from a stream and returning the median at any time.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- addNum(1), addNum(2), findMedian() -> 1.5
- addNum(3), findMedian() -> 2.0

Brute-force: addNum $O(1)$, findMedian $O(n \log n)$
findMedian $O(1)$

Best solution: addNum $O(\log n)$,

Brute-force idea

The easy version stores all numbers and sorts a fresh copy every time median is requested.

Brute-force solution

```
class MedianFinderBruteforce:
    def __init__(self):
        self.data = []

    def addNum(self, num):
        self.data.append(num)

    def findMedian(self):
        values = sorted(self.data)
        n = len(values)
        mid = n // 2
        if n % 2 == 1:
            return float(values[mid])
        return (values[mid - 1] + values[mid]) / 2.0
```

Best solution

Two heaps keep the lower half and upper half balanced, so the median is always sitting at the top.

Best solution

```
import heapq

class MedianFinderBest:
    def __init__(self):
        self.small = []
        self.large = []

    def addNum(self, num):
        heapq.heappush(self.small, -num)

        if self.large and -self.small[0] > self.large[0]:
            heapq.heappush(self.large, -heapq.heappop(self.small))

        if len(self.small) > len(self.large) + 1:
            heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self):
        if len(self.small) > len(self.large):
            return float(-self.small[0])
        return (-self.small[0] + self.large[0]) / 2.0
```

11.4 Kth Largest Element in a Stream

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/kth-largest-element-in-a-stream/>

Problem statement

Design a class to find the kth largest element in a stream of numbers.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- k = 3, nums = [4,5,8,2]; add(3) -> 4
- then add(10) -> 5

Brute-force: $O(n \log n)$ setup, $O(\log n)$ add with full sort **Best solution:** $O(n \log k)$ setup, $O(\log k)$ add

Brute-force idea

A simple baseline stores all values and sorts on every add, which is easy to reason about but wasteful.

Brute-force solution

```
class KthLargestBruteforce:
    def __init__(self, k, nums):
        self.k = k
        self.values = nums[:]

    def add(self, val):
        self.values.append(val)
        self.values.sort(reverse=True)
        return self.values[self.k - 1]
```

Best solution

A min-heap of size k keeps exactly the k largest elements seen so far. The heap root is the kth largest.

Best solution

```
import heapq
```

```

class KthLargestBest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = nums[:]
        heapq.heapify(self.heap)
        while len(self.heap) > k:
            heapq.heappop(self.heap)

    def add(self, val):
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, val)
        elif val > self.heap[0]:
            heapq.heapreplace(self.heap, val)
        return self.heap[0]

```

11.5 Last Stone Weight

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/last-stone-weight/>

Problem statement

We repeatedly smash the two heaviest stones together. Return the weight of the last stone, or 0 if none remain.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- stones = [2,7,4,1,8,1] -> 1
- stones = [1] -> 1

Brute-force: $O(n^2 \log n)$, space $O(n)$

Best solution: $O(n \log n)$, space $O(n)$

Brute-force idea

A baseline repeatedly sorts the stones, removes the two largest, and inserts any remaining difference.

Brute-force solution

```

def last_stone_weight_bruteforce(stones):
    stones = stones[:]

```

```

while len(stones) > 1:
    stones.sort()
    y = stones.pop()
    x = stones.pop()
    if y != x:
        stones.append(y - x)
return stones[0] if stones else 0

```

Best solution

A max-heap makes it cheap to pull the two heaviest stones every round.

Best solution

```

import heapq

def last_stone_weight_best(stones):
    heap = [-stone for stone in stones]
    heapq.heapify(heap)

    while len(heap) > 1:
        y = -heapq.heappop(heap)
        x = -heapq.heappop(heap)
        if y != x:
            heapq.heappush(heap, -(y - x))
    return -heap[0] if heap else 0

```

11.6 Kth Largest Element in an Array

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/kth-largest-element-in-an-array/>

Problem statement

Given an integer array `nums` and an integer `k`, return the `k`th largest element.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [3,2,1,5,6,4]`, `k = 2` -> 5
- `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4` -> 4

Brute-force: $O(n \log n)$, space $O(1)$ **Best solution:** $O(n \log k)$, space $O(k)$

Brute-force idea

The direct baseline sorts the whole array and indexes the k th largest value.

Brute-force solution

```
def find_kth_largest_bruteforce(nums, k):
    nums.sort(reverse=True)
    return nums[k - 1]
```

Best solution

A size- k min-heap avoids sorting everything when you only care about the top k values.

Best solution

```
import heapq

def find_kth_largest_best(nums, k):
    heap = []
    for num in nums:
        if len(heap) < k:
            heapq.heappush(heap, num)
        elif num > heap[0]:
            heapq.heapreplace(heap, num)
    return heap[0]
```

11.7 Design Twitter

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/design-twitter/>

Problem statement

Design a simplified Twitter where users can post tweets, follow or unfollow others, and get the 10 most recent tweet ids in their news feed.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `postTweet(1,5); getNewsFeed(1) -> [5]`
- `follow(1,2); postTweet(2,6); getNewsFeed(1) -> [6,5]`

Brute-force: post $O(1)$, feed $O(T \log T)$ with sorting
 feed $O((F+1) \log F)$

Best solution: post $O(1)$,

Brute-force idea

A simple baseline stores all tweets globally and filters plus sorts them whenever a feed is requested.

Brute-force solution

```
class TwitterBruteforce:
    def __init__(self):
        self.time = 0
        self.tweets = []
        self.following = {}

    def postTweet(self, userId, tweetId):
        self.time += 1
        self.tweets.append((self.time, userId, tweetId))

    def getNewsFeed(self, userId):
        allowed = self.following.get(userId, set()) | {userId}
        feed = [tweet_id for _, uid, tweet_id in sorted(self.tweets, reverse=True) if uid in
        allowed]
        return feed[:10]

    def follow(self, followerId, followeeId):
        self.following.setdefault(followerId, set()).add(followeeId)

    def unfollow(self, followerId, followeeId):
        self.following.setdefault(followerId, set()).discard(followeeId)
```

Best solution

A better design stores each user's tweets in reverse chronological order and uses a heap to merge the newest tweet streams.

Best solution

```
import heapq

class TwitterBest:
    def __init__(self):
        self.time = 0
        self.tweets = {}
        self.following = {}
```

```

def postTweet(self, userId, tweetId):
    self.time += 1
    self.tweets.setdefault(userId, []).append((self.time, tweetId))

def getNewsFeed(self, userId):
    users = self.following.get(userId, set()) | {userId}
    heap = []
    for uid in users:
        tweets = self.tweets.get(uid, [])
        if tweets:
            idx = len(tweets) - 1
            time, tweet_id = tweets[idx]
            heapq.heappush(heap, (-time, tweet_id, uid, idx - 1))

    feed = []
    while heap and len(feed) < 10:
        neg_time, tweet_id, uid, idx = heapq.heappop(heap)
        feed.append(tweet_id)
        if idx >= 0:
            time, next_tweet = self.tweets[uid][idx]
            heapq.heappush(heap, (-time, next_tweet, uid, idx - 1))
    return feed

def follow(self, followerId, followeeId):
    self.following.setdefault(followerId, set()).add(followeeId)

def unfollow(self, followerId, followeeId):
    self.following.setdefault(followerId, set()).discard(followeeId)

```

11.8 Merge K Sorted Lists (Heap Focus)

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/merge-k-sorted-lists/>

Problem statement

Revisit Merge K Sorted Lists through the priority-queue lens.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `lists = [[1,4,5],[1,3,4],[2,6]]` -> merged sorted list
- `lists = []` -> `[]`

Brute-force: $O(N \log N)$ **Best solution:** $O(N \log k)$, space $O(k)$

Brute-force idea

A simple baseline sorts all values after collecting them from every list.

Brute-force solution

```
def merge_k_lists_heap_bruteforce(lists):
    values = []
    for node in lists:
        while node:
            values.append(node.val)
            node = node.next
    values.sort()
    dummy = ListNode(0)
    tail = dummy
    for value in values:
        tail.next = ListNode(value)
        tail = tail.next
    return dummy.next
```

Best solution

The heap-oriented view pushes only the current head of each list, which keeps the frontier tiny.

Best solution

```
import heapq

def merge_k_lists_heap_best(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
    dummy = ListNode(0)
    tail = dummy
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    tail.next = None
    return dummy.next
```

Graphs

Chapter overview

Graph problems become much less intimidating once you decide what the edges mean and whether you need reachability, ordering, or shortest path. BFS and DFS do a lot of the heavy lifting.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

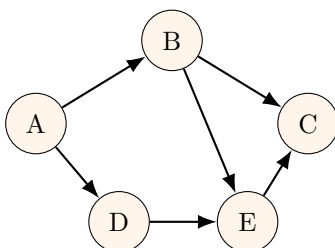
Graphs primer

This chapter opening folds in the handbook theory for graphs so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Graphs are where DSA starts to feel like modeling the real world: routes, friendships, dependencies, networks, states, and transitions all fit naturally here.

Graph picture

Nodes are entities. Edges are relationships.



BFS

Breadth-first search expands level by level, which makes it perfect for shortest path in an unweighted graph.

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    order = []
    queue = deque([start])
    while queue:
        node = queue.popleft()
        order.append(node)
        for nei in graph[node]:
            if nei not in visited:
                visited.add(nei)
                queue.append(nei)
    return order
```

DFS

Depth-first search dives deep before backing up. It is great for connectivity, components, cycle detection, and backtracking-style exploration.

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    for nei in graph[start]:
        if nei not in visited:
            dfs(graph, nei, visited)
    return visited
```

Topological sort

When a graph is a DAG, topological sort gives you a valid dependency order.

12.1 Number of Islands

Problem statement

Given a 2D grid of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands vertically or horizontally.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- A grid with one large land mass -> 1
- A grid with three separate land masses -> 3

Brute-force: Time $O((mn)^2)$, space $O(mn)$ **Best solution:** Time $O(mn)$, space $O(mn)$

Brute-force idea

A really slow version can label each component by scanning neighbors repeatedly until no more changes happen.

Brute-force solution

```
def num_islands_bruteforce(grid):
    rows, cols = len(grid), len(grid[0])
    labels = [[0] * cols for _ in range(rows)]
    island_id = 0

    changed = True
    while changed:
        changed = False
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == '1' and labels[r][c] == 0:
                    island_id += 1
                    labels[r][c] = island_id
                    changed = True
                if grid[r][c] == '1':
                    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < rows and 0 <= nc < cols and labels[nr][nc] != 0:
                            if labels[r][c] == 0 or labels[r][c] > labels[nr][nc]:
                                labels[r][c] = labels[nr][nc]
                                changed = True

    return len({labels[r][c] for r in range(rows) for c in range(cols) if labels[r][c]})
```

Best solution

Just DFS or BFS from each unvisited land cell and mark the whole island at once.

Best solution

```
def num_islands_best(grid):
    rows, cols = len(grid), len(grid[0])
    visited = set()

    def dfs(r, c):
        if (
            r < 0 or r >= rows or
            c < 0 or c >= cols or
            grid[r][c] != '1' or
            (r, c) in visited
        ):
            return
        visited.add((r, c))
        dfs(r + 1, c)
```

```

        dfs(r - 1, c)
        dfs(r, c + 1)
        dfs(r, c - 1)

    islands = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1' and (r, c) not in visited:
                islands += 1
                dfs(r, c)

    return islands

```

12.2 Course Schedule

Problem statement

There are numCourses labeled from 0 to numCourses - 1. You are given prerequisite pairs [a, b] meaning you must take b before a. Return True if you can finish all courses.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- numCourses = 2, prerequisites = [[1, 0]] -> True
- numCourses = 2, prerequisites = [[1, 0], [0, 1]] -> False

Brute-force: Time $O(V * (V+E))$, space $O(V+E)$

Best solution: Time $O(V+E)$, space $O(V+E)$

Brute-force idea

A naive strategy repeatedly picks courses whose prerequisites are currently satisfied until progress stops.

Brute-force solution

```

def course_schedule_bruteforce(num_courses, prerequisites):
    remaining = set(range(num_courses))
    finished = set()

    changed = True
    while changed:
        changed = False
        for course in list(remaining):
            needed = [pre for pre, nxt in prerequisites if nxt == course]
            if all(pre in finished for pre in needed):

```

```

        finished.add(course)
        remaining.remove(course)
        changed = True

    return not remaining

```

Best solution

Topological sort with indegrees is the clean standard answer.

Best solution

```

from collections import defaultdict, deque

def course_schedule_best(num_courses, prerequisites):
    graph = defaultdict(list)
    indegree = [0] * num_courses

    for course, prereq in prerequisites:
        graph[prereq].append(course)
        indegree[course] += 1

    queue = deque([i for i in range(num_courses) if indegree[i] == 0])
    taken = 0

    while queue:
        course = queue.popleft()
        taken += 1
        for nxt in graph[course]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)

    return taken == num_courses

```

12.3 Alien Dictionary

Problem statement

You are given a sorted list of words from an unknown alphabet. Return a valid ordering of the letters. If no valid ordering exists, return an empty string.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- ['wrt', 'wrf', 'er', 'ett', 'rftt'] -> 'wertf'
- ['z', 'x', 'z'] -> ''

Brute-force: Time $O(C^3 + \text{total chars})$, space $O(C^2)$ **Best solution:** Time $O(C + \text{total chars})$, space $O(C + \text{edges})$

Brute-force idea

A brute-force flavor can build all pairwise ordering guesses and repeatedly eliminate letters with no incoming constraints.

Brute-force solution

```
def alien_dictionary_bruteforce(words):
    chars = sorted(set(''.join(words)))
    before = {ch: set() for ch in chars}

    for i in range(len(words) - 1):
        a, b = words[i], words[i + 1]
        if len(a) > len(b) and a.startswith(b):
            return ''
        for x, y in zip(a, b):
            if x != y:
                before[y].add(x)
                break

    order = []
    remaining = set(chars)
    while remaining:
        candidates = sorted([ch for ch in remaining if not (before[ch] & remaining)])
        if not candidates:
            return ''
        ch = candidates[0]
        order.append(ch)
        remaining.remove(ch)

    return ''.join(order)
```

Best solution

Model characters as a graph, derive edges from adjacent words, then run topological sort.

Best solution

```
from collections import defaultdict, deque

def alien_dictionary_best(words):
    graph = defaultdict(set)
    indegree = {ch: 0 for word in words for ch in word}

    for i in range(len(words) - 1):
        first, second = words[i], words[i + 1]
        if len(first) > len(second) and first.startswith(second):
            return ''

        for a, b in zip(first, second):
```

```

    if a != b:
        if b not in graph[a]:
            graph[a].add(b)
            indegree[b] += 1
        break

queue = deque([ch for ch, degree in indegree.items() if degree == 0])
order = []

while queue:
    ch = queue.popleft()
    order.append(ch)
    for nxt in graph[ch]:
        indegree[nxt] -= 1
        if indegree[nxt] == 0:
            queue.append(nxt)

return ''.join(order) if len(order) == len(indegree) else ''

```

12.4 Word Ladder

Problem statement

Given `beginWord`, `endWord`, and a dictionary `wordList`, return the length of the shortest transformation sequence where you change one letter at a time and every intermediate word is in `wordList`.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `begin = 'hit', end = 'cog', words = ['hot', 'dot', 'dog', 'lot', 'log', 'cog']` -> 5
- Same setup without 'cog' in the list -> 0

Brute-force: Time $O(n^2 * L)$, space $O(n)$ **Best solution:** Time $O(n * L^2)$, space $O(n * L)$

Brute-force idea

A brute-force BFS can compare every word against every other word to see whether they differ by one letter.

Brute-force solution

```

from collections import deque

def _one_letter_apart(a, b):

```

```

diff = 0
for x, y in zip(a, b):
    if x != y:
        diff += 1
return diff == 1

def word_ladder_bruteforce(begin_word, end_word, word_list):
    words = set(word_list)
    if end_word not in words:
        return 0

    queue = deque([(begin_word, 1)])
    visited = {begin_word}
    candidates = list(words)

    while queue:
        word, steps = queue.popleft()
        if word == end_word:
            return steps

        for nxt in candidates:
            if nxt not in visited and _one_letter_apart(word, nxt):
                visited.add(nxt)
                queue.append((nxt, steps + 1))

    return 0

```

Best solution

Precomputing wildcard patterns makes neighbor lookup much faster during BFS.

Best solution

```

from collections import defaultdict, deque

def word_ladder_best(begin_word, end_word, word_list):
    if end_word not in word_list:
        return 0

    patterns = defaultdict(list)
    words = word_list + [begin_word]
    length = len(begin_word)

    for word in words:
        for i in range(length):
            patterns[word[:i] + '*' + word[i + 1:]].append(word)

    queue = deque([(begin_word, 1)])
    visited = {begin_word}

    while queue:
        word, steps = queue.popleft()
        if word == end_word:
            return steps

        for i in range(length):

```

```

        pattern = word[:i] + '*' + word[i + 1:]
        for nxt in patterns[pattern]:
            if nxt not in visited:
                visited.add(nxt)
                queue.append((nxt, steps + 1))
        patterns[pattern] = []

    return 0

```

12.5 Clone Graph

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/clone-graph/>

Problem statement

Given a reference of a node in a connected undirected graph, return a deep copy of the graph.
Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- A 4-node square graph should clone to an identical independent graph
- None -> None

Brute-force: $O(V+E)$, space $O(V)$

Best solution: $O(V+E)$, space $O(V)$

Brute-force idea

A baseline DFS or BFS still works, but the key idea is the same: map each original node to its cloned node.

Brute-force solution

```

def clone_graph_bruteforce(node):
    if not node:
        return None

    clones = {}

    def dfs(curr):
        if curr in clones:
            return clones[curr]

```

```

        copy = Node(curr.val)
        clones[curr] = copy
        for nei in curr.neighbors:
            copy.neighbors.append(dfs(nei))
        return copy

    return dfs(node)

```

Best solution

The standard BFS version builds clones lazily and wires neighbors as it traverses the graph.

Best solution

```

from collections import deque

def clone_graph_best(node):
    if not node:
        return None

    clones = {node: Node(node.val)}
    queue = deque([node])

    while queue:
        curr = queue.popleft()
        for nei in curr.neighbors:
            if nei not in clones:
                clones[nei] = Node(nei.val)
                queue.append(nei)
            clones[curr].neighbors.append(clones[nei])

    return clones[node]

```

12.6 Pacific Atlantic Water Flow

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/pacific-atlantic-water-flow/>

Problem statement

Given a matrix of heights, return all coordinates where water can flow to both the Pacific and Atlantic oceans.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- heights = [[1,2,2],[3,2,3],[2,4,5]] -> cells reaching both oceans
- single cell matrix -> [[0,0]]

Brute-force: $O(mn(m+n))$, space $O(mn)$ **Best solution:** $O(mn)$, space $O(mn)$

Brute-force idea

A brute-force baseline starts a search from every cell and checks whether that cell can reach each ocean.

Brute-force solution

```
def pacific_atlantic_bruteforce(heights):
    rows, cols = len(heights), len(heights[0])

    def can_reach(r, c, ocean):
        stack = [(r, c)]
        seen = {(r, c)}
        while stack:
            x, y = stack.pop()
            if ocean == 'pacific' and (x == 0 or y == 0):
                return True
            if ocean == 'atlantic' and (x == rows - 1 or y == cols - 1):
                return True
            for dx, dy in ((1,0), (-1,0), (0,1), (0,-1)):
                nx, ny = x + dx, y + dy
                if 0 <= nx < rows and 0 <= ny < cols and (nx, ny) not in seen and heights[nx][ny]
<= heights[x][y]:
                    seen.add((nx, ny))
                    stack.append((nx, ny))
        return False

    answer = []
    for r in range(rows):
        for c in range(cols):
            if can_reach(r, c, 'pacific') and can_reach(r, c, 'atlantic'):
                answer.append([r, c])
    return answer
```

Best solution

The efficient trick reverses the search: start from each ocean border and move uphill. Cells reached by both searches belong in the answer.

Best solution

```
def pacific_atlantic_best(heights):
    rows, cols = len(heights), len(heights[0])
    pacific = set()
```

```

atlantic = set()

def dfs(r, c, seen):
    seen.add((r, c))
    for dx, dy in ((1,0),(-1,0),(0,1),(0,-1)):
        nr, nc = r + dx, c + dy
        if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in seen and heights[nr][nc] >=
heights[r][c]:
            dfs(nr, nc, seen)

for r in range(rows):
    dfs(r, 0, pacific)
    dfs(r, cols - 1, atlantic)
for c in range(cols):
    dfs(0, c, pacific)
    dfs(rows - 1, c, atlantic)

return [[r, c] for r, c in pacific & atlantic]

```

12.7 Number of Connected Components in an Undirected Graph

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/number-of-connected-components-in-an-undirected-graph/>

Problem statement

Given n nodes and a list of undirected edges, return the number of connected components.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $n = 5$, edges = $[[0,1],[1,2],[3,4]] \rightarrow 2$
- $n = 5$, edges = $[] \rightarrow 5$

Brute-force: $O(n(n+e))$, space $O(n)$

Best solution: $O(n+e)$, space $O(n+e)$

Brute-force idea

A baseline can start a graph search from every node with a fresh visited set and count how many unique groups appear.

Brute-force solution

```
def count_components_bruteforce(n, edges):
    graph = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    seen_groups = set()

    for start in range(n):
        stack = [start]
        seen = set([start])
        while stack:
            node = stack.pop()
            for nei in graph[node]:
                if nei not in seen:
                    seen.add(nei)
                    stack.append(nei)
            seen_groups.add(tuple(sorted(seen)))
    return len(seen_groups)
```

Best solution

The normal solution builds an adjacency list once and runs DFS or BFS only from unvisited nodes.

Best solution

```
def count_components_best(n, edges):
    graph = [[] for _ in range(n)]
    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    visited = set()
    count = 0

    for node in range(n):
        if node in visited:
            continue
        count += 1
        stack = [node]
        visited.add(node)
        while stack:
            current = stack.pop()
            for nei in graph[current]:
                if nei not in visited:
                    visited.add(nei)
                    stack.append(nei)
    return count
```

12.8 Rotting Oranges

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/rotting-oranges/>

Problem statement

Given a grid with empty cells, fresh oranges, and rotten oranges, return the minimum minutes needed until no fresh orange remains, or -1 if impossible.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- grid = [[2,1,1],[1,1,0],[0,1,1]] -> 4
- grid = [[0,2]] -> 0

Brute-force: $O((mn)^2)$, space $O(mn)$

Best solution: $O(mn)$, space $O(mn)$

Brute-force idea

A naive baseline simulates the grid minute by minute by repeatedly scanning for fresh oranges next to rotten ones.

Brute-force solution

```
def oranges_rotting_bruteforce(grid):
    rows, cols = len(grid), len(grid[0])
    minutes = 0

    while True:
        changed = []
        fresh = 0
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    fresh += 1
                    for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
                        nr, nc = r + dr, c + dc
                        if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 2:
                            changed.append((r, c))
                            break
        if not changed:
            return 0 if fresh == 0 else -1
        for r, c in changed:
            grid[r][c] = 2
        minutes += 1
```

```

minutes += 1
if all(grid[r][c] != 1 for r in range(rows) for c in range(cols)):
    return minutes

```

Best solution

Multi-source BFS is better: start with all rotten oranges in the queue and spread the rot level by level.

Best solution

```

from collections import deque

def oranges_rotting_best(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0

    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c, 0))
            elif grid[r][c] == 1:
                fresh += 1

    minutes = 0
    while queue:
        r, c, minutes = queue.popleft()
        for dr, dc in ((1,0), (-1,0), (0,1), (0,-1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                grid[nr][nc] = 2
                fresh -= 1
                queue.append((nr, nc, minutes + 1))

    return minutes if fresh == 0 else -1

```

12.9 Network Delay Time

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/network-delay-time/>

Problem statement

Given directed travel times times, number of nodes n, and a start node k, return how long it takes for all nodes to receive the signal, or -1 if impossible.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2 -> 2
- times = [[1,2,1]], n = 2, k = 1 -> 1

Brute-force: $O(VE)$, space $O(V+E)$ **Best solution:** $O((V+E) \log V)$, space $O(V+E)$ **Brute-force idea**

Bellman-Ford is a clean baseline because it relaxes all edges repeatedly and handles general weighted graphs.

Brute-force solution

```
def network_delay_time_bruteforce(times, n, k):
    dist = [float("inf")] * (n + 1)
    dist[k] = 0
    for _ in range(n - 1):
        changed = False
        for u, v, w in times:
            if dist[u] != float("inf") and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    ans = max(dist[1:])
    return ans if ans < float("inf") else -1
```

Best solution

Because weights are non-negative, Dijkstra's algorithm is the best fit here.

Best solution

```
import heapq

def network_delay_time_best(times, n, k):
    graph = [[] for _ in range(n + 1)]
    for u, v, w in times:
        graph[u].append((v, w))

    heap = [(0, k)]
    dist = {}

    while heap:
        time, node = heapq.heappop(heap)
        if node in dist:
            continue
        dist[node] = time
        for nei, weight in graph[node]:
```

```
        if nei not in dist:
            heapq.heappush(heap, (time + weight, nei))

    if len(dist) != n:
        return -1
    return max(dist.values())
```

Sorting and Searching

Chapter overview

Sorting gives structure, and binary search takes advantage of that structure. A lot of interview magic here is really just recognizing the search space correctly.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

Searching Algorithms primer

This chapter opening folds in the handbook theory for searching algorithms so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Searching questions are great practice because they force you to connect data properties with algorithm choice. The big interview move is recognizing when the input is sorted.

Linear search

Use it when the data is unsorted or tiny. Simple and reliable, but not scalable for large inputs.

```
def linear_search(nums, target):  
    for i, num in enumerate(nums):  
        if num == target:  
            return i  
    return -1
```

Binary search

Binary search works only when the search space is ordered in a way that lets you safely discard half of it each step.

```
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Binary search on answer

Sometimes you are not searching an array - you are searching the answer space itself. The pattern is: guess a value, test if it works, then move left or right.

Rotated sorted array

One side is always sorted. Figure out which side that is, then check whether the target belongs there.

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

Sorting Algorithms primer

This chapter opening folds in the handbook theory for sorting algorithms so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Sorting is where you start to see how algorithm design choices affect both speed and clarity. Instead of memorizing every sort equally, learn when and why each family is useful.

Comparison view

Bubble, selection, and insertion sort are great for learning mechanics. Merge sort and quicksort are the heavy hitters for divide-and-conquer thinking. Heap sort adds a different data-structure angle.

Merge sort

Merge sort is calm and predictable: split, sort each half, then merge.

```
def merge_sort(nums):
    if len(nums) <= 1:
        return nums
    mid = len(nums) // 2
    left = merge_sort(nums[:mid])
    right = merge_sort(nums[mid:])
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

Quick sort

Quick sort is often fast in practice because partitioning is cheap and the recursive subproblems are small when pivots behave well.

```
def quick_sort(nums):
    if len(nums) <= 1:
        return nums
    pivot = nums[len(nums) // 2]
    left = [x for x in nums if x < pivot]
    middle = [x for x in nums if x == pivot]
    right = [x for x in nums if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
```

Heap sort idea

A heap lets you keep pulling the smallest or largest remaining value. It is a good reminder that sorting is not just about swaps - sometimes the right structure makes ordering natural.

13.1 Find Minimum in Rotated Sorted Array

Problem statement

Suppose an array sorted in ascending order is rotated at some pivot. Return the minimum element.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [3, 4, 5, 1, 2] -> 1`
- `nums = [11, 13, 15, 17] -> 11`

Brute-force: Time $O(n)$, space $O(1)$ **Best solution:** Time $O(\log n)$, space $O(1)$ **Brute-force idea**

A full scan always finds the minimum, so it is a great baseline.

Brute-force solution

```
def find_min_rotated_bruteforce(nums):
    answer = nums[0]
    for num in nums:
        if num < answer:
            answer = num
    return answer
```

Best solution

Binary search works because one half of the array is always sorted and tells you where the pivot cannot be.

Best solution

```
def find_min_rotated_best(nums):
    left, right = 0, len(nums) - 1

    while left < right:
        mid = (left + right) // 2
        if nums[mid] > nums[right]:
            left = mid + 1
        else:
            right = mid

    return nums[left]
```

13.2 Koko Eating Bananas

Problem statement

Koko can choose an eating speed k bananas per hour. Given piles and a number of hours h , return the minimum integer k so she can finish all bananas within h hours.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- piles = [3, 6, 7, 11], h = 8 -> 4
- piles = [30, 11, 23, 4, 20], h = 5 -> 30

Brute-force: Time $O(\max(\text{piles}) * n)$, space $O(1)$ **Best solution:** Time $O(n \log \max(\text{piles}))$, space $O(1)$

Brute-force idea

Try every speed from 1 up to the largest pile and check whether it works.

Brute-force solution

```
import math

def koko_bruteforce(piles, h):
    for speed in range(1, max(piles) + 1):
        hours = sum(math.ceil(pile / speed) for pile in piles)
        if hours <= h:
            return speed
    return max(piles)
```

Best solution

This is binary search on the answer: if a speed works, every faster speed also works.

Best solution

```
import math

def koko_best(piles, h):
    left, right = 1, max(piles)

    while left < right:
        mid = (left + right) // 2
        hours = sum(math.ceil(pile / mid) for pile in piles)
        if hours <= h:
            right = mid
        else:
            left = mid + 1

    return left
```

13.3 Median of Two Sorted Arrays

Problem statement

Given two sorted arrays `nums1` and `nums2`, return the median of the two sorted arrays.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums1 = [1, 3], nums2 = [2] -> 2.0`
- `nums1 = [1, 2], nums2 = [3, 4] -> 2.5`

Brute-force: Time $O(m+n)$, space $O(m+n)$ **Best solution:** Time $O(\log(\min(m, n)))$, space $O(1)$

Brute-force idea

Merging both arrays and then reading the middle is perfectly valid, just not optimal.

Brute-force solution

```
def median_two_sorted_bruteforce(nums1, nums2):
    merged = []
    i = j = 0

    while i < len(nums1) and j < len(nums2):
        if nums1[i] <= nums2[j]:
            merged.append(nums1[i])
            i += 1
        else:
            merged.append(nums2[j])
            j += 1

    merged.extend(nums1[i:])
    merged.extend(nums2[j:])

    n = len(merged)
    mid = n // 2
    if n % 2 == 1:
        return float(merged[mid])
    return (merged[mid - 1] + merged[mid]) / 2.0
```

Best solution

The best solution binary searches the partition on the smaller array until the left halves and right halves line up correctly.

Best solution

```
def median_two_sorted_best(nums1, nums2):
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1

    total = len(nums1) + len(nums2)
    half = total // 2
    left, right = 0, len(nums1)

    while True:
        i = (left + right) // 2
        j = half - i

        left1 = nums1[i - 1] if i > 0 else float("-inf")
        right1 = nums1[i] if i < len(nums1) else float("inf")
        left2 = nums2[j - 1] if j > 0 else float("-inf")
        right2 = nums2[j] if j < len(nums2) else float("inf")

        if left1 <= right2 and left2 <= right1:
            if total % 2 == 1:
                return float(min(right1, right2))
            return (max(left1, left2) + min(right1, right2)) / 2.0
        elif left1 > right2:
            right = i - 1
        else:
            left = i + 1
```

13.4 Binary Search

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/binary-search/>

Problem statement

Given a sorted array `nums` and a target, return the index of target, or -1 if it is not in the array.
Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [-1,0,3,5,9,12]`, target = 9 -> 4
- `nums = [-1,0,3,5,9,12]`, target = 2 -> -1

Brute-force: $O(n)$, space $O(1)$

Best solution: $O(\log n)$, space $O(1)$

Brute-force idea

The baseline is a linear scan from left to right.

Brute-force solution

```
def binary_search_bruteforce(nums, target):
    for i, num in enumerate(nums):
        if num == target:
            return i
    return -1
```

Best solution

Binary search halves the remaining range on every comparison.

Best solution

```
def binary_search_best(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

13.5 Search in Rotated Sorted Array

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/search-in-rotated-sorted-array/>

Problem statement

There is an integer array `nums` sorted in ascending order and then rotated. Return the index of `target`, or `-1` if it is not found.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [4,5,6,7,0,1,2]`, `target = 0` -> 4
- `nums = [4,5,6,7,0,1,2]`, `target = 3` -> -1

Brute-force: $O(n)$, space $O(1)$ **Best solution:** $O(\log n)$, space $O(1)$ **Brute-force idea**

A direct baseline scans every element until it finds the target.

Brute-force solution

```
def search_rotated_bruteforce(nums, target):
    for i, num in enumerate(nums):
        if num == target:
            return i
    return -1
```

Best solution

At each step, one side is still sorted. Use that fact to decide which half can still contain the target.

Best solution

```
def search_rotated_best(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

13.6 Search a 2D Matrix

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/search-a-2d-matrix/>

Problem statement

You are given an $m \times n$ integer matrix where each row is sorted and the first integer of each row is greater than the last integer of the previous row. Return true if target is in the matrix.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- matrix = [[1,3,5],[7,9,11]], target = 9 -> True
- matrix = [[1,3,5],[7,9,11]], target = 6 -> False

Brute-force: $O(mn)$, space $O(1)$

Best solution: $O(\log(mn))$, space $O(1)$

Brute-force idea

A full scan of the matrix works but ignores the sorted structure.

Brute-force solution

```
def search_matrix_bruteforce(matrix, target):
    for row in matrix:
        for value in row:
            if value == target:
                return True
    return False
```

Best solution

Treat the matrix like one sorted array and binary-search by converting a flat index back into row and column positions.

Best solution

```
def search_matrix_best(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1
    while left <= right:
```

```

mid = (left + right) // 2
value = matrix[mid // cols][mid % cols]
if value == target:
    return True
if value < target:
    left = mid + 1
else:
    right = mid - 1
return False

```

13.7 Time Based Key-Value Store

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/time-based-key-value-store/>

Problem statement

Design a time-based key-value data structure that can store multiple values for the same key at different timestamps and retrieve the value with the largest timestamp less than or equal to a given timestamp.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `set('foo','bar',1); get('foo',1) -> 'bar'`
- `set('foo','bar2',4); get('foo',3) -> 'bar'`

Brute-force: set $O(1)$, get $O(n)$

Best solution: set $O(1)$, get $O(\log n)$

Brute-force idea

A straightforward design stores the full history for each key and scans backward during get.

Brute-force solution

```

class TimeMapBruteforce:
    def __init__(self):
        self.store = {}

    def set(self, key, value, timestamp):
        self.store.setdefault(key, []).append((timestamp, value))

```

```
def get(self, key, timestamp):
    for ts, value in reversed(self.store.get(key, [])):
        if ts <= timestamp:
            return value
    return ""
```

Best solution

Because timestamps for each key are appended in order, binary search is the natural upgrade for get.

Best solution

```
class TimeMapBest:
    def __init__(self):
        self.store = {}

    def set(self, key, value, timestamp):
        self.store.setdefault(key, []).append((timestamp, value))

    def get(self, key, timestamp):
        values = self.store.get(key, [])
        left, right = 0, len(values) - 1
        answer = ""
        while left <= right:
            mid = (left + right) // 2
            ts, value = values[mid]
            if ts <= timestamp:
                answer = value
                left = mid + 1
            else:
                right = mid - 1
        return answer
```

13.8 Find Peak Element

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/find-peak-element/>

Problem statement

A peak element is strictly greater than its neighbors. Return the index of any peak element.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1,2,3,1]` -> 2
- `nums = [1,2,1,3,5,6,4]` -> 1 or 5

Brute-force: $O(n)$, space $O(1)$ **Best solution:** $O(\log n)$, space $O(1)$ **Brute-force idea**

The baseline linearly checks each position until it finds one that is greater than its neighbors.

Brute-force solution

```
def find_peak_element_bruteforce(nums):
    for i in range(len(nums)):
        left = nums[i - 1] if i > 0 else float("-inf")
        right = nums[i + 1] if i + 1 < len(nums) else float("-inf")
        if nums[i] > left and nums[i] > right:
            return i
```

Best solution

If `nums[mid]` is lower than the value to its right, a peak must exist on the right side; otherwise it is on the left including mid.

Best solution

```
def find_peak_element_best(nums):
    left, right = 0, len(nums) - 1
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid
    return left
```

13.9 Kth Smallest Element in a Sorted Matrix

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/kth-smallest-element-in-a-sorted-matrix/>

Problem statement

Given an $n \times n$ matrix where each row and column is sorted in ascending order, return the k th smallest element.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- matrix = [[1,5,9],[10,11,13],[12,13,15]], k = 8 -> 13
- matrix = [[-5]], k = 1 -> -5

Brute-force: $O(n^2 \log(n^2))$, space $O(n^2)$ **Best solution:** $O(k \log n)$, space $O(n)$

Brute-force idea

A baseline flattens the matrix, sorts the values, and picks the k th item.

Brute-force solution

```
def kth_smallest_bruteforce(matrix, k):
    values = [value for row in matrix for value in row]
    values.sort()
    return values[k - 1]
```

Best solution

A min-heap seeded with the first element of each row lets you pop the next smallest value k times.

Best solution

```
import heapq

def kth_smallest_best(matrix, k):
    n = len(matrix)
    heap = [(matrix[r][0], r, 0) for r in range(n)]
    heapq.heapify(heap)

    for _ in range(k - 1):
        value, r, c = heapq.heappop(heap)
        if c + 1 < len(matrix[r]):
            heapq.heappush(heap, (matrix[r][c + 1], r, c + 1))

    return heapq.heappop(heap)[0]
```

Two Pointers and Sliding Window

Chapter overview

These problems reward control over a moving boundary. The most useful question is whether the window or pointer movement can be justified from the input structure: sorted order, positive numbers, or a frequency condition.

Sliding Window and Two Pointers primer

This chapter opening folds in the handbook theory for sliding window and two pointers so the workbook stays self-contained. Read this primer first, then move into the interview problems.

These patterns are all about controlled movement. Instead of checking every possible subarray or substring from scratch, you move boundaries intelligently.

Fixed-size window

Keep exactly k elements in view and update the running state as one value enters and another leaves.

Variable-size window

Expand until a condition breaks, then shrink until it becomes valid again.

Longest substring without repeating characters

This is one of the best practice problems for turning an intuitive idea into a precise window.

```
def longest_unique_substring(text):
    last_seen = {}
    left = 0
    best = 0
    for right, ch in enumerate(text):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        best = max(best, right - left + 1)
```

```
return best
```

Two-sum in sorted array

When the array is sorted, two pointers can replace hashing.

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        current = nums[left] + nums[right]
        if current == target:
            return [left, right]
        if current < target:
            left += 1
        else:
            right -= 1
    return []
```

14.1 Two Sum II - Input Array Is Sorted

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/two-sum-ii-input-array-is-sorted/>

Problem statement

Given a 1-indexed sorted array, find two numbers such that they add up to target and return their indices.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- numbers = [2,7,11,15], target = 9 -> [1,2]
- numbers = [2,3,4], target = 6 -> [1,3]

Brute-force: $O(n^2)$, space $O(1)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

A baseline checks every pair.

Brute-force solution

```
def two_sum_ii_bruteforce(numbers, target):
    for i in range(len(numbers)):
        for j in range(i + 1, len(numbers)):
            if numbers[i] + numbers[j] == target:
                return [i + 1, j + 1]
```

Best solution

Because the array is sorted, use two pointers and move the side that makes the sum too small or too large.

Best solution

```
def two_sum_ii_best(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]
        if total < target:
            left += 1
        else:
            right -= 1
```

14.2 3Sum**Problem quick facts**

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/3sum/>

Problem statement

Given an integer array `nums`, return all unique triplets `[nums[i], nums[j], nums[k]]` such that their sum is zero.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [-1,0,1,2,-1,-4] -> [[-1,-1,2],[-1,0,1]]`
- `nums = [0,0,0] -> [[0,0,0]]`

Brute-force: $O(n^3)$, space $O(k)$ **Best solution:** $O(n^2)$, space $O(k)$

Brute-force idea

The brute-force baseline tries every triple and uses a set to deduplicate sorted triplets.

Brute-force solution

```
def three_sum_bruteforce(nums):
    result = set()
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            for k in range(j + 1, n):
                if nums[i] + nums[j] + nums[k] == 0:
                    result.add(tuple(sorted((nums[i], nums[j], nums[k]))))
    return [list(t) for t in result]
```

Best solution

Sort first. Then fix one number and solve the remaining two-sum problem with two pointers.

Best solution

```
def three_sum_best(nums):
    nums.sort()
    result = []
    for i in range(len(nums)):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total == 0:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
                while left < right and nums[right] == nums[right + 1]:
                    right -= 1
            elif total < 0:
                left += 1
            else:
                right -= 1
    return result
```

14.3 Container With Most Water

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/container-with-most-water/>

Problem statement

Find two lines that together with the x-axis form a container that holds the most water.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- height = [1,8,6,2,5,4,8,3,7] -> 49
- height = [1,1] -> 1

Brute-force: $O(n^2)$, space $O(1)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

The baseline checks every pair of lines and computes the area.

Brute-force solution

```
def max_area_bruteforce(height):
    best = 0
    for i in range(len(height)):
        for j in range(i + 1, len(height)):
            best = max(best, min(height[i], height[j]) * (j - i))
    return best
```

Best solution

Use two pointers and always move the shorter line, because the shorter wall is the bottleneck.

Best solution

```
def max_area_best(height):
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        best = max(best, min(height[left], height[right]) * (right - left))
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
```

```

        left += 1
    else:
        right -= 1
    return best

```

14.4 Permutation in String

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/permutation-in-string/>

Problem statement

Given two strings *s1* and *s2*, return true if *s2* contains a permutation of *s1* as a substring.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- *s1* = 'ab', *s2* = 'eidbaooo' -> True
- *s1* = 'ab', *s2* = 'eidboooo' -> False

Brute-force: $O((n-m+1) * m)$, space $O(\text{alphabet})$

Best solution: $O(n)$, space $O(\text{alphabet})$

Brute-force idea

A baseline checks every $\text{length}-|s1|$ window by sorting or counting it from scratch.

Brute-force solution

```

from collections import Counter

def check_inclusion_bruteforce(s1, s2):
    need = Counter(s1)
    k = len(s1)
    for i in range(len(s2) - k + 1):
        if Counter(s2[i:i + k]) == need:
            return True
    return False

```

Best solution

A sliding window updates counts incrementally instead of rebuilding them for every position.

Best solution

```
from collections import Counter

def check_inclusion_best(s1, s2):
    need = Counter(s1)
    window = Counter()
    left = 0

    for right, ch in enumerate(s2):
        window[ch] += 1
        if right - left + 1 > len(s1):
            left_ch = s2[left]
            window[left_ch] -= 1
            if window[left_ch] == 0:
                del window[left_ch]
            left += 1
        if window == need:
            return True
    return False
```

14.5 Minimum Size Subarray Sum

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/minimum-size-subarray-sum/>

Problem statement

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a contiguous subarray whose sum is at least `target`. If none exists, return 0.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `target = 7, nums = [2,3,1,2,4,3]` -> 2
- `target = 4, nums = [1,4,4]` -> 1

Brute-force: $O(n^2)$, space $O(1)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

The baseline tries every starting index and grows the sum until it reaches the target.

Brute-force solution

```
def min_sub_array_len_bruteforce(target, nums):
    best = float("inf")
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total >= target:
                best = min(best, j - i + 1)
                break
    return 0 if best == float("inf") else best
```

Best solution

Because all numbers are positive, once the window reaches the target you can safely shrink it from the left.

Best solution

```
def min_sub_array_len_best(target, nums):
    left = 0
    total = 0
    best = float("inf")
    for right, num in enumerate(nums):
        total += num
        while total >= target:
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best == float("inf") else best
```

14.6 Find All Anagrams in a String

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/find-all-anagrams-in-a-string/>

Problem statement

Given strings *s* and *p*, return all start indices of *p*'s anagrams in *s*.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $s = \text{'cbaebabacd'}$, $p = \text{'abc'}$ -> $[0,6]$
- $s = \text{'abab'}$, $p = \text{'ab'}$ -> $[0,1,2]$

Brute-force: $O((n-m+1) * m)$, space $O(\text{alphabet})$

Best solution: $O(n)$, space $O(\text{alphabet})$

Brute-force idea

A baseline checks every window of length $\text{len}(p)$ with a fresh counter.

Brute-force solution

```
from collections import Counter

def find_anagrams_bruteforce(s, p):
    need = Counter(p)
    k = len(p)
    result = []
    for i in range(len(s) - k + 1):
        if Counter(s[i:i + k]) == need:
            result.append(i)
    return result
```

Best solution

The better version slides one fixed-size window and updates counts by removing the left char and adding the right char.

Best solution

```
from collections import Counter

def find_anagrams_best(s, p):
    need = Counter(p)
    window = Counter()
    result = []
    left = 0

    for right, ch in enumerate(s):
        window[ch] += 1
        if right - left + 1 > len(p):
            left_ch = s[left]
            window[left_ch] -= 1
            if window[left_ch] == 0:
                del window[left_ch]
            left += 1
        if right - left + 1 == len(p) and window == need:
            result.append(left)
```

```
return result
```

14.7 Trapping Rain Water (Two-Pointer Focus)

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/trapping-rain-water/>

Problem statement

Revisit Trapping Rain Water through the two-pointer pattern.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- height = [0,1,0,2,1,0,1,3,2,1,2,1] -> 6
- height = [4,2,0,3,2,5] -> 9

Brute-force: $O(n^2)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

The baseline recomputes the highest wall to the left and right of every index.

Brute-force solution

```
def trap_two_pointer_bruteforce(height):
    water = 0
    for i in range(len(height)):
        water += min(max(height[:i + 1]), max(height[i:])) - height[i]
    return water
```

Best solution

Two pointers make the pattern explicit: move the side with the smaller boundary because that side limits the trapped water.

Best solution

```
def trap_two_pointer_best(height):
    left, right = 0, len(height) - 1
    left_max = right_max = 0
    water = 0
    while left < right:
        if height[left] < height[right]:
            left_max = max(left_max, height[left])
            water += left_max - height[left]
            left += 1
        else:
            right_max = max(right_max, height[right])
            water += right_max - height[right]
            right -= 1
    return water
```

14.8 Minimum Window Substring (Sliding-Window Focus)

Problem quick facts

Difficulty: Hard**LeetCode:** <https://leetcode.com/problems/minimum-window-substring/>

Problem statement

Revisit Minimum Window Substring as a sliding-window flagship problem.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $s = \text{'ADOBECODEBANC'}, t = \text{'ABC'} \rightarrow \text{'BANC'}$
- $s = \text{'a'}, t = \text{'a'} \rightarrow \text{'a'}$

Brute-force: $O(n^2)$ **Best solution:** $O(n)$, space $O(\text{alphabet})$

Brute-force idea

The baseline expands every starting point until all required characters are covered.

Brute-force solution

```
from collections import Counter
```

```
def min_window_sw_bruteforce(s, t):
    need = Counter(t)
    answer = ""
    for i in range(len(s)):
        have = Counter()
        for j in range(i, len(s)):
            have[s[j]] += 1
            if all(have[c] >= need[c] for c in need):
                candidate = s[i:j + 1]
                if not answer or len(candidate) < len(answer):
                    answer = candidate
                break
    return answer
```

Best solution

The polished version tracks when the window becomes valid, then shrinks greedily to keep only the minimum valid cover.

Best solution

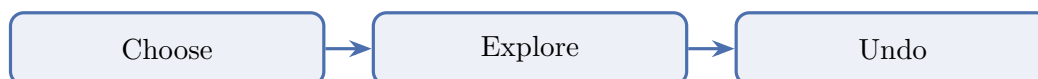
```
from collections import Counter

def min_window_sw_best(s, t):
    need = Counter(t)
    have = {}
    formed = 0
    required = len(need)
    left = 0
    best = (float("inf"), 0, 0)
    for right, ch in enumerate(s):
        have[ch] = have.get(ch, 0) + 1
        if ch in need and have[ch] == need[ch]:
            formed += 1
        while formed == required:
            if right - left + 1 < best[0]:
                best = (right - left + 1, left, right)
            left_ch = s[left]
            have[left_ch] -= 1
            if left_ch in need and have[left_ch] < need[left_ch]:
                formed -= 1
            left += 1
    return "" if best[0] == float("inf") else s[best[1]:best[2] + 1]
```

Recursion and Backtracking

Chapter overview

Backtracking is basically controlled trial and error. You make a choice, explore that path, and undo the choice before trying the next option.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

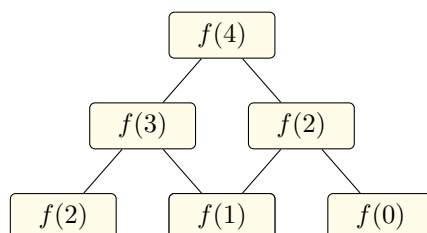
Recursion and Backtracking primer

This chapter opening folds in the handbook theory for recursion and backtracking so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Recursion becomes much less scary when you stop thinking about the whole call stack at once. Focus on one function call, one smaller subproblem, and one clean base case.

Recursion tree intuition

The tree below shows why naive recursive Fibonacci repeats work. That repeated overlap is exactly why memoization helps.



Simple recursion

A recursive function needs a stopping point first. Write the base case before you write the recursive step.

```
def factorial(n):  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)
```

Backtracking pattern

Backtracking is basically controlled trial and error. Choose, explore, undo, then try the next option.

```
def subsets(nums):  
    result = []  
    path = []  
  
    def dfs(index):  
        if index == len(nums):  
            result.append(path[:])  
            return  
        path.append(nums[index])  
        dfs(index + 1)  
        path.pop()  
        dfs(index + 1)  
  
    dfs(0)  
    return result
```

N-Queens framing

The core idea is not the board itself. It is the discipline of checking whether a partial choice is still valid before exploring deeper.

15.1 Subsets

Problem statement

Given an integer array `nums` of unique elements, return all possible subsets.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1, 2, 3]` -> 8 subsets
- `nums = [0]` -> `[], [0]`

Brute-force: Time $O(n * 2^n)$, space $O(2^n)$ **Best solution:** Time $O(n * 2^n)$, space $O(2^n)$

Brute-force idea

You can loop through every bitmask from 0 to $2^n - 1$ and decode which elements are chosen.

Brute-force solution

```
def subsets_bruteforce(nums):
    result = []
    n = len(nums)

    for mask in range(1 << n):
        subset = []
        for i in range(n):
            if mask & (1 << i):
                subset.append(nums[i])
        result.append(subset)

    return result
```

Best solution

The backtracking version feels more natural in interviews because it exposes the choose-explore-unchoose pattern clearly.

Best solution

```
def subsets_best(nums):
    result = []
    path = []

    def backtrack(index):
        if index == len(nums):
            result.append(path[:])
            return

        backtrack(index + 1)

        path.append(nums[index])
        backtrack(index + 1)
        path.pop()

    backtrack(0)
    return result
```

15.2 Permutations

Problem statement

Given an array `nums` of distinct integers, return all the possible permutations.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1, 2, 3]` -> 6 permutations
- `nums = [0, 1]` -> `[[0, 1], [1, 0]]`

Brute-force: Time $O(n * n!)$, space $O(n!)$

Best solution: Time $O(n * n!)$, space $O(n!)$

Brute-force idea

Python's `itertools` can generate all permutations, but that dodges the real interview thinking a bit.

Brute-force solution

```
from itertools import permutations

def permutations_bruteforce(nums):
    return [list(p) for p in permutations(nums)]
```

Best solution

Backtracking with swapping or a used-array is the real reusable pattern.

Best solution

```
def permutations_best(nums):
    result = []
    used = [False] * len(nums)
    path = []

    def backtrack():
        if len(path) == len(nums):
            result.append(path[:])
            return

        for i, num in enumerate(nums):
            if used[i]:
                continue
            used[i] = True
            path.append(num)
            backtrack()
            path.pop()
            used[i] = False

    backtrack()
    return result
```

15.3 Combination Sum

Problem statement

Given distinct candidate numbers and a target, return all unique combinations where the chosen numbers sum to target. You may use the same number unlimited times.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- candidates = [2, 3, 6, 7], target = 7 -> [[2, 2, 3], [7]]
- candidates = [2, 3, 5], target = 8 -> [[2, 2, 2, 2], [2, 3, 3], [3, 5]]

Brute-force: Time exponential

Best solution: Time exponential with pruning

Brute-force idea

A brute-force idea is to generate many sequences and filter those that hit the target, but it explodes quickly.

Brute-force solution

```
def combination_sum_bruteforce(candidates, target):
    result = set()

    def generate(path, total):
        if total == target:
            result.add(tuple(sorted(path)))
            return
        if total > target:
            return
        for value in candidates:
            generate(path + [value], total + value)

    generate([], 0)
    return [list(item) for item in sorted(result)]
```

Best solution

Backtracking prunes the branch as soon as the remaining target goes negative.

Best solution

```
def combination_sum_best(candidates, target):
    result = []
    path = []
```

```
def backtrack(start, remaining):
    if remaining == 0:
        result.append(path[:])
        return
    if remaining < 0:
        return

    for i in range(start, len(candidates)):
        path.append(candidates[i])
        backtrack(i, remaining - candidates[i])
        path.pop()

backtrack(0, target)
return result
```

15.4 N-Queens

Problem statement

Place n queens on an $n \times n$ chessboard so that no two queens attack each other. Return all distinct board configurations.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $n = 4 \rightarrow 2$ solutions
- $n = 1 \rightarrow [['Q']]$

Brute-force: Time $O((n^2 \text{ choose } n) * n^2)$

Best solution: Time $O(n!)$ with pruning

Brute-force idea

A brute-force angle tries many board placements and validates the whole board at the end.

Brute-force solution

```
from itertools import combinations

def n_queens_bruteforce(n):
    cells = range(n * n)
    solutions = []

    def is_valid(positions):
        rows = set()
        cols = set()
```

```

diag1 = set()
diag2 = set()
for pos in positions:
    r, c = divmod(pos, n)
    if r in rows or c in cols or (r - c) in diag1 or (r + c) in diag2:
        return False
    rows.add(r)
    cols.add(c)
    diag1.add(r - c)
    diag2.add(r + c)
return True

for positions in combinations(cells, n):
    if is_valid(positions):
        board = [['.' * n for _ in range(n)]]
        for pos in positions:
            r, c = divmod(pos, n)
            board[r][c] = 'Q'
        solutions.append([''.join(row) for row in board])

return solutions

```

Best solution

Track blocked columns and diagonals as you build the board row by row.

Best solution

```

def n_queens_best(n):
    result = []
    board = [['.' * n for _ in range(n)]]
    cols = set()
    diag1 = set()
    diag2 = set()

    def backtrack(row):
        if row == n:
            result.append([''.join(r) for r in board])
            return

        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue

            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)
            board[row][col] = 'Q'

            backtrack(row + 1)

            board[row][col] = '.'
            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)

    backtrack(0)

    return result

```

```
backtrack(0)
return result
```

15.5 Word Search

Problem statement

Given a board of letters and a word, return True if the word exists in the grid using horizontally or vertically adjacent cells without reusing the same cell.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- Board contains 'ABCCED' -> True
- Board does not contain 'ABCB' because it would reuse a cell -> False

Brute-force: Time $O(mn * 4^L)$, space $O(L)$ **Best solution:** Time $O(mn * 4^L)$, space $O(L)$

Brute-force idea

A brute-force version can launch DFS from every cell and carry a fresh visited set around each branch.

Brute-force solution

```
def word_search_bruteforce(board, word):
    rows, cols = len(board), len(board[0])

    def dfs(r, c, index, visited):
        if index == len(word):
            return True
        if (
            r < 0 or r >= rows or
            c < 0 or c >= cols or
            board[r][c] != word[index] or
            (r, c) in visited
        ):
            return False

        visited.add((r, c))
        found = (
            dfs(r + 1, c, index + 1, visited) or
            dfs(r - 1, c, index + 1, visited) or
            dfs(r, c + 1, index + 1, visited) or
            dfs(r, c - 1, index + 1, visited)
        )
        visited.remove((r, c))
```

```

        return found

    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0, set()):
                return True
    return False

```

Best solution

The optimized version still uses DFS, but it marks and restores the board in place to reduce overhead.

Best solution

```

def word_search_best(board, word):
    rows, cols = len(board), len(board[0])

    def dfs(r, c, index):
        if index == len(word):
            return True
        if (
            r < 0 or r >= rows or
            c < 0 or c >= cols or
            board[r][c] != word[index]
        ):
            return False

        temp = board[r][c]
        board[r][c] = '#'

        found = (
            dfs(r + 1, c, index + 1) or
            dfs(r - 1, c, index + 1) or
            dfs(r, c + 1, index + 1) or
            dfs(r, c - 1, index + 1)
        )

        board[r][c] = temp
        return found

    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0):
                return True
    return False

```

15.6 Palindrome Partitioning

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/palindrome-partitioning/>

Problem statement

Given a string *s*, partition *s* such that every substring in the partition is a palindrome. Return all possible palindrome partitionings.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- *s* = 'aab' -> [['a','a','b'], ['aa','b']]
- *s* = 'a' -> [['a']]

Brute-force: $O(n \cdot 2^n)$ with repeated checks **Best solution:** $O(n \cdot 2^n)$ overall, with cleaner pruning

Brute-force idea

A baseline tries every cut recursively and checks palindromes whenever it wants to place a substring.

Brute-force solution

```
def partition_palindromes_bruteforce(s):
    result = []

    def dfs(start, path):
        if start == len(s):
            result.append(path[:])
            return
        for end in range(start + 1, len(s) + 1):
            piece = s[start:end]
            if piece == piece[::-1]:
                path.append(piece)
                dfs(end, path)
                path.pop()

    dfs(0, [])
    return result
```

Best solution

The optimized version precomputes which substrings are palindromes, so backtracking does not recheck the same slices again and again.

Best solution

```
def partition_palindromes_best(s):
    n = len(s)
    pal = [[False] * n for _ in range(n)]
    for end in range(n):
        for start in range(end + 1):
            if s[start] == s[end] and (end - start <= 2 or pal[start + 1][end - 1]):
                pal[start][end] = True

    result = []

    def dfs(start, path):
        if start == n:
            result.append(path[:])
            return
        for end in range(start, n):
            if pal[start][end]:
                path.append(s[start:end + 1])
                dfs(end + 1, path)
                path.pop()

    dfs(0, [])
    return result
```

15.7 Letter Combinations of a Phone Number

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/letter-combinations-of-a-phone-number/>

Problem statement

Given a string of digits from 2-9, return all possible letter combinations that the number could represent.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- digits = '23' -> ['ad','ae','af','bd','be','bf','cd','ce','cf']
- digits = "" -> []

Brute-force: $O(4^n)$, space $O(n)$ **Best solution:** $O(4^n)$, space $O(n)$

Brute-force idea

The core idea is already backtracking: choose one letter for the current digit, then recurse to the next digit.

Brute-force solution

```
def letter_combinations_bruteforce(digits):
    if not digits:
        return []
    phone = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }
    result = []

    def dfs(i, path):
        if i == len(digits):
            result.append("".join(path))
            return
        for ch in phone[digits[i]]:
            path.append(ch)
            dfs(i + 1, path)
            path.pop()

    dfs(0, [])
    return result
```

Best solution

A compact iterative build also works well: grow the answer list digit by digit.

Best solution

```
def letter_combinations_best(digits):
    if not digits:
        return []
    phone = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }
    result = [""]
    for digit in digits:
        result = [prefix + ch for prefix in result for ch in phone[digit]]
    return result
```

15.8 Combination Sum II

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/combination-sum-ii/>

Problem statement

Given a collection of candidate numbers and a target, return all unique combinations where each number may be used at most once.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- candidates = [10,1,2,7,6,1,5], target = 8 -> [[1,1,6],[1,2,5],[1,7],[2,6]]
- candidates = [2,5,2,1,2], target = 5 -> [[1,2,2],[5]]

Brute-force: $O(2^n \log n)$ with set deduplication
pruning and skip-duplicates

Best solution: $O(2^n)$ with

Brute-force idea

A baseline can explore all subsets and deduplicate finished combinations with a set.

Brute-force solution

```
def combination_sum_ii_bruteforce(candidates, target):
    result = set()
    n = len(candidates)

    def dfs(i, total, path):
        if total == target:
            result.add(tuple(sorted(path)))
            return
        if i == n or total > target:
            return
        dfs(i + 1, total + candidates[i], path + [candidates[i]])
        dfs(i + 1, total, path)

    dfs(0, 0, [])
    return [list(t) for t in sorted(result)]
```

Best solution

Sort first. Then backtrack while skipping duplicate values at the same depth so each combination is generated only once.

Best solution

```
def combination_sum_ii_best(candidates, target):
    candidates.sort()
    result = []

    def dfs(start, total, path):
        if total == target:
            result.append(path[:])
            return
        if total > target:
            return

        prev = None
        for i in range(start, len(candidates)):
            if candidates[i] == prev:
                continue
            path.append(candidates[i])
            dfs(i + 1, total + candidates[i], path)
            path.pop()
            prev = candidates[i]

    dfs(0, 0, [])
    return result
```

Greedy Algorithms

Chapter overview

Greedy problems are all about making a choice that feels locally safe and then proving it never paints you into a corner. When they work, the code becomes refreshingly short.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

Greedy Algorithms primer

This chapter opening folds in the handbook theory for greedy algorithms so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Greedy solutions make the best local choice they can right now. The catch is that not every problem allows that strategy, so the real skill is recognizing when the greedy choice stays safe globally.

Activity selection

A classic greedy idea: pick the activity that finishes earliest so you leave as much room as possible for what comes next.

```
def activity_selection(intervals):
    intervals.sort(key=lambda item: item[1])
    chosen = []
    last_end = float('-inf')
    for start, end in intervals:
        if start >= last_end:
            chosen.append((start, end))
            last_end = end
    return chosen
```

Fractional knapsack

When you are allowed to take part of an item, value density matters. That makes the greedy choice provably correct.

16.1 Jump Game

Problem statement

Given an array `nums` where each element is your maximum jump length at that position, return `True` if you can reach the last index.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [2, 3, 1, 1, 4]` -> `True`
- `nums = [3, 2, 1, 0, 4]` -> `False`

Brute-force: Time $O(2^n)$, space $O(n)$

Best solution: Time $O(n)$, space $O(1)$

Brute-force idea

The brute-force recursion tries every possible jump from each position.

Brute-force solution

```
def jump_game_bruteforce(nums):
    def dfs(index):
        if index >= len(nums) - 1:
            return True
        furthest = min(len(nums) - 1, index + nums[index])
        for nxt in range(index + 1, furthest + 1):
            if dfs(nxt):
                return True
        return False

    return dfs(0)
```

Best solution

Track the farthest index you can currently reach. If you ever fall behind it, the game is over.

Best solution

```
def jump_game_best(nums):
    farthest = 0
    for i, jump in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + jump)
    return True
```

16.2 Merge Intervals

Problem statement

Given an array of intervals where $\text{intervals}[i] = [\text{start}, \text{end}]$, merge all overlapping intervals and return the resulting list.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $[[1, 3], [2, 6], [8, 10], [15, 18]] \rightarrow [[1, 6], [8, 10], [15, 18]]$
- $[[1, 4], [4, 5]] \rightarrow [[1, 5]]$

Brute-force: Time $O(n^2)$, space $O(n)$

Best solution: Time $O(n \log n)$, space $O(n)$

Brute-force idea

A brute-force approach repeatedly looks for any overlapping pair and merges until stable.

Brute-force solution

```
def merge_intervals_bruteforce(intervals):
    intervals = [interval[:] for interval in intervals]
    changed = True

    while changed:
        changed = False
        result = []
        used = [False] * len(intervals)

        for i in range(len(intervals)):
            if used[i]:
                continue
            start, end = intervals[i]
            for j in range(i + 1, len(intervals)):
                if used[j]:
```

```

        continue
    if not (end < intervals[j][0] or intervals[j][1] < start):
        start = min(start, intervals[j][0])
        end = max(end, intervals[j][1])
        used[j] = True
        changed = True
    used[i] = True
    result.append([start, end])

    intervals = result

return sorted(intervals)

```

Best solution

Sort by start time once, then sweep from left to right and merge into the current interval.

Best solution

```

def merge_intervals_best(intervals):
    if not intervals:
        return []

    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]

    for start, end in intervals[1:]:
        if start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])

    return merged

```

16.3 Gas Station

Problem statement

There are n gas stations arranged in a circle. $\text{gas}[i]$ is the fuel at station i and $\text{cost}[i]$ is the fuel needed to travel to the next station. Return the starting station index if you can travel around once, or -1 otherwise.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $\text{gas} = [1, 2, 3, 4, 5]$, $\text{cost} = [3, 4, 5, 1, 2]$ -> 3
- $\text{gas} = [2, 3, 4]$, $\text{cost} = [3, 4, 3]$ -> -1

Brute-force: Time $O(n^2)$, space $O(1)$ **Best solution:** Time $O(n)$, space $O(1)$

Brute-force idea

Try every starting position and simulate the full circle.

Brute-force solution

```
def gas_station_bruteforce(gas, cost):
    n = len(gas)
    for start in range(n):
        tank = 0
        possible = True
        for step in range(n):
            i = (start + step) % n
            tank += gas[i] - cost[i]
            if tank < 0:
                possible = False
                break
        if possible:
            return start
    return -1
```

Best solution

If total gas is enough, the greedy reset trick identifies the only viable starting zone.

Best solution

```
def gas_station_best(gas, cost):
    if sum(gas) < sum(cost):
        return -1

    tank = 0
    start = 0

    for i in range(len(gas)):
        tank += gas[i] - cost[i]
        if tank < 0:
            tank = 0
            start = i + 1

    return start
```

16.4 Partition Labels

Problem statement

Given a string *s*, partition it into as many parts as possible so that each letter appears in at most one part. Return a list of the partition sizes.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- *s* = 'ababcbacadefegdehijhklij' -> [9, 7, 8]
- *s* = 'eccbbbbbdec' -> [10]

Brute-force: Time $O(n^2)$, space $O(1)$

Best solution: Time $O(n)$, space $O(1)$

Brute-force idea

A brute-force path tries cut points and checks whether letters leak into later segments.

Brute-force solution

```
def partition_labels_bruteforce(s):
    result = []
    start = 0

    while start < len(s):
        end = start
        changed = True
        while changed:
            changed = False
            for i in range(start, end + 1):
                last = s.rfind(s[i])
                if last > end:
                    end = last
                    changed = True
            result.append(end - start + 1)
            start = end + 1

    return result
```

Best solution

Record the last index of every character, then extend the current partition until it safely closes.

Best solution

```
def partition_labels_best(s):
    last = {ch: i for i, ch in enumerate(s)}
    result = []
    start = end = 0

    for i, ch in enumerate(s):
        end = max(end, last[ch])
        if i == end:
            result.append(end - start + 1)
            start = i + 1

    return result
```

16.5 Jump Game II

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/jump-game-ii/>

Problem statement

Given an array of non-negative integers `nums`, where each element represents your maximum jump length, return the minimum number of jumps to reach the last index.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [2,3,1,1,4]` -> 2
- `nums = [2,3,0,1,4]` -> 2

Brute-force: $O(2^n)$ **Best solution:** $O(n)$, space $O(1)$

Brute-force idea

A brute-force recursion tries every possible jump from each position and returns the minimum.

Brute-force solution

```
def jump_game_ii_bruteforce(nums):
    def dfs(i):
        if i >= len(nums) - 1:
```

```

        return 0
    best = float("inf")
    for step in range(1, nums[i] + 1):
        best = min(best, 1 + dfs(i + step))
    return best
return dfs(0)

```

Best solution

The greedy sweep tracks the farthest place you can reach in the current jump range. When you leave that range, you must spend one jump.

Best solution

```

def jump_game_ii_best(nums):
    jumps = 0
    current_end = 0
    farthest = 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == current_end:
            jumps += 1
            current_end = farthest
    return jumps

```

16.6 Candy

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/candy/>

Problem statement

There are n children standing in a line. Each child must have at least one candy, and children with a higher rating than an adjacent child must get more candies. Return the minimum candies you need.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- ratings = [1,0,2] -> 5
- ratings = [1,2,2] -> 4

Brute-force: $O(n^2)$, space $O(n)$ **Best solution:** $O(n)$, space $O(n)$

Brute-force idea

A baseline keeps adjusting candies until all local rating constraints are satisfied.

Brute-force solution

```
def candy_bruteforce(ratings):
    candies = [1] * len(ratings)
    changed = True
    while changed:
        changed = False
        for i in range(len(ratings)):
            if i > 0 and ratings[i] > ratings[i - 1] and candies[i] <= candies[i - 1]:
                candies[i] = candies[i - 1] + 1
                changed = True
            if i + 1 < len(ratings) and ratings[i] > ratings[i + 1] and candies[i] <= candies[i + 1]:
                candies[i] = candies[i + 1] + 1
                changed = True
    return sum(candies)
```

Best solution

Two passes are enough: left-to-right handles rising slopes, right-to-left fixes falling slopes.

Best solution

```
def candy_best(ratings):
    candies = [1] * len(ratings)

    for i in range(1, len(ratings)):
        if ratings[i] > ratings[i - 1]:
            candies[i] = candies[i - 1] + 1

    for i in range(len(ratings) - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            candies[i] = max(candies[i], candies[i + 1] + 1)

    return sum(candies)
```

16.7 Non-overlapping Intervals

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/non-overlapping-intervals/>

Problem statement

Given a collection of intervals, return the minimum number of intervals you need to remove to make the rest non-overlapping.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- intervals = [[1,2],[2,3],[3,4],[1,3]] -> 1
- intervals = [[1,2],[1,2],[1,2]] -> 2

Brute-force: $O(n^2)$, space $O(n)$

Best solution: $O(n \log n)$, space $O(1)$

Brute-force idea

A brute-force search can try different keep/remove choices and track the smallest number of removals.

Brute-force solution

```
def erase_overlap_intervals_bruteforce(intervals):
    intervals.sort()
    best = len(intervals)

    def dfs(i, last_end, removed):
        nonlocal best
        if removed >= best:
            return
        if i == len(intervals):
            best = min(best, removed)
            return
        start, end = intervals[i]
        dfs(i + 1, last_end, removed + 1)
        if last_end is None or start >= last_end:
            dfs(i + 1, end, removed)

    dfs(0, None, 0)
    return best
```

Best solution

The greedy rule is to keep the interval that ends earliest, because it leaves the most room for future intervals.

Best solution

```
def erase_overlap_intervals_best(intervals):
    intervals.sort(key=lambda x: x[1])
```

```

removed = 0
end = float("-inf")
for start, finish in intervals:
    if start < end:
        removed += 1
    else:
        end = finish
return removed

```

16.8 Minimum Number of Arrows to Burst Balloons

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons/>

Problem statement

Given intervals representing balloon diameters, return the minimum number of arrows required to burst all balloons.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- points = [[10,16],[2,8],[1,6],[7,12]] -> 2
- points = [[1,2],[3,4],[5,6],[7,8]] -> 4

Brute-force: $O(n^2)$, space $O(n)$

Best solution: $O(n \log n)$, space $O(1)$

Brute-force idea

A baseline can repeatedly choose one balloon, fire through its overlap range, remove everything burst, and repeat.

Brute-force solution

```

def find_min_arrow_shots_bruteforce(points):
    points = points[:]
    arrows = 0
    while points:
        start, end = points.pop()
        overlap_start, overlap_end = start, end
        remaining = []

```

```

for s, e in points:
    if e < overlap_start or s > overlap_end:
        remaining.append([s, e])
    else:
        overlap_start = max(overlap_start, s)
        overlap_end = min(overlap_end, e)
points = remaining
arrows += 1
return arrows

```

Best solution

Sort by end position and greedily fire the next arrow at the earliest finishing balloon that is not yet covered.

Best solution

```

def find_min_arrow_shots_best(points):
    points.sort(key=lambda x: x[1])
    arrows = 0
    current_end = float("-inf")
    for start, end in points:
        if start > current_end:
            arrows += 1
            current_end = end
    return arrows

```

16.9 Queue Reconstruction by Height

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/queue-reconstruction-by-height/>

Problem statement

You are given people where $\text{people}[i] = [h, k]$. Reconstruct the queue so that each person has exactly k people in front of them who have height greater than or equal to h .

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $\text{people} = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]] \rightarrow [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]$
- $\text{people} = [[6,0],[5,0],[4,0]] \rightarrow [[4,0],[5,0],[6,0]]$

Brute-force: $O(n^2 \log n)$
output

Best solution: $O(n^2)$, space $O(1)$ extra aside from

Brute-force idea

A baseline can try inserting people in many candidate orders and validate counts, but that becomes expensive quickly.

Brute-force solution

```
def reconstruct_queue_bruteforce(people):
    queue = []
    for person in sorted(people):
        placed = False
        for i in range(len(queue) + 1):
            candidate = queue[:i] + [person] + queue[i:]
            ok = True
            for j, (h, k) in enumerate(candidate):
                if sum(1 for ph, _ in candidate[:j] if ph >= h) != k:
                    ok = False
                    break
            if ok:
                queue = candidate
                placed = True
                break
        if not placed:
            queue.append(person)
    return queue
```

Best solution

The greedy trick is to place taller people first. Once heights are descending, inserting a person at index k guarantees exactly k taller-or-equal people stand in front.

Best solution

```
def reconstruct_queue_best(people):
    people.sort(key=lambda x: (-x[0], x[1]))
    queue = []
    for person in people:
        queue.insert(person[1], person)
    return queue
```

Dynamic Programming

Chapter overview

Dynamic programming looks intimidating at first, but the habit is simple: define the state, write the recurrence, and stop recomputing the same work. Once you see overlapping subproblems, you are usually in DP territory.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

Dynamic Programming primer

This chapter opening folds in the handbook theory for dynamic programming so the workbook stays self-contained. Read this primer first, then move into the interview problems.

DP feels much friendlier once you stop treating it like a bag of tricks. At heart, DP is just organized reuse of overlapping subproblems.

Memoization vs tabulation

Memoization is top-down: solve only what you need and cache it. Tabulation is bottom-up: fill a table from the smallest solved states upward.

Climbing stairs

This tiny problem captures the DP mindset really well: define a state, find the recurrence, and store intermediate results.

```
def climb_stairs(n):  
    if n <= 2:  
        return n  
    a, b = 1, 2
```

```

for _ in range(3, n + 1):
    a, b = b, a + b
return b

```

0/1 Knapsack

For each item, you either take it or leave it. That binary choice turns into a table indexed by item position and remaining capacity.

```

def knapsack(weights, values, capacity):
    dp = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for c in range(capacity, w - 1, -1):
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[capacity]

```

LCS idea

Longest Common Subsequence is the classic reminder that DP often comes from comparing prefixes of two inputs.

17.1 Word Break

Problem statement

Given a string *s* and a dictionary *wordDict*, return *True* if *s* can be segmented into a space-separated sequence of dictionary words.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- *s* = 'leetcode', *wordDict* = ['leet', 'code'] -> *True*
- *s* = 'catsandog', *wordDict* = ['cats', 'dog', 'sand', 'and', 'cat'] -> *False*

Brute-force: Time exponential

Best solution: Time $O(n^2)$, space $O(n)$

Brute-force idea

The brute-force recursion tries every possible prefix at every position.

Brute-force solution

```

def word_break_bruteforce(s, word_dict):
    words = set(word_dict)

    def dfs(start):

```

```

    if start == len(s):
        return True
    for end in range(start + 1, len(s) + 1):
        if s[start:end] in words and dfs(end):
            return True
    return False

return dfs(0)

```

Best solution

DP remembers whether the prefix ending at each index is buildable, so you never solve the same suffix twice.

Best solution

```

def word_break_best(s, word_dict):
    words = set(word_dict)
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for end in range(1, len(s) + 1):
        for start in range(end):
            if dp[start] and s[start:end] in words:
                dp[end] = True
                break

    return dp[-1]

```

17.2 Decode Ways

Problem statement

A message containing digits can be decoded using 1 -> A through 26 -> Z. Given a string *s* containing only digits, return the number of ways to decode it.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- *s* = '12' -> 2
- *s* = '226' -> 3

Brute-force: Time exponential

Best solution: Time $O(n)$, space $O(n)$

Brute-force idea

The recursive version branches on using one digit or two digits whenever both are valid.

Brute-force solution

```
def decode_ways_bruteforce(s):
    def dfs(i):
        if i == len(s):
            return 1
        if s[i] == '0':
            return 0

        ways = dfs(i + 1)
        if i + 1 < len(s) and 10 <= int(s[i:i + 2]) <= 26:
            ways += dfs(i + 2)
        return ways

    return dfs(0)
```

Best solution

DP caches the number of decoding ways from each position onward.

Best solution

```
def decode_ways_best(s):
    n = len(s)
    dp = [0] * (n + 1)
    dp[n] = 1

    for i in range(n - 1, -1, -1):
        if s[i] == '0':
            dp[i] = 0
            continue

        dp[i] = dp[i + 1]
        if i + 1 < n and 10 <= int(s[i:i + 2]) <= 26:
            dp[i] += dp[i + 2]

    return dp[0]
```

17.3 Burst Balloons

Problem statement

You are given n balloons. Bursting balloon i earns $\text{nums}[\text{left}] * \text{nums}[i] * \text{nums}[\text{right}]$ coins, where left and right are the nearest remaining balloons. Return the maximum coins you can collect.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [3, 1, 5, 8] -> 167`
- `nums = [1, 5] -> 10`

Brute-force: Time $O(n!)$, space $O(n)$ **Best solution:** Time $O(n^3)$, space $O(n^2)$

Brute-force idea

Trying every burst order is the honest brute-force story, but it blows up very quickly.

Brute-force solution

```
def burst_balloons_bruteforce(nums):
    def solve(arr):
        if not arr:
            return 0

        best = 0
        for i in range(len(arr)):
            left = arr[i - 1] if i > 0 else 1
            right = arr[i + 1] if i + 1 < len(arr) else 1
            coins = left * arr[i] * right
            best = max(best, coins + solve(arr[:i] + arr[i + 1:]))
        return best

    return solve(nums)
```

Best solution

Interval DP flips the perspective: pick the last balloon to burst in each interval.

Best solution

```
def burst_balloons_best(nums):
    nums = [1] + nums + [1]
    n = len(nums)
    dp = [[0] * n for _ in range(n)]

    for length in range(2, n):
        for left in range(0, n - length):
            right = left + length
            for mid in range(left + 1, right):
                dp[left][right] = max(
                    dp[left][right],
                    dp[left][mid] + dp[mid][right] + nums[left] * nums[mid] * nums[right]
                )

    return dp[0][n - 1]
```

17.4 Longest Increasing Subsequence

Problem statement

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [10, 9, 2, 5, 3, 7, 101, 18]` -> 4
- `nums = [7, 7, 7, 7, 7]` -> 1

Brute-force: Time $O(2^n)$, space $O(n)$

Best solution: Time $O(n \log n)$, space $O(n)$

Brute-force idea

A recursive brute-force version chooses whether to take or skip each number.

Brute-force solution

```
def lis_bruteforce(nums):
    def dfs(index, prev):
        if index == len(nums):
            return 0

        best = dfs(index + 1, prev)
        if prev is None or nums[index] > prev:
            best = max(best, 1 + dfs(index + 1, nums[index]))
        return best

    return dfs(0, None)
```

Best solution

The patience-sorting idea keeps the smallest possible tail for each subsequence length.

Best solution

```
from bisect import bisect_left

def lis_best(nums):
    tails = []
    for num in nums:
        i = bisect_left(tails, num)
        if i == len(tails):
            tails.append(num)
```

```
    else:
        tails[i] = num
    return len(tails)
```

17.5 Climbing Stairs

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/climbing-stairs/>

Problem statement

You are climbing a staircase. Each time you can climb 1 or 2 steps. Return the number of distinct ways to reach the top.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $n = 2 \rightarrow 2$
- $n = 3 \rightarrow 3$

Brute-force: $O(2^n)$, space $O(n)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

The baseline recursion branches into taking 1 step or 2 steps from the current position.

Brute-force solution

```
def climb_stairs_bruteforce(n):
    def dfs(steps_left):
        if steps_left == 0:
            return 1
        if steps_left < 0:
            return 0
        return dfs(steps_left - 1) + dfs(steps_left - 2)
    return dfs(n)
```

Best solution

This is just Fibonacci in disguise. Keep only the last two answers instead of a whole table.

Best solution

```
def climb_stairs_best(n):
    a, b = 1, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

17.6 House Robber

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/house-robber/>

Problem statement

Given an integer array `nums` where each element is the amount of money in a house, return the maximum amount you can rob without robbing adjacent houses.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1,2,3,1]` -> 4
- `nums = [2,7,9,3,1]` -> 12

Brute-force: $O(2^n)$, space $O(n)$ **Best solution:** $O(n)$, space $O(1)$

Brute-force idea

The brute-force recursion decides at each house whether to rob it and skip the next house, or skip it now.

Brute-force solution

```
def house_robber_bruteforce(nums):
    def dfs(i):
        if i >= len(nums):
            return 0
        return max(nums[i] + dfs(i + 2), dfs(i + 1))
    return dfs(0)
```

Best solution

DP keeps two running states: the best total if you include the current house and the best if you skip it.

Best solution

```
def house_robber_best(nums):
    rob = skip = 0
    for num in nums:
        rob, skip = skip + num, max(skip, rob)
    return max(rob, skip)
```

17.7 Coin Change

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/coin-change/>

Problem statement

Given an integer array `coins` and an integer `amount`, return the fewest number of coins needed to make up that amount, or -1 if it is not possible.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `coins = [1,2,5]`, `amount = 11` -> 3
- `coins = [2]`, `amount = 3` -> -1

Brute-force: $O(k^{\text{amount}})$ **Best solution:** $O(\text{amount} * \text{len}(\text{coins}))$, space $O(\text{amount})$

Brute-force idea

A recursive baseline tries each coin choice at every remaining amount.

Brute-force solution

```
def coin_change_bruteforce(coins, amount):
    def dfs(remaining):
        if remaining == 0:
            return 0
        if remaining < 0:
            return float("inf")
```

```

    best = float("inf")
    for coin in coins:
        best = min(best, 1 + dfs(remaining - coin))
    return best

answer = dfs(amount)
return -1 if answer == float("inf") else answer

```

Best solution

Bottom-up DP builds the minimum coins needed for every amount from 1 to target.

Best solution

```

def coin_change_best(coins, amount):
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for total in range(1, amount + 1):
        for coin in coins:
            if total - coin >= 0:
                dp[total] = min(dp[total], 1 + dp[total - coin])
    return -1 if dp[amount] == float("inf") else dp[amount]

```

17.8 Longest Common Subsequence

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/longest-common-subsequence/>

Problem statement

Given two strings `text1` and `text2`, return the length of their longest common subsequence.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `text1 = 'abcde', text2 = 'ace' -> 3`
- `text1 = 'abc', text2 = 'def' -> 0`

Brute-force: $O(2^{\{m+n\}})$

Best solution: $O(mn)$, space $O(mn)$

Brute-force idea

The recursive baseline branches on either matching the current characters or skipping one side.

Brute-force solution

```
def longest_common_subsequence_bruteforce(text1, text2):
    def dfs(i, j):
        if i == len(text1) or j == len(text2):
            return 0
        if text1[i] == text2[j]:
            return 1 + dfs(i + 1, j + 1)
        return max(dfs(i + 1, j), dfs(i, j + 1))
    return dfs(0, 0)
```

Best solution

DP stores the best LCS length for every suffix pair, so overlapping subproblems are solved once.

Best solution

```
def longest_common_subsequence_best(text1, text2):
    rows, cols = len(text1), len(text2)
    dp = [[0] * (cols + 1) for _ in range(rows + 1)]
    for i in range(rows - 1, -1, -1):
        for j in range(cols - 1, -1, -1):
            if text1[i] == text2[j]:
                dp[i][j] = 1 + dp[i + 1][j + 1]
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j + 1])
    return dp[0][0]
```

17.9 Partition Equal Subset Sum

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/partition-equal-subset-sum/>

Problem statement

Given a non-empty array `nums`, return `true` if the array can be partitioned into two subsets with equal sum.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1,5,11,5] -> True`
- `nums = [1,2,3,5] -> False`

Brute-force: $O(2^n)$ **Best solution:** $O(n * \text{target})$, space $O(\text{target})$ **Brute-force idea**

The brute-force search tries putting each number into the subset you are currently building toward half the total sum.

Brute-force solution

```
def can_partition_bruteforce(nums):
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2

    def dfs(i, current):
        if current == target:
            return True
        if i == len(nums) or current > target:
            return False
        return dfs(i + 1, current + nums[i]) or dfs(i + 1, current)

    return dfs(0, 0)
```

Best solution

The DP view is subset sum: which totals are reachable after processing each number?

Best solution

```
def can_partition_best(nums):
    total = sum(nums)
    if total % 2:
        return False
    target = total // 2
    reachable = {0}
    for num in nums:
        reachable |= {value + num for value in list(reachable) if value + num <= target}
        if target in reachable:
            return True
    return False
```

17.10 Maximum Product Subarray

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/maximum-product-subarray/>

Problem statement

Given an integer array `nums`, find a contiguous non-empty subarray with the largest product.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [2,3,-2,4] -> 6`
- `nums = [-2,0,-1] -> 0`

Brute-force: $O(n^2)$, space $O(1)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

The brute-force baseline multiplies every subarray and tracks the best product seen.

Brute-force solution

```
def max_product_bruteforce(nums):
    best = nums[0]
    for i in range(len(nums)):
        product = 1
        for j in range(i, len(nums)):
            product *= nums[j]
            best = max(best, product)
    return best
```

Best solution

A negative number can turn a tiny minimum into the next maximum, so track both max-ending-here and min-ending-here.

Best solution

```
def max_product_best(nums):
    current_max = current_min = best = nums[0]
    for num in nums[1:]:
```

```
    candidates = (num, current_max * num, current_min * num)
    current_max = max(candidates)
    current_min = min(candidates)
    best = max(best, current_max)
return best
```

Tries and Advanced Structures

Chapter overview

This chapter is where the toolkit gets sharper. Tries help with prefix queries, and union-find gives you a near-magical way to detect connectivity changes.



Mindset for this chapter: start with the honest brute-force idea, then ask what repeated work you can avoid. That question usually points straight at the best solution.

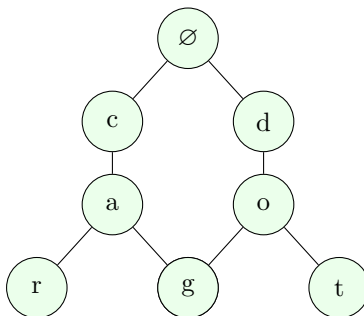
Tries primer

This chapter opening folds in the handbook theory for tries so the workbook stays self-contained. Read this primer first, then move into the interview problems.

A trie is basically a character-by-character decision tree. It is especially useful when lots of words share prefixes.

Trie picture

Shared prefixes are where tries earn their keep.



Minimal trie implementation

Start with insert and search. Once those make sense, prefix matching and autocomplete become much easier to build.

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.end = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.end
```

Segment Trees and Fenwick Trees primer

This chapter opening folds in the handbook theory for segment trees and fenwick trees so the workbook stays self-contained. Read this primer first, then move into the interview problems.

These structures are worth learning when repeated range queries start to make brute force feel painful. You pay some setup cost to make later queries much faster.

When to use them

If you need many range sums, mins, maxes, or updates, specialized range-query structures are often the right move.

Fenwick tree pattern

A Fenwick tree is usually easier to implement than a segment tree, so it is a nice stepping stone.

```
class FenwickTree:
    def __init__(self, n):
        self.tree = [0] * (n + 1)

    def update(self, index, delta):
        index += 1
        while index < len(self.tree):
            self.tree[index] += delta
            index += index & -index
```

```
def query(self, index):
    index += 1
    total = 0
    while index > 0:
        total += self.tree[index]
        index -= index & -index
    return total

def range_sum(self, left, right):
    return self.query(right) - self.query(left - 1)
```

Segment tree intuition

A segment tree breaks the array into nested intervals. That makes queries feel like combining answers from a few relevant pieces instead of scanning everything.

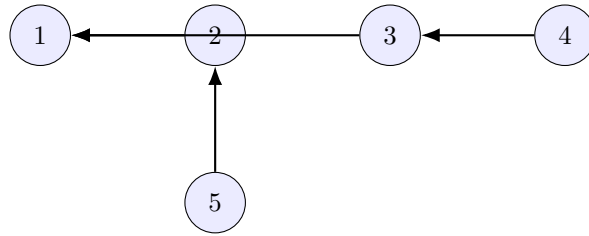
Disjoint Set Union primer

This chapter opening folds in the handbook theory for disjoint set union so the workbook stays self-contained. Read this primer first, then move into the interview problems.

DSU, also called Union-Find, is one of those structures that feels incredibly useful the moment you see the right kind of problem: repeated connectivity checks.

Visual intuition

Each set has a representative. Parent pointers help us find it.



Parent pointers point to the set representative.

Union-Find implementation

Path compression and union by rank are the two classic optimizations.

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True
```

Where it appears

DSU shows up in Kruskal's algorithm, dynamic connectivity, grouping problems, and anytime you need to merge components efficiently.

Advanced Topics primer

This chapter opening folds in the handbook theory for advanced topics so the workbook stays self-contained. Read this primer first, then move into the interview problems.

This final technical chapter is here to help you orient yourself. You do not need to master every advanced algorithm at once. The win is knowing what category of problem each one belongs to.

String algorithms

KMP, Z-algorithm, Rabin-Karp, and Manacher each solve different pattern or palindrome questions more efficiently than brute force.

Graph algorithms

Tarjan helps with strongly connected components. Bellman-Ford handles negative edges. Floyd-Warshall solves all-pairs shortest paths. Bridges and articulation points identify fragile parts of a graph.

Range-query tools

Mo's algorithm, square-root decomposition, Fenwick trees, and segment trees all live in the world of repeated range work with different trade-offs.

Practical advice

Treat advanced topics like a map, not a checklist. First learn what kind of problem each tool solves. Then go deeper only when you need it.

18.1 Implement Trie

Problem statement

Implement a trie with insert, search, and startsWith methods.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- insert('apple'), search('apple') -> True, search('app') -> False, startsWith('app') -> True
- insert('app'), search('app') -> True

Brute-force: insert/search $O(m)$ with linear word scan **Best solution:** insert/search $O(m)$ using prefix tree

Brute-force idea

The brute-force version stores full words in a set and scans them for prefix checks.

Brute-force solution

```
class TrieBruteforce:
    def __init__(self):
        self.words = set()

    def insert(self, word):
        self.words.add(word)

    def search(self, word):
        return word in self.words

    def startsWith(self, prefix):
        for word in self.words:
            if word.startswith(prefix):
                return True
        return False
```

Best solution

A trie shares prefix work across words and keeps the logic clean.

Best solution

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_word = False

class TrieBest:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_word = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_word

    def startsWith(self, prefix):
        node = self.root
        for ch in prefix:
```

```

    if ch not in node.children:
        return False
    node = node.children[ch]
    return True

```

18.2 Design Add and Search Words Data Structure

Problem statement

Design a data structure that supports adding words and searching with `''` as a wildcard matching any single letter.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `addWord('bad')`, `addWord('dad')`, `addWord('mad')`, `search('pad')` -> `False`
- `search('.ad')` -> `True`, `search('b.')` -> `True`

Brute-force: search $O(nm)$ **Best solution:** search $O(m)$ average, exponential only on wildcard branching

Brute-force idea

A direct approach stores all words and tests each one against the pattern.

Brute-force solution

```

class WordDictionaryBruteforce:
    def __init__(self):
        self.words = []

    def addWord(self, word):
        self.words.append(word)

    def search(self, pattern):
        for word in self.words:
            if len(word) != len(pattern):
                continue
            if all(p == '.' or p == w for p, w in zip(pattern, word)):
                return True
        return False

```

Best solution

A trie plus DFS only explores branches that fit the current pattern character.

Best solution

```

class WordNode:
    def __init__(self):
        self.children = {}
        self.is_word = False

class WordDictionaryBest:
    def __init__(self):
        self.root = WordNode()

    def addWord(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = WordNode()
            node = node.children[ch]
        node.is_word = True

    def search(self, word):
        def dfs(node, index):
            if index == len(word):
                return node.is_word

            ch = word[index]
            if ch == '.':
                for child in node.children.values():
                    if dfs(child, index + 1):
                        return True
                return False

            if ch not in node.children:
                return False
            return dfs(node.children[ch], index + 1)

        return dfs(self.root, 0)

```

18.3 Redundant Connection

Problem statement

A tree with n nodes has one extra edge added. Return the edge that can be removed so the graph becomes a tree again.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- edges = [[1, 2], [1, 3], [2, 3]] -> [2, 3]
- edges = [[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]] -> [1, 4]

Brute-force: Time $O(E * (V+E))$, space $O(V+E)$ **Best solution:** Time near $O(E \alpha(V))$, space $O(V)$

Brute-force idea

The brute-force version removes each edge one by one and checks whether the graph becomes a tree.

Brute-force solution

```
from collections import defaultdict, deque

def redundant_connection_bruteforce(edges):
    def is_tree(skip_index):
        graph = defaultdict(list)
        nodes = set()

        for i, (u, v) in enumerate(edges):
            if i == skip_index:
                continue
            graph[u].append(v)
            graph[v].append(u)
            nodes.add(u)
            nodes.add(v)

        start = next(iter(nodes))
        queue = deque([start])
        visited = {start}

        while queue:
            node = queue.popleft()
            for nxt in graph[node]:
                if nxt not in visited:
                    visited.add(nxt)
                    queue.append(nxt)

        return len(visited) == len(nodes) and len(edges) - 1 == len(nodes) - 1

    for i in range(len(edges) - 1, -1, -1):
        if is_tree(i):
            return edges[i]
    return []
```

Best solution

Union-find catches the first edge that tries to connect two nodes already in the same component.

Best solution

```
def redundant_connection_best(edges):
    parent = {}
    rank = {}

    def find(x):
```

```

    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False

    if rank[ra] < rank[rb]:
        parent[ra] = rb
    elif rank[ra] > rank[rb]:
        parent[rb] = ra
    else:
        parent[rb] = ra
        rank[ra] += 1
    return True

for u, v in edges:
    if u not in parent:
        parent[u] = u
        rank[u] = 0
    if v not in parent:
        parent[v] = v
        rank[v] = 0
    if not union(u, v):
        return [u, v]
return []

```

18.4 Word Search II

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/word-search-ii/>

Problem statement

Given an $m \times n$ board of letters and a list of words, return all words from the list that can be formed in the board.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- board = [['o','a','a','n'], ['e','t','a','e'], ['i','h','k','r'], ['i','f','l','v']], words = ['oath','pea','eat','rain'] -> ['eat','oath']
- empty word list -> []

Brute-force: $O(W * mn * 4^L)$
pruning

Best solution: much better in practice with trie

Brute-force idea

A baseline runs the single-word search for every candidate word independently.

Brute-force solution

```
def find_words_bruteforce(board, words):
    rows, cols = len(board), len(board[0])

    def exists(word):
        def dfs(r, c, i, seen):
            if i == len(word):
                return True
            if not (0 <= r < rows and 0 <= c < cols):
                return False
            if (r, c) in seen or board[r][c] != word[i]:
                return False
            seen.add((r, c))
            found = (
                dfs(r + 1, c, i + 1, seen) or
                dfs(r - 1, c, i + 1, seen) or
                dfs(r, c + 1, i + 1, seen) or
                dfs(r, c - 1, i + 1, seen)
            )
            seen.remove((r, c))
            return found

        for r in range(rows):
            for c in range(cols):
                if dfs(r, c, 0, set()):
                    return True
        return False

    return [word for word in words if exists(word)]
```

Best solution

A trie shares prefixes across all words, so one board traversal can explore many candidates at once and prune dead branches early.

Best solution

```
def find_words_best(board, words):
    trie = {}
    for word in words:
        node = trie
        for ch in word:
            node = node.setdefault(ch, {})
        node['$'] = word
```

```

rows, cols = len(board), len(board[0])
found = []

def dfs(r, c, node):
    ch = board[r][c]
    if ch not in node:
        return
    nxt = node[ch]
    word = nxt.pop('$', None)
    if word:
        found.append(word)

    board[r][c] = '#'
    for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != '#':
            dfs(nr, nc, nxt)
    board[r][c] = ch

    if not nxt:
        node.pop(ch)

for r in range(rows):
    for c in range(cols):
        dfs(r, c, trie)

return found

```

18.5 Prefix and Suffix Search

Problem quick facts

Difficulty: Hard

LeetCode: <https://leetcode.com/problems/prefix-and-suffix-search/>

Problem statement

Design a structure that returns the index of the word with the highest index that starts with a given prefix and ends with a given suffix.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- words = ['apple']; f('a','e') -> 0
- f('b','') with no match -> -1

Brute-force: query $O(nL)$ **Best solution:** build $O(nL^2)$, query $O(1)$

Brute-force idea

A basic design scans the words from highest index to lowest and checks prefix and suffix directly.

Brute-force solution

```
class WordFilterBruteforce:
    def __init__(self, words):
        self.words = words

    def f(self, pref, suff):
        for i in range(len(self.words) - 1, -1, -1):
            word = self.words[i]
            if word.startswith(pref) and word.endswith(suff):
                return i
        return -1
```

Best solution

A fast hash-based approach precomputes every prefix/suffix pair and stores the latest index for that pair.

Best solution

```
class WordFilterBest:
    def __init__(self, words):
        self.lookup = {}
        for index, word in enumerate(words):
            prefixes = [word[:i] for i in range(len(word) + 1)]
            suffixes = [word[j:] for j in range(len(word) + 1)]
            for pref in prefixes:
                for suff in suffixes:
                    self.lookup[(pref, suff)] = index

    def f(self, pref, suff):
        return self.lookup.get((pref, suff), -1)
```

18.6 Replace Words

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/replace-words/>

Problem statement

Given a dictionary of roots and a sentence, replace successors in the sentence with the shortest root that forms them.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- dictionary = ['cat','bat','rat'], sentence = 'the cattle was rattled by the battery' -> 'the cat was rat by the bat'
- dictionary = ['a','b','c'], sentence = 'aadsfasf absbs bbab cadsfafs' -> 'a a b c'

Brute-force: $O(D + W * R)$

Best solution: $O(\text{total chars})$, space $O(\text{total chars})$

Brute-force idea

A baseline checks every root against every word and keeps the shortest matching root.

Brute-force solution

```
def replace_words_bruteforce(dictionary, sentence):
    roots = sorted(dictionary, key=len)
    result = []
    for word in sentence.split():
        replacement = word
        for root in roots:
            if word.startswith(root):
                replacement = root
                break
        result.append(replacement)
    return " ".join(result)
```

Best solution

A trie lets each word walk character by character until it either finds a root ending or proves no root applies.

Best solution

```
def replace_words_best(dictionary, sentence):
    trie = {}
    for root in dictionary:
        node = trie
        for ch in root:
            node = node.setdefault(ch, {})
        node['$'] = root
```

```
def replace(word):
    node = trie
    for ch in word:
        if '$' in node:
            return node['$']
        if ch not in node:
            return word
        node = node[ch]
    return node.get('$', word)

return " ".join(replace(word) for word in sentence.split())
```

18.7 Maximum XOR of Two Numbers in an Array

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/maximum-xor-of-two-numbers-in-an-array/>

Problem statement

Given an integer array `nums`, return the maximum result of `nums[i] XOR nums[j]`.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [3,10,5,25,2,8]` -> 28
- `nums = [0]` -> 0

Brute-force: $O(n^2)$, space $O(1)$

Best solution: $O(32n)$, space $O(32n)$

Brute-force idea

The baseline checks every pair and keeps the largest XOR value.

Brute-force solution

```
def find_maximum_xor_bruteforce(nums):
    best = 0
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            best = max(best, nums[i] ^ nums[j])
    return best
```

Best solution

A bitwise trie tries to take the opposite bit at each position, because opposite bits maximize XOR.

Best solution

```
def find_maximum_xor_best(nums):
    trie = {}
    for num in nums:
        node = trie
        for bit in range(31, -1, -1):
            curr = (num >> bit) & 1
            node = node.setdefault(curr, {})

    best = 0
    for num in nums:
        node = trie
        value = 0
        for bit in range(31, -1, -1):
            curr = (num >> bit) & 1
            want = 1 - curr
            if want in node:
                value |= 1 << bit
                node = node[want]
            else:
                node = node[curr]
        best = max(best, value)
    return best
```

Bit Manipulation

Chapter overview

Bit problems often look strange until you translate them into a small identity: XOR cancels pairs, shifting divides the search space, and masks isolate the exact bit you care about. Once you name that identity, the solution usually gets much shorter.

Bit Manipulation primer

This chapter opening folds in the handbook theory for bit manipulation so the workbook stays self-contained. Read this primer first, then move into the interview problems.

Bit tricks seem cryptic at first, but they become much more approachable when you connect them to small, concrete questions like parity, powers of two, and toggling flags.

Core operations

AND keeps common 1-bits. OR turns on any bit present in either value. XOR highlights differences. Shifts move bits left or right.

Helpful mini-patterns

```
def is_even(n):
    return (n & 1) == 0

def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0

def count_set_bits(n):
    count = 0
    while n:
        n &= n - 1
        count += 1
    return count
```

Why XOR matters

XOR is great when pairs cancel each other out. That is why it shows up in single-number problems.

19.1 Number of 1 Bits

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/number-of-1-bits/>

Problem statement

Given a positive integer n , return the number of set bits it has.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $n = 11 \rightarrow 3$
- $n = 128 \rightarrow 1$

Brute-force: $O(32)$, space $O(1)$ **Best solution:** $O(\text{number of set bits})$, space $O(1)$

Brute-force idea

A direct baseline inspects every bit position.

Brute-force solution

```
def hamming_weight_bruteforce(n):
    count = 0
    while n:
        count += n & 1
        n >>= 1
    return count
```

Best solution

Brian Kernighan's trick removes the lowest set bit each time, so the loop runs only once per 1 bit.

Best solution

```
def hamming_weight_best(n):
    count = 0
```

```

while n:
    n &= n - 1
    count += 1
return count

```

19.2 Counting Bits

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/counting-bits/>

Problem statement

Given an integer n , return an array `ans` where `ans[i]` is the number of 1 bits in i for every $0 \leq i \leq n$.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $n = 2 \rightarrow [0, 1, 1]$
- $n = 5 \rightarrow [0, 1, 1, 2, 1, 2]$

Brute-force: $O(n \log n)$, space $O(n)$

Best solution: $O(n)$, space $O(n)$

Brute-force idea

A baseline simply counts the set bits of every number independently.

Brute-force solution

```

def count_bits_bruteforce(n):
    def bits(x):
        count = 0
        while x:
            count += x & 1
            x >>= 1
        return count
    return [bits(i) for i in range(n + 1)]

```

Best solution

The DP recurrence is compact: $\text{bits}[i] = \text{bits}[i \gg 1] + (i \& 1)$.

Best solution

```
def count_bits_best(n):
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        dp[i] = dp[i >> 1] + (i & 1)
    return dp
```

19.3 Missing Number

Problem quick facts

Difficulty: Easy**LeetCode:** <https://leetcode.com/problems/missing-number/>

Problem statement

Given nums containing n distinct numbers in the range $[0, n]$, return the only number missing from the array.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [3,0,1] -> 2`
- `nums = [0,1] -> 2`

Brute-force: $O(n \log n)$, space $O(1)$ **Best solution:** $O(n)$, space $O(1)$

Brute-force idea

Sorting then scanning for the first mismatch is the easiest baseline.

Brute-force solution

```
def missing_number_bruteforce(nums):
    nums.sort()
    for i, num in enumerate(nums):
        if i != num:
            return i
    return len(nums)
```

Best solution

XOR works beautifully because equal values cancel out.

Best solution

```
def missing_number_best(nums):
    answer = len(nums)
    for i, num in enumerate(nums):
        answer ^= i ^ num
    return answer
```

19.4 Reverse Bits

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/reverse-bits/>

Problem statement

Reverse the bits of a given 32-bit unsigned integer.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $n = 43261596 \rightarrow 964176192$
- $n = 0 \rightarrow 0$

Brute-force: $O(32)$, space $O(1)$

Best solution: $O(32)$, space $O(1)$

Brute-force idea

A readable baseline converts to a 32-bit string, reverses it, then converts back.

Brute-force solution

```
def reverse_bits_bruteforce(n):
    bits = bin(n)[2:].zfill(32)
    return int(bits[::-1], 2)
```

Best solution

The bitwise version builds the reversed answer one bit at a time.

Best solution

```
def reverse_bits_best(n):
    answer = 0
    for _ in range(32):
        answer = (answer << 1) | (n & 1)
        n >>= 1
    return answer
```

19.5 Sum of Two Integers

Problem quick facts

Difficulty: Medium**LeetCode:** <https://leetcode.com/problems/sum-of-two-integers/>

Problem statement

Return the sum of two integers without using the + or - operators.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- a = 1, b = 2 -> 3
- a = 2, b = 3 -> 5

Brute-force: $O(|b|)$, space $O(1)$ **Best solution:** $O(1)$ for fixed-width integers

Brute-force idea

A toy baseline can simulate addition by repeatedly incrementing or decrementing.

Brute-force solution

```
def get_sum_bruteforce(a, b):
    while b > 0:
        a += 1
        b -= 1
    while b < 0:
        a -= 1
        b += 1
    return a
```

Best solution

Bitwise addition splits the work into carry and non-carry parts until no carry remains.

Best solution

```
def get_sum_best(a, b):
    MASK = 0xFFFFFFFF
    MAX_INT = 0x7FFFFFFF

    while b != 0:
        a, b = (a ^ b) & MASK, ((a & b) << 1) & MASK

    return a if a <= MAX_INT else ~(a ^ MASK)
```

19.6 Single Number

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/single-number/>

Problem statement

Every element appears twice except for one. Find that single one.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [2,2,1]` -> 1
- `nums = [4,1,2,1,2]` -> 4

Brute-force: $O(n)$, space $O(n)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

A frequency map is the most direct baseline.

Brute-force solution

```
def single_number_bruteforce(nums):
    counts = {}
    for num in nums:
        counts[num] = counts.get(num, 0) + 1
```

```
for num, count in counts.items():
    if count == 1:
        return num
```

Best solution

XOR cancels matching pairs and leaves the unique value behind.

Best solution

```
def single_number_best(nums):
    answer = 0
    for num in nums:
        answer ^= num
    return answer
```

19.7 Single Number III

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/single-number-iii/>

Problem statement

Exactly two elements appear only once and all the other elements appear exactly twice. Return the two single numbers.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- `nums = [1,2,1,3,2,5] -> [3,5]` in any order
- `nums = [-1,0] -> [-1,0]`

Brute-force: $O(n)$, space $O(n)$

Best solution: $O(n)$, space $O(1)$

Brute-force idea

A frequency map can still solve it by collecting the values that appear once.

Brute-force solution

```
def single_number_iii_bruteforce(nums):
    counts = {}
    for num in nums:
        counts[num] = counts.get(num, 0) + 1
    return [num for num, count in counts.items() if count == 1]
```

Best solution

First XOR all numbers to get $x \oplus y$. Then use one differing bit to split the numbers into two groups and recover each singleton.

Best solution

```
def single_number_iii_best(nums):
    xor_all = 0
    for num in nums:
        xor_all ^= num

    diff = xor_all & -xor_all
    a = b = 0
    for num in nums:
        if num & diff:
            a ^= num
        else:
            b ^= num
    return [a, b]
```

Math and Numbers

Chapter overview

This chapter is about staying calm with number mechanics: digit handling, overflow, repeated squaring, counting factors, and simulation shortcuts. Strong interview answers explain the invariant first and only then drop into arithmetic details.

Mathematics for DSA primer

This chapter opening folds in the handbook theory for mathematics for dsa so the workbook stays self-contained. Read this primer first, then move into the interview problems.

You do not need to be a mathematician to do well in DSA, but a few math tools show up everywhere. This chapter focuses on the ones that pay off the fastest.

Prime numbers and divisibility

A prime has exactly two positive divisors: 1 and itself. In practice, the key trick is that you only need to test possible factors up to the square root.

```
def is_prime(n):  
    if n < 2:  
        return False  
    factor = 2  
    while factor * factor <= n:  
        if n % factor == 0:  
            return False  
        factor += 1  
    return True
```

GCD and LCM

The Euclidean algorithm is one of those small ideas that feels magical the first time you use it. Repeated remainders shrink the problem very quickly.

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return a * b // gcd(a, b)
```

Fast exponentiation

Instead of multiplying a number by itself again and again, square when you can. That turns repeated multiplication into a logarithmic process.

```
def fast_pow(base, exp):
    result = 1
    while exp > 0:
        if exp & 1:
            result *= base
        base *= base
        exp >>= 1
    return result
```

Sieve of Eratosthenes

When you need all primes up to n , build them in one pass instead of testing each number independently.

```
def sieve(n):
    if n < 2:
        return []
    prime = [True] * (n + 1)
    prime[0] = prime[1] = False
    p = 2
    while p * p <= n:
        if prime[p]:
            multiple = p * p
            while multiple <= n:
                prime[multiple] = False
                multiple += p
        p += 1
    return [i for i, ok in enumerate(prime) if ok]
```

20.1 Count Primes

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/count-primes/>

Problem statement

Count the number of prime numbers strictly less than n .

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $n = 10 \rightarrow 4$
- $n = 0 \rightarrow 0$

Brute-force: $O(n \sqrt{n})$, space $O(1)$ **Best solution:** $O(n \log \log n)$, space $O(n)$

Brute-force idea

A baseline tests each number for primality by trial division.

Brute-force solution

```
def count_primes_bruteforce(n):
    def is_prime(x):
        if x < 2:
            return False
        d = 2
        while d * d <= x:
            if x % d == 0:
                return False
            d += 1
        return True
    return sum(1 for x in range(n) if is_prime(x))
```

Best solution

The Sieve of Eratosthenes marks multiples in bulk instead of primality-testing each number from scratch.

Best solution

```
def count_primes_best(n):
    if n < 2:
        return 0
    prime = [True] * n
    prime[0] = prime[1] = False
    p = 2
    while p * p < n:
        if prime[p]:
            for multiple in range(p * p, n, p):
                prime[multiple] = False
        p += 1
    return sum(prime)
```

20.2 Pow(x, n)

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/powx-n/>

Problem statement

Implement $\text{pow}(x, n)$, which computes x raised to the power n .

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $x = 2.0, n = 10 \rightarrow 1024.0$
- $x = 2.0, n = -2 \rightarrow 0.25$

Brute-force: $O(|n|)$, space $O(1)$

Best solution: $O(\log |n|)$, space $O(\log |n|)$

Brute-force idea

The straightforward baseline multiplies x by itself $|n|$ times.

Brute-force solution

```
def my_pow_bruteforce(x, n):
    if n == 0:
        return 1.0
    power = abs(n)
    answer = 1.0
    for _ in range(power):
        answer *= x
    return answer if n > 0 else 1 / answer
```

Best solution

Fast power squares the base each round and uses the current bit of the exponent to decide whether to multiply it into the answer.

Best solution

```
def my_pow_best(x, n):
    def fast_pow(base, exp):
        if exp == 0:
```

```

        return 1.0
    half = fast_pow(base, exp // 2)
    if exp % 2 == 0:
        return half * half
    return half * half * base

if n < 0:
    x = 1 / x
    n = -n
return fast_pow(x, n)

```

20.3 Multiply Strings

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/multiply-strings/>

Problem statement

Given two non-negative integers num1 and num2 represented as strings, return their product also as a string.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- num1 = '2', num2 = '3' -> '6'
- num1 = '123', num2 = '456' -> '56088'

Brute-force: $O(1)$ in code but not interview-valid
 $O(m+n)$

Best solution: $O(mn)$, space

Brute-force idea

A baseline can rely on Python's big integers, but that dodges the real interview exercise.

Brute-force solution

```

def multiply_strings_bruteforce(num1, num2):
    return str(int(num1) * int(num2))

```

Best solution

The standard grade-school multiplication stores partial sums in an array of digits.

Best solution

```
def multiply_strings_best(num1, num2):
    if num1 == "0" or num2 == "0":
        return "0"
    result = [0] * (len(num1) + len(num2))
    for i in range(len(num1) - 1, -1, -1):
        for j in range(len(num2) - 1, -1, -1):
            product = (ord(num1[i]) - 48) * (ord(num2[j]) - 48)
            pos_low = i + j + 1
            pos_high = i + j
            total = product + result[pos_low]
            result[pos_low] = total % 10
            result[pos_high] += total // 10
    start = 0
    while start < len(result) and result[start] == 0:
        start += 1
    return "".join(str(d) for d in result[start:])
```

20.4 Reverse Integer

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/reverse-integer/>

Problem statement

Given a signed 32-bit integer x , return x with its digits reversed. If reversing x causes overflow, return 0.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $x = 123 \rightarrow 321$
- $x = -123 \rightarrow -321$

Brute-force: $O(d)$, space $O(d)$

Best solution: $O(d)$, space $O(1)$

Brute-force idea

A baseline can convert to a string, reverse the digits, and restore the sign.

Brute-force solution

```
def reverse_integer_bruteforce(x):
    sign = -1 if x < 0 else 1
    value = int(str(abs(x))[::-1])
    value *= sign
    return value if -(2 ** 31) <= value <= 2 ** 31 - 1 else 0
```

Best solution

The arithmetic version pops digits from the right and pushes them into the reversed number while checking overflow.

Best solution

```
def reverse_integer_best(x):
    sign = -1 if x < 0 else 1
    x = abs(x)
    result = 0
    while x:
        result = result * 10 + (x % 10)
        x //= 10
    result *= sign
    return result if -(2 ** 31) <= result <= 2 ** 31 - 1 else 0
```

20.5 Plus One

Problem quick facts

Difficulty: Easy

LeetCode: <https://leetcode.com/problems/plus-one/>

Problem statement

You are given a large integer represented as an array of digits. Increment the integer by one and return the resulting array.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- digits = [1,2,3] -> [1,2,4]
- digits = [9,9,9] -> [1,0,0,0]

Brute-force: $O(n)$, space $O(n)$ **Best solution:** $O(n)$, space $O(1)$ or $O(n)$ for output

Brute-force idea

A baseline can join the digits into an integer, add one, and split the result back into digits.

Brute-force solution

```
def plus_one_bruteforce(digits):
    return [int(ch) for ch in str(int("".join(map(str, digits))) + 1)]
```

Best solution

The in-place digit walk handles carries from right to left without converting the whole number.

Best solution

```
def plus_one_best(digits):
    for i in range(len(digits) - 1, -1, -1):
        if digits[i] < 9:
            digits[i] += 1
            return digits
        digits[i] = 0
    return [1] + digits
```

20.6 Divide Two Integers

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/divide-two-integers/>

Problem statement

Given two integers dividend and divisor, divide them without using multiplication, division, or mod. Truncate toward zero.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- dividend = 10, divisor = 3 -> 3
- dividend = 7, divisor = -3 -> -2

Brute-force: $O(|\text{quotient}|)$ **Best solution:** $O(\log |\text{quotient}|)$, space $O(1)$

Brute-force idea

A brute-force baseline repeatedly subtracts the divisor until the dividend is too small.

Brute-force solution

```
def divide_bruteforce(dividend, divisor):
    sign = -1 if (dividend < 0) ^ (divisor < 0) else 1
    dividend, divisor = abs(dividend), abs(divisor)
    quotient = 0
    while dividend >= divisor:
        dividend -= divisor
        quotient += 1
    quotient *= sign
    return min(max(-(2 ** 31), quotient), 2 ** 31 - 1)
```

Best solution

The faster method subtracts doubled divisors (powers of two multiples) instead of one divisor at a time.

Best solution

```
def divide_best(dividend, divisor):
    INT_MAX = 2 ** 31 - 1
    INT_MIN = -(2 ** 31)
    if dividend == INT_MIN and divisor == -1:
        return INT_MAX

    sign = -1 if (dividend < 0) ^ (divisor < 0) else 1
    dividend, divisor = abs(dividend), abs(divisor)
    quotient = 0

    while dividend >= divisor:
        temp = divisor
        multiple = 1
        while dividend >= (temp << 1):
            temp <<= 1
            multiple <<= 1
        dividend -= temp
        quotient += multiple

    return sign * quotient
```

20.7 Factorial Trailing Zeroes

Problem quick facts

Difficulty: Medium

LeetCode: <https://leetcode.com/problems/factorial-trailing-zeroes/>

Problem statement

Given an integer n , return the number of trailing zeroes in $n!$.

Interview focus: clarify the expected return format, key constraints, and the edge cases you plan to handle before you start coding.

Test cases

- $n = 3 \rightarrow 0$
- $n = 30 \rightarrow 7$

Brute-force: $O(n)$

Best solution: $O(\log_5 n)$, space $O(1)$

Brute-force idea

A baseline counts how many times factors of 10 appear by explicitly accumulating the factor counts from 1 to n .

Brute-force solution

```
def trailing_zeroes_bruteforce(n):
    count = 0
    for value in range(1, n + 1):
        while value % 5 == 0:
            count += 1
            value //= 5
    return count
```

Best solution

Trailing zeros come from pairs of 2 and 5, and 5s are the limiting factor. So count how many factors of 5 appear in $n!$.

Best solution

```
def trailing_zeroes_best(n):
    count = 0
    while n:
```

```
    n //= 5
    count += n
return count
```

Common Patterns and Templates

A lot of interview progress comes from recognizing the same skeleton under different wording. That is why reusable templates matter.

21.1 Sliding window template

```
def variable_window(nums):
    left = 0
    state = {}
    best = 0

    for right, value in enumerate(nums):
        # add nums[right] into the state
        while False:  # replace with 'while window is invalid'
            # remove nums[left] from the state
            left += 1
        best = max(best, right - left + 1)

    return best
```

21.2 DFS template

```
def dfs(node, graph, visited):
    visited.add(node)
    for nei in graph[node]:
        if nei not in visited:
            dfs(nei, graph, visited)
```

21.3 Binary search on answer template

```
def binary_search_answer(low, high, works):
    answer = high
    while low <= high:
        mid = (low + high) // 2
        if works(mid):
```

```
        answer = mid
        high = mid - 1
    else:
        low = mid + 1
return answer
```

Common Beginner Mistakes

Everyone makes these. The goal is not to avoid every mistake forever. The goal is to catch them earlier and recover faster.

22.1 Edge cases

Empty input, one-element input, duplicate values, negative numbers, and already-sorted input are worth checking before you celebrate.

22.2 Off-by-one errors

Inclusive and exclusive boundaries cause a lot of quiet bugs. When in doubt, write down what left and right mean.

22.3 Using the wrong structure

If you need fast membership tests, a set might help. If you need repeated min or max access, a heap might help. If you need order, a list alone may not be enough.

22.4 A calm debugging checklist

- Restate what each variable means.
- Walk through the smallest non-trivial example by hand.
- Check loop boundaries carefully.
- Print or log intermediate state when the logic gets slippery.
- Only optimize after the correct version works.

Closing note

This combined edition keeps the workbook structure, then folds in the handbook theory so each topic starts with a clearer mental model before the interview problems begin. Use the chapter primers for understanding, then use the problems for recall, speed, and interview communication.